INDIAN INSTITUTE OF INFORMATION TECHNOLOGY KALYANI

Department of Computer Science and Engineering

Software Engineering – Project Assignment

MediFlow

Hospital Management System

Assignment 1: Hospital Management System

Software Requirements Specification · Software Design Document (UML) · Software Test Plan · Software Test Report

GROUP 11

Sl.	Name of the member	Registration number
1	Akash Chakraborty	CSE/23011/1059
2	Anipriyo Mandal	CSE/23017/1065
3	Arpan Mandal	CSE/23027/1075
4	Himanshu Patel	CSE/23052/1100
5	Pranjali Sirari	CSE/23072/1120

Submitted through Google Classroom · October 2026

Group Details

Table 1: Group and assignment details

Field	Value
Group number	11
Assignment number and title	Assignment 1 – Hospital Management System
Project name	MediFlow – Hospital Management System
Problem statement (brief)	Develop a system to manage patients, doctors, appointments, prescriptions, billing and medical records.
Deliverables	SRS, SDD with UML diagrams, Software Test Plan and Test Report (this document) + working prototype (source code)

Table 2: Group members

Sl.	Name	Registration number
1	Akash Chakraborty	CSE/23011/1059
2	Anipriyo Mandal	CSE/23017/1065
3	Arpan Mandal	CSE/23027/1075
4	Himanshu Patel	CSE/23052/1100
5	Pranjali Sirari	CSE/23072/1120

Document control

Table 3: Revision history

Version	Date	Description
0.1	15-Oct-2026	Project selected (Assignment 1 – Hospital Management System)
0.2	18-Oct-2026	Scope, user roles and feature set agreed (core, hospital operations, distinctive features)
0.5	21-Oct-2026	Requirements baselined (50 FR, 10 NFR, 12 BR); architecture and UML diagrams
0.9	21-Oct-2026	Prototype implemented; unit, integration, system and acceptance tests executed
0.95	22-Oct-2026	Robustness / edge-case testing; additional defects fixed and regression-tested
1.0	22-Oct-2026	Final report for submission

Summary

MediFlow is a web-based Hospital Management System that manages **patients, doctors, appointments, prescriptions, billing and medical records**, extended with laboratory, pharmacy and in-patient (bed and admission) management. Beyond the core requirements, it introduces a **smart queue** with tokens and waiting-time estimates, a **live bed board** with a discharge workflow tied to billing clearance, and **role-based access control with an append-only audit trail** for six staff roles.

This report contains the **Software Requirements Specification** (Part A: 50 functional and 10 non-functional requirements, 12 business rules, 14 user requirements), the **Software Design Document** (Part B: layered architecture, 13 modules and 16 UML diagrams – use case, class, sequence, state, activity, component and deployment – plus a logical data model), the **Software Test Plan** (Part C) and the **Software Test Report** (Part D). A working prototype (React + Node.js + SQLite) was built so that the test report contains real results: **161 of 161 automated test cases passed** in the final run, and 33 defects found during testing are documented with their resolution.

> How to run the prototype: install Node.js 20+, then in the project folder run `npm install` and `npm run dev`, and open http://localhost:5173. Demo accounts (password Mediflow@123): admin, reception, dr.sharma, lab, pharmacy, billing. See Appendix B.

Table of Contents

Group Details.....2
Document control.....2
Summary.....3
Table of Contents.....4
PART A – SOFTWARE REQUIREMENTS SPECIFICATION (SRS).....6
1. Introduction.....6
1.1 Purpose.....6
1.2 Intended audience.....6
1.3 Product overview.....6
1.4 Definitions, acronyms and abbreviations.....6
1.5 References.....7
1.6 Document overview.....7
2. Problem Statement.....7
3. Objectives.....7
4. Scope of the System.....8
4.1 In scope.....8
4.2 Out of scope (this release).....8
4.3 User classes.....8
5. Functional Requirements.....9
5.1 Business rules.....9
5.2 Authentication & Access Control (AUTH).....10
5.3 User Administration (USR).....10
5.4 Patient Management (PAT).....10
5.5 Doctor & Department Management (DOC).....11
5.6 Appointment Management (APT).....11
5.7 Smart Queue Management (QUE).....11
5.8 Consultation & Medical Records (CON).....12
5.9 Laboratory Management (LAB).....12
5.10 Pharmacy & Inventory (PHA).....12
5.11 Admission, Bed & Discharge Management (IPD).....12
5.12 Billing & Payments (BIL).....13
5.13 Dashboard & Reports (RPT).....13
5.14 Audit Trail (AUD).....13
6. Non-functional Requirements.....13
7. User Requirements.....14
8. System Requirements.....15
8.1 System features (stimulus → response).....15
8.2 External interface requirements.....16
8.3 Data requirements.....18
9. Hardware and Software Requirements.....19
9.1 Hardware requirements.....19
9.2 Software requirements.....19
10. Assumptions and Constraints.....19
10.1 Assumptions.....19
10.2 Constraints.....20
10.3 Dependencies.....20
PART B – SOFTWARE DESIGN DOCUMENT (SDD).....21

1. System Architecture..... 21
1.1 Design decisions and rationale..... 22
1.2 Deployment view..... 23
2. Module / Component Description..... 23
2.1 Cross-cutting components..... 24
3. Database Design (optional)..... 24
4. Class Diagram..... 25
5. Use Case Diagram..... 28
5.1 Use case specifications..... 30
6. Sequence Diagrams..... 32
7. State Diagrams..... 36
7.1 Activity view of the outpatient journey..... 38
8. Security Design..... 39
9. User Interface Design..... 40
PART C – SOFTWARE TEST PLAN..... 48
1. Testing Objectives..... 48
2. Testing Scope..... 48
2.1 In scope..... 48
2.2 Out of scope..... 48
3. Testing Strategy..... 48
3.1 Test design techniques..... 49
3.2 Test environment..... 49
3.3 Entry and exit criteria..... 50
3.4 Defect management..... 50
3.5 Roles, schedule and deliverables..... 50
3.6 Risks and mitigations..... 50
4. Unit Testing..... 51
5. Integration Testing..... 51
6. System Testing..... 51
7. Acceptance Testing..... 52
PART D – SOFTWARE TEST REPORT..... 53
8. Test Cases..... 53
8.1 Unit test cases..... 53
8.2 Integration test cases..... 57
8.3 System test cases..... 64
8.4 Acceptance test cases..... 66
9. Expected Results..... 67
10. Actual Results..... 67
10.1 Execution history..... 67
10.2 Code coverage..... 68
10.3 Non-functional results..... 68
11. Pass/Fail Status..... 68
12. Defect Reporting..... 69
12.1 Detailed defect reports..... 71
12.2 Defect summary and analysis..... 79
13. Conclusion..... 79
APPENDIX A – REQUIREMENTS TRACEABILITY MATRIX..... 80
APPENDIX B – RUNNING THE PROTOTYPE..... 83

PART A – SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

> This SRS follows the structure required by the assignment and is informed by IEEE Std 830-1998 / ISO/IEC/IEEE 29148:2018. Every requirement has a unique identifier (FR-xx, NFR-xx, BR-xx, UR-xx) that is traced to design elements and test cases in Part D and Appendix A.

1. Introduction

1.1 Purpose

This document specifies the requirements of **MediFlow**, a web-based Hospital Management System (HMS) developed for Assignment 1 of the Software Engineering course. It describes what the system must do (functional requirements), how well it must do it (non-functional requirements), who will use it and under which assumptions and constraints. It is the baseline for the Software Design Document (Part B) and the Software Test Plan and Report (Parts C and D).

1.2 Intended audience

- **Course instructor and evaluators** – to assess completeness and quality of the specification.
- **Development team (Group 11)** – as the single source of truth for design, implementation and testing.
- **Hospital stakeholders (future)** – administrators, doctors, front-desk, laboratory, pharmacy and billing staff who would validate the requirements.

1.3 Product overview

MediFlow connects the reception desk, doctors, laboratory, pharmacy, wards and billing through one shared patient record. A patient's journey – **registration → appointment → check-in and queue token → consultation → prescription / lab tests → billing → payment** (and for in-patients **admission → bed allocation → transfer → discharge**) – flows through the system without re-entering data. Three features distinguish the product from a basic HMS: a **smart queue** with tokens and waiting-time estimates, a **live bed board** with a safe admission/discharge workflow tied to billing clearance, and **role-based access control with an append-only audit trail**.

1.4 Definitions, acronyms and abbreviations

Table 19: Definitions and acronyms

Term	Meaning
HMS	Hospital Management System
OPD / IPD	Out-patient department (visits without admission) / In-patient department (admitted patients)
RBAC	Role-Based Access Control – permissions are granted to roles, users get one role
Token	Sequential queue number issued to a patient at check-in, unique per doctor per day
ETA	Estimated waiting time until the patient is called
STAT	Urgent laboratory priority (processed before ROUTINE orders)
Charge	A billable event (consultation fee, lab test, medicine, bed-day) waiting to be invoiced
Billing clearance	Condition that all charges of an admission are invoiced and fully paid
JWT	JSON Web Token – signed session token used to authenticate API calls
SPA / REST	Single-Page Application / Representational State Transfer (HTTP + JSON API)
FR / NFR / BR / UR	Functional / Non-functional requirement, Business rule, User requirement

1.5 References

- IEEE Std 830-1998, *Recommended Practice for Software Requirements Specifications*; ISO/IEC/IEEE 29148:2018, *Requirements engineering*.
- IEEE Std 1016-2009, *Software Design Descriptions*; IEEE Std 829-2008 / ISO/IEC/IEEE 29119-3, *Software test documentation*.
- OMG Unified Modeling Language (UML) 2.5.1 specification.
- Assignment brief: "Software Development project problems – 1. Hospital Management System", Software Engineering course, 2026.

1.6 Document overview

Section 2 states the problem, Section 3 the objectives and Section 4 the scope. Sections 5 and 6 list functional and non-functional requirements, Section 7 the user requirements, Section 8 the detailed system requirements (features, interfaces and data), Section 9 the hardware and software requirements, and Section 10 the assumptions and constraints.

2. Problem Statement

Many small and mid-sized hospitals still run their front desk, wards and billing on paper registers and disconnected spreadsheets. This causes recurring problems:

- **Duplicate and incomplete patient records** – the same patient is registered several times, so history, allergies and previous results are not available to the doctor.
- **Double-booked doctors and long, unpredictable waits** – appointments are written in diaries, walk-ins and booked patients compete, and nobody can tell a patient how long they will wait.
- **Lost or illegible clinical information** – prescriptions and lab orders on paper are misread, results arrive late and abnormal values are not highlighted.
- **Bed confusion** – the front desk does not know which beds are free, being cleaned or under maintenance; occasionally a bed is promised to two patients.
- **Revenue leakage** – consultation fees, tests, medicines and bed-days are recorded in different places, invoices are compiled by hand, and patients are sometimes discharged with unpaid bills.
- **No accountability** – it is impossible to tell who changed a record, cancelled an appointment or authorised a discharge.

The hospital therefore needs a single, secure system that manages **patients, doctors, appointments, prescriptions, billing and medical records** (as required by the assignment) as one continuous workflow, with the right information available to the right staff member at the right time.

3. Objectives

Table 20: Project objectives

ID	Objective	Measure of success
O1	Maintain one permanent electronic record per patient with complete visit, clinical, laboratory, admission and billing history.	Unique patient ID; duplicate warning (FR-07 – FR-12).
O2	Eliminate double-booking and make waiting times predictable.	Zero overlapping bookings; token + ETA for every checked-in patient (FR-15 – FR-25).
O3	Digitise consultations, prescriptions and lab orders and flag abnormal results automatically.	Diagnosis mandatory; abnormal flag on out-of-range results (FR-26 – FR-34).

ID	Objective	Measure of success
O4	Control medicine stock and prevent dispensing of expired or unavailable drugs.	Dispensing blocked when short/expired; alerts for low/expiring stock (FR-35 – FR-37).
O5	Give real-time bed visibility and a safe admission → transfer → discharge workflow.	No bed double-allocation; discharge only after billing clearance (FR-38 – FR-43).
O6	Capture every billable event automatically and produce accurate invoices and receipts.	Invoices built from charges; overpayment refused (FR-44 – FR-47).
O7	Protect patient data and make every action traceable.	RBAC on every API; lock-out; append-only audit log (FR-01 – FR-06, FR-50).
O8	Provide managers with live KPIs and period reports.	Role dashboards and admin reports (FR-48, FR-49).

4. Scope of the System

4.1 In scope

- **Patient management** – registration with unique ID, validation, duplicate detection, search, update and full history.
- **Doctor and department management** – profiles, fees, working days/hours, slot length and average consultation time.
- **Appointments** – free-slot calculation, booking, rescheduling, cancellation and no-show handling.
- **Smart queue** – check-in tokens, live queue board, waiting-time estimates, urgent prioritisation.
- **Consultation and medical records** – vitals, symptoms, diagnosis, notes, follow-up, e-prescriptions, lab orders.
- **Laboratory** – worklist, sample collection, result entry and automatic abnormal flagging.
- **Pharmacy** – inventory with batch/expiry, restocking, low-stock and expiry alerts, dispensing.
- **In-patient management** – wards and beds, bed board, admission, transfer, discharge request and discharge with billing clearance.
- **Billing** – automatic charges, invoices, discounts, part payments (cash, card, UPI), receipts and printing.
- **Administration** – users and roles, dashboards, management reports and audit trail.

4.2 Out of scope (this release)

- Patient self-service portal and online appointment booking by patients.
- Online payment gateway and insurance / TPA claim processing (payments are recorded, not processed).
- Integration with laboratory analysers, PACS / radiology imaging and external pharmacies.
- SMS / e-mail notifications, staff rostering and payroll, operation-theatre scheduling, inventory purchasing.
- AI-based features (e.g. clinical note summarisation) – excluded for privacy and reliability reasons.

4.3 User classes

Table 21: User classes (actors)

Actor	Description and main responsibilities
Administrator	Manages users, doctors, departments, wards/beds and system configuration; views reports and the audit trail; may approve discharge overrides.
Receptionist	Registers patients, books/reschedules/cancels appointments, checks patients in (queue tokens), admits patients, allocates/transfers beds and completes discharge.

Actor	Description and main responsibilities
Doctor	Works through own queue, records consultations, writes prescriptions, orders lab tests, views patient history and lab results, requests admission/discharge.
Lab Technician	Processes lab test orders: records sample collection and enters results.
Pharmacist	Maintains medicine inventory, receives stock alerts and dispenses prescriptions.
Billing Staff	Generates invoices from unbilled charges, applies discounts, records payments and prints receipts; clears patients for discharge.

Patients are **external** to the system: they do not log in; their data is entered and used by staff.

Use Case Diagram 1 - System overview (context level)

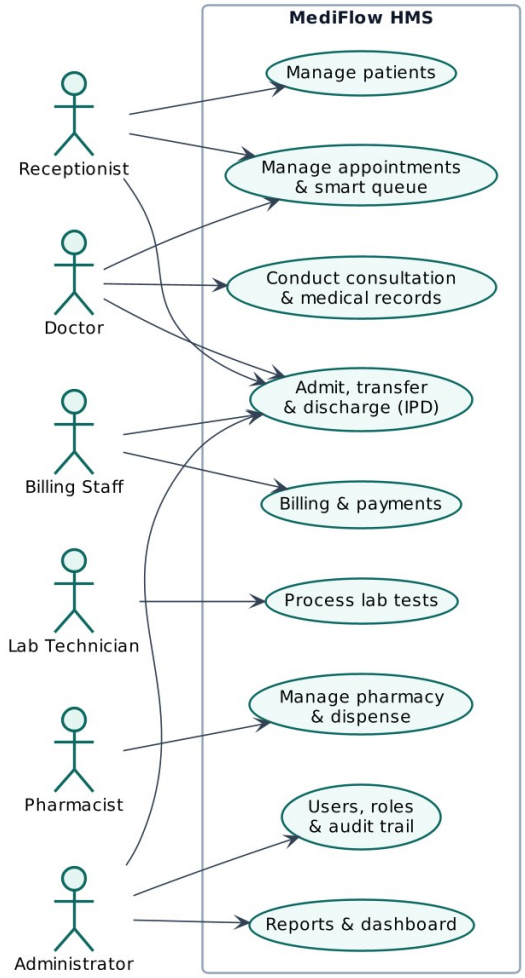


Figure 1: System context – actors and major functions

5. Functional Requirements

The system shall satisfy the 50 functional requirements below, grouped into 13 modules. Priority follows MoSCoW-style ranking: **High** = must have for the first release, **Medium** = should have, **Low** = could have. All requirements were implemented in the prototype.

5.1 Business rules

Table 22: Business rules referenced by the functional requirements

ID	Business rule
BR-01	Patient ID format is P followed by 5 digits, assigned sequentially and never reused.

ID	Business rule
BR-02	Appointment slots start at the doctor's work start time and repeat every slot length until work end; bookings must align to a slot boundary.
BR-03	Active appointment states are SCHEDULED, CHECKED*IN and IN*CONSULTATION; only these block a slot.
BR-04	Appointment life cycle: SCHEDULED → CHECKED*IN* → *IN*CONSULTATION → COMPLETED; SCHEDULED → CANCELLED; SCHEDULED → NO_SHOW.
BR-05	Queue order: URGENT before NORMAL, then earlier check-in first. Wait estimate = ahead × average consultation minutes + max(average − elapsed, 1) when a consultation is in progress.
BR-06	Lab order life cycle: ORDERED → SAMPLE_COLLECTED → COMPLETED; ORDERED → CANCELLED.
BR-07	Low stock when quantity ≤ reorder level; near-expiry when expiry is within 30 days; expired medicines cannot be dispensed.
BR-08	Bed state machine: AVAILABLE→RESERVED, AVAILABLE→OCCUPIED, AVAILABLE→MAINTENANCE, RESERVED→OCCUPIED, RESERVED→AVAILABLE, OCCUPIED→CLEANING, CLEANING→AVAILABLE, CLEANING→MAINTENANCE, MAINTENANCE→AVAILABLE.
BR-09	Admission life cycle: ADMITTED → DISCHARGE_PENDING → DISCHARGED. Bed days = ceil(hours stayed / 24), minimum 1, charged at the ward's daily rate per stay.
BR-10	Invoice life cycle: UNPAID → PARTIALLY_PAID → PAID; UNPAID → CANCELLED (only when no payment exists). Discount 0–50% only while UNPAID.
BR-11	Payments: amount > 0 and ≤ outstanding balance; methods CASH, CARD, UPI; receipt number R-YYYYMMDD-NNNN.
BR-12	Account lockout: 5 consecutive failures lock the account for 15 minutes; a successful login resets the counter.

5.2 Authentication & Access Control (AUTH)

Table 6: Functional requirements – Authentication & Access Control

ID	Requirement	Priority
FR-01	The system shall authenticate users with a username and password and issue a session token valid for 8 hours.	High
FR-02	The system shall lock an account for 15 minutes after 5 consecutive failed login attempts and shall reject logins for deactivated accounts.	High
FR-03	The system shall restrict every operation according to the role-permission matrix (Administrator, Receptionist, Doctor, Lab Technician, Pharmacist, Billing Staff) and show each role only its permitted menus.	High
FR-04	The system shall allow a logged-in user to log out and shall reject requests carrying a missing, invalid or expired token.	Medium

5.3 User Administration (USR)

Table 7: Functional requirements – User Administration

ID	Requirement	Priority
FR-05	The Administrator shall be able to create user accounts with a unique username, full name, role and initial password (minimum 8 characters).	High
FR-06	The Administrator shall be able to activate/deactivate user accounts and reset passwords; an Administrator cannot deactivate their own account.	Medium

5.4 Patient Management (PAT)

Table 8: Functional requirements – Patient Management

ID	Requirement	Priority
FR-07	The Receptionist shall be able to register a patient; the system shall generate a unique patient ID in the format P00001.	High

ID	Requirement	Priority
FR-08	The system shall validate patient data: first and last name required, 10-digit mobile number, date of birth not in the future, valid gender and blood group values.	High
FR-09	The system shall warn about a possible duplicate registration when a patient with the same mobile number and date of birth, or the same full name and date of birth, already exists; the Receptionist may confirm and proceed.	Medium
FR-10	Authorised users shall be able to search patients by patient ID, name or mobile number.	High
FR-11	The Receptionist shall be able to update patient demographic and contact details; the patient ID shall never change.	Medium
FR-12	The system shall display a patient's history: appointments, consultations, prescriptions, lab results, admissions and invoices (clinical items only to clinical roles).	High

5.5 Doctor & Department Management (DOC)

Table 9: Functional requirements – Doctor & Department Management

ID	Requirement	Priority
FR-13	The Administrator shall be able to manage doctor profiles: department, specialisation, qualification, consultation fee, working days and hours, slot length and average consultation time.	High
FR-14	The system shall list departments and the doctors belonging to each department.	Low

5.6 Appointment Management (APT)

Table 10: Functional requirements – Appointment Management

ID	Requirement	Priority
FR-15	The system shall show the free appointment slots of a doctor for a selected date, based on working days, working hours, slot length and existing bookings.	High
FR-16	The Receptionist shall be able to book an appointment for a registered patient with a doctor in a free slot that is not in the past and lies within the doctor's working hours.	High
FR-17	The system shall reject a booking that overlaps an existing active appointment of the same doctor or of the same patient.	High
FR-18	The Receptionist shall be able to reschedule an appointment while it is in the SCHEDULED state, subject to the same slot rules.	Medium
FR-19	The Receptionist shall be able to cancel a SCHEDULED appointment with a mandatory reason; the freed slot becomes bookable again.	Medium
FR-20	The system shall list appointments filtered by date, doctor and status.	Medium

5.7 Smart Queue Management (QUE)

Table 11: Functional requirements – Smart Queue Management

ID	Requirement	Priority
FR-21	On check-in of a SCHEDULED appointment for today, the system shall issue a queue token unique per doctor per day and set the appointment state to CHECKED_IN.	High
FR-22	The system shall display a live queue board per doctor showing waiting, in-consultation and served patients, ordered by priority (URGENT first) and then check-in time.	High
FR-23	The system shall estimate each waiting patient's waiting time as (patients ahead × doctor's average consultation time) + remaining time of the consultation in progress.	High
FR-24	Receptionists, Doctors and Administrators shall be able to mark a waiting patient URGENT with a mandatory reason; the action shall be audited.	Medium
FR-25	A Doctor shall be able to call the next patient from their queue only when no consultation of theirs is in progress; the appointment moves to IN_CONSULTATION.	High

5.8 Consultation & Medical Records (CON)

Table 12: Functional requirements – Consultation & Medical Records

ID	Requirement	Priority
FR-26	A Doctor shall be able to record a consultation for an appointment in IN_CONSULTATION: symptoms, vitals, diagnosis, clinical notes and optional follow-up date; diagnosis is required to complete it.	High
FR-27	A Doctor shall be able to issue a prescription of one or more medicines with dosage, frequency, duration and quantity (> 0) during a consultation.	High
FR-28	A Doctor shall be able to order one or more lab tests for a patient during a consultation or an admission.	High
FR-29	Completing a consultation shall set the appointment to COMPLETED and add the doctor's consultation fee as an unbilled charge.	High
FR-30	Only clinical roles (Doctor, Administrator) shall view diagnoses and clinical notes; Receptionist and Billing Staff shall be denied.	High

5.9 Laboratory Management (LAB)

Table 13: Functional requirements – Laboratory Management

ID	Requirement	Priority
FR-31	The Lab Technician shall see a worklist of lab orders filtered by status (ORDERED, SAMPLE_COLLECTED, COMPLETED, CANCELLED).	High
FR-32	The Lab Technician shall record sample collection (ORDERED → SAMPLE*COLLECTED) and then enter the result (SAMPLE*COLLECTED → COMPLETED); other transitions shall be rejected.	High
FR-33	The system shall automatically flag a numeric result outside the test's reference range as ABNORMAL.	Medium
FR-34	Ordering a lab test shall add the test price as an unbilled charge; cancelling an ORDERED test shall remove the unbilled charge.	Medium

5.10 Pharmacy & Inventory (PHA)

Table 14: Functional requirements – Pharmacy & Inventory

ID	Requirement	Priority
FR-35	The Pharmacist shall manage the medicine inventory: add medicines, restock quantities and record batch and expiry date.	High
FR-36	The system shall raise alerts for medicines at or below their reorder level and for medicines expiring within 30 days or already expired.	Medium
FR-37	The Pharmacist shall dispense a PENDING prescription only if every item is in stock and not expired; dispensing reduces stock and adds medicine charges.	High

5.11 Admission, Bed & Discharge Management (IPD)

Table 15: Functional requirements – Admission, Bed & Discharge Management

ID	Requirement	Priority
FR-38	The system shall show a ward-wise bed board with each bed's status: AVAILABLE, RESERVED, OCCUPIED, CLEANING or MAINTENANCE.	High
FR-39	The Receptionist shall admit a patient to an AVAILABLE bed; a patient may have only one active admission and a bed may hold only one patient.	High
FR-40	The Receptionist shall transfer an admitted patient to another AVAILABLE bed; the previous bed moves to CLEANING and bed charges are kept per stay.	Medium

ID	Requirement	Priority
FR-41	Bed status changes shall follow the bed state machine (AVAILABLE→RESERVED→OCCUPIED→CLEANING→AVAILABLE, AVAILABLE ↔ MAINTENANCE, RESERVED→AVAILABLE); invalid transitions shall be rejected.	High
FR-42	A Doctor shall request discharge with a discharge summary (ADMITTED → DISCHARGE_PENDING); bed charges (days × daily rate, minimum 1 day) are then added as unbilled charges.	High
FR-43	Discharge shall be completed only when all charges of the admission are invoiced and fully paid, unless an Administrator overrides with a recorded reason; the bed then moves to CLEANING.	High

5.12 Billing & Payments (BIL)

Table 16: Functional requirements – Billing & Payments

ID	Requirement	Priority
FR-44	Billing Staff shall generate an invoice for a patient from all of that patient's unbilled charges (consultation, lab, medicine, bed), with a unique invoice number.	High
FR-45	Billing Staff shall be able to apply a discount of 0–50% with a reason to an UNPAID invoice.	Medium
FR-46	Billing Staff shall record full or partial payments (Cash, Card, UPI); a payment must be greater than zero and not exceed the outstanding balance. Invoice status moves UNPAID → PARTIALLY_PAID → PAID.	High
FR-47	The system shall generate a printable receipt with a unique receipt number for every payment and a printable invoice.	Medium

5.13 Dashboard & Reports (RPT)

Table 17: Functional requirements – Dashboard & Reports

ID	Requirement	Priority
FR-48	The dashboard shall show today's KPIs: appointments, patients waiting, bed occupancy, revenue collected, pending lab orders and stock alerts.	Medium
FR-49	The Administrator shall view reports for a date range: appointments per day and status, revenue per day and payment method, bed occupancy per ward and top diagnoses.	Medium

5.14 Audit Trail (AUD)

Table 18: Functional requirements – Audit Trail

ID	Requirement	Priority
FR-50	The system shall record an append-only audit entry (user, role, action, entity, entity id, timestamp, summary) for every create, update and state change, without storing passwords; Administrators can search it.	High

6. Non-functional Requirements

Table 23: Non-functional requirements

ID	Category	Requirement
NFR-01	Performance	95% of API requests shall complete within 500 ms on a standard laptop with the demo dataset; pages shall render within 2 seconds.
NFR-02	Security	Passwords shall be stored only as bcrypt hashes; all API routes except login shall require a signed JWT; all SQL shall use parameterised statements.
NFR-03	Security / Privacy	Access to clinical data shall follow the principle of least privilege defined in the role-permission matrix; audit logs shall not contain passwords or tokens.
NFR-04	Reliability / Integrity	Multi-step operations (admission + bed update, dispensing + stock update, invoice generation, payment recording) shall execute in a single database transaction.

ID	Category	Requirement
NFR-05	Usability	The interface shall be responsive (≥ 360 px width), use consistent colour-coded status badges, show field-level validation messages and allow booking an appointment in at most 4 steps.
NFR-06	Maintainability	The code shall follow a layered architecture (UI → REST API → Services → Domain rules → Data access); business rules shall be isolated in pure, unit-tested domain modules.
NFR-07	Portability	The system shall run on Windows, macOS and Linux with Node.js 20 or later, and on current Chrome, Edge, Firefox and Safari browsers.
NFR-08	Auditability	Every state-changing operation shall be traceable to an authenticated user and timestamp through the audit trail.
NFR-09	Availability	The production deployment should target 99.5% availability during hospital operating hours, with daily database backups (out of scope for the prototype).
NFR-10	Scalability	The design shall support at least 50 concurrent staff users and 100,000 patient records without architectural change.

NFR-01 to NFR-08 are verified by automated tests (see Part D); NFR-09 and NFR-10 are design targets for a production deployment and are addressed by the architecture (Part B) rather than measured on the prototype.

7. User Requirements

User requirements describe, in the language of each user class, what they need to achieve. They are written as user stories with acceptance criteria and are refined into the functional requirements listed in the last column. The acceptance tests (AT-01 … AT-05) in Part D validate them end-to-end.

Table 24: User requirements (user stories)

ID	As a …	I want to …	So that …	Acceptance criteria	FRs
UR-01	Receptionist	register a new patient in under two minutes and be warned if the patient may already exist	so that every patient has exactly one record and one permanent ID	Unique ID P00001… is shown on save; invalid mobile/DOB is highlighted on the field; a possible duplicate (same mobile + DOB or same name + DOB) is listed and must be confirmed.	FR-07, FR-08, FR-09
UR-02	Receptionist / Doctor	find a patient by ID, name or mobile and open the full history	so that no information has to be asked or typed twice	Search returns matches as I type; the history shows visits, admissions and bills; clinical notes are visible to doctors only.	FR-10, FR-11, FR-12, FR-30
UR-03	Receptionist	book, reschedule and cancel appointments only in free slots	so that doctors are never double-booked	Free / booked / past slots are shown for the chosen date; booking a taken slot is refused with a clear message; cancellation asks for a reason.	FR-15 – FR-20
UR-04	Receptionist	check patients in and give them a token with an expected waiting time	so that the waiting area is orderly and patients know when they will be seen	Token numbers are sequential per doctor per day; the queue board shows position and ETA; urgent patients can be moved to the front with a recorded reason.	FR-21 – FR-24
UR-05	Doctor	call the next patient and record vitals, diagnosis, prescription and lab orders on one screen	so that the consultation is fast and complete	Only one consultation can be open at a time; diagnosis is mandatory to complete; completing the visit charges the fee automatically.	FR-25 – FR-29
UR-06	Lab Technician	see a worklist of pending tests and enter results that are flagged when abnormal	so that urgent abnormal results reach the doctor quickly	STAT orders appear first; a result can only be entered after sample collection; values outside the reference range are marked ABNORMAL.	FR-31 – FR-34

ID	As a ...	I want to ...	So that ...	Acceptance criteria	FRs
UR-07	Pharmacist	dispense prescriptions only when stock is sufficient and be alerted about low or expiring stock	so that patients never receive expired medicine and stock-outs are avoided	Dispensing is blocked when any item is short or expired; low-stock and expiry (≤ 30 days) alerts are listed; restocking records batch and expiry.	FR-35 – FR-37
UR-08	Receptionist	see which beds are free and admit or transfer patients safely	so that two patients are never assigned the same bed	The bed board is colour-coded by status; only AVAILABLE/RESERVED beds can be assigned; a transferred patient's old bed goes to CLEANING.	FR-38 – FR-41
UR-09	Doctor / Receptionist	discharge a patient only after the bill is settled, with an emergency override for the administrator	so that revenue is not lost and exceptions are accountable	Discharge is refused while charges are uninvoiced or unpaid; an administrator can override with a mandatory reason that is audited.	FR-42, FR-43
UR-10	Billing Staff	generate an invoice from all pending charges, apply approved discounts and accept part payments	so that billing is accurate and quick	Invoice lines are filled automatically from consultations, tests, medicines and bed days; discount 0–50%; overpayment is refused; each payment prints a receipt.	FR-44 – FR-47
UR-11	Administrator	manage staff accounts, roles and doctor schedules	so that each person can do exactly their job	Usernames are unique; passwords ≥ 8 characters; accounts can be (de)activated; doctor working days, hours and fees are editable.	FR-05, FR-06, FR-13, FR-14
UR-12	Administrator	see today's key figures and period reports	so that I can monitor workload, occupancy and revenue	Dashboard KPIs per role; reports for a date range with charts and CSV export.	FR-48, FR-49
UR-13	All staff	log in securely and see only the functions of my role	so that patient data is protected	Account locks for 15 minutes after 5 wrong passwords; menus and APIs are restricted by role; I can log out.	FR-01 – FR-04
UR-14	Administrator	trace who did what and when	so that clinical and financial actions are accountable	Every state change is in an append-only audit trail that can be searched by entity, action, user and date.	FR-50

8. System Requirements

System requirements refine the user requirements into precise behaviour of the software system, its interfaces and its data.

8.1 System features (stimulus → response)

Table 25: System features

Feature	Stimulus / input	System response / output
Authentication	Username + password; Bearer token on every API call	Issues an 8-hour JWT bound to a server-side session; increments failed attempts and locks after 5; rejects revoked/expired tokens with HTTP 401 and unauthorised roles with HTTP 403.
Patient registration	Demographics, contact, blood group, allergies	Validates fields; searches existing records with the same DOB for phone/name matches; returns HTTP 409 POSSIBLE_DUPLICATE unless confirmed; assigns the next ID P#####.
Slot calculation	Doctor + date	Generates slots from work start to end at slot length on working days; marks slots BOOKED (overlap with an active appointment) or PAST (before current time).
Appointment booking	Patient, doctor, date, start time, reason	Validates working day/hours, slot alignment and future time; rejects doctor or patient overlap (HTTP 409); stores SCHEDULED appointment in a transaction.

Feature	Stimulus / input	System response / output
Check-in & queue	Today's SCHEDULED appointment	Assigns token = max(token)+1 for doctor/day; sets CHECKED_IN; queue ordered URGENT → check-in time; ETA = ahead × average + remaining time of current consultation.
Consultation	Vitals, symptoms, diagnosis, notes, prescription items, lab tests	Validates vitals ranges and BP format; stores prescription (PENDING) and lab orders (ORDERED) with charges; on completion requires diagnosis, closes appointment and adds the consultation fee charge.
Laboratory	Sample collected; result value / findings	Enforces ORDERED → SAMPLE*COLLECTED* → *COMPLETED*; *computes ABNORMAL when the value lies outside [ref*low, ref_high]; cancelling removes the unbilled charge.
Pharmacy	Restock (qty, batch, expiry); dispense request	Refuses expired batches; checks every item for stock and expiry inside one transaction; decrements stock and creates medicine charges; produces low-stock / near-expiry / expired alerts.
Beds & admissions	Patient, attending doctor, bed; transfer bed; discharge	Allows only valid bed transitions; one active admission per patient and one patient per bed (enforced by unique indexes); bed-day charges = max(1, ⌈hours/24⌉) shared across stays; discharge only when clearance holds or an administrator overrides with a reason.
Billing	Patient (+ admission); discount %; payment amount, method, reference	Builds an invoice from unbilled charges with number INV-YYYYMMDD-NNNN; discount 0–50% only while UNPAID; payment 0 < amount ≤ balance; reference required for CARD/UPI; receipt R-YYYYMMDD-NNNN.
Reporting	Date range	Aggregates appointments by day/status, revenue by day/method/source, occupancy by ward, top diagnoses and doctor performance.
Audit	Any create / update / state change	Appends user, role, action, entity, id, timestamp and summary; credentials are scrubbed; UPDATE/DELETE on the log are blocked by database triggers.

8.2 External interface requirements

User interface. A responsive single-page web application (minimum width 360 px) with a sidebar showing only the modules permitted for the logged-in role, colour-coded status badges, modal dialogs for actions, field-level validation messages, toast notifications and print-optimised invoice and receipt layouts.

Software interface (REST API). All functions are exposed as JSON over HTTP under `/api`. Every route except login and health requires a Bearer token; each is protected by the permission shown below (generated from the source code).

Table 26: REST API endpoints (78)

Method	Endpoint	Required permission
GET	`/api/health`	public
POST	`/api/auth/login`	public
GET	`/api/auth/me`	any authenticated user
POST	`/api/auth/logout`	any authenticated user
GET	`/api/users`	users:manage
POST	`/api/users`	users:manage
PATCH	`/api/users/:id`	users:manage
POST	`/api/users/:id/activate`	users:manage
POST	`/api/users/:id/deactivate`	users:manage
POST	`/api/users/:id/reset-password`	users:manage
GET	`/api/departments`	doctors:read
POST	`/api/departments`	doctors:manage

Method	Endpoint	Required permission
GET	/api/doctors	doctors:read
GET	/api/doctors/:id	doctors:read
POST	/api/doctors	doctors:manage
PATCH	/api/doctors/:id	doctors:manage
GET	/api/doctors/:id/slots	appointments:read
GET	/api/patients	patients:read
POST	/api/patients/check-duplicates	patients:write
POST	/api/patients	patients:write
GET	/api/patients/:id	patients:read
PATCH	/api/patients/:id	patients:write
GET	/api/patients/:id/history	patients:read
GET	/api/appointments	appointments:read
POST	/api/appointments	appointments:write
GET	/api/appointments/:id	appointments:read
PATCH	/api/appointments/:id/reschedule	appointments:write
POST	/api/appointments/:id/cancel	appointments:write
POST	/api/appointments/:id/no-show	appointments:write
POST	/api/appointments/:id/check-in	queue:checkin
GET	/api/queue	queue:read
GET	/api/queue/:doctorId	queue:read
POST	/api/queue/:appointmentId/priority	queue:priority
POST	/api/queue/:doctorId/call-next	consultation:write
GET	/api/consultations	clinical:read
GET	/api/consultations/current	consultation:write
GET	/api/consultations/:id	clinical:read
PUT	/api/consultations/:id	consultation:write
PUT	/api/consultations/:id/prescription	consultation:write
POST	/api/consultations/:id/complete	consultation:write
GET	/api/lab/tests	lab:read or lab:order
GET	/api/lab/orders	lab:read
GET	/api/lab/stats	lab:read
POST	/api/lab/orders	lab:order
GET	/api/lab/orders/:id	lab:read
POST	/api/lab/orders/:id/collect	lab:process
POST	/api/lab/orders/:id/result	lab:process
POST	/api/lab/orders/:id/cancel	lab:process or lab:order
GET	/api/pharmacy/medicines	pharmacy:read
POST	/api/pharmacy/medicines	pharmacy:manage
PATCH	/api/pharmacy/medicines/:id	pharmacy:manage
POST	/api/pharmacy/medicines/:id/restock	pharmacy:manage
GET	/api/pharmacy/alerts	pharmacy:read
GET	/api/pharmacy/prescriptions	prescriptions:read
GET	/api/pharmacy/prescriptions/:id	prescriptions:read

Method	Endpoint	Required permission
POST	/api/pharmacy/prescriptions/:id/dispense	prescriptions:dispense
GET	/api/beds	beds:read or admissions:read
GET	/api/beds/available	admissions:write
POST	/api/beds/:id/status	beds:manage
GET	/api/admissions	admissions:read
POST	/api/admissions	admissions:write
GET	/api/admissions/:id	admissions:read
POST	/api/admissions/:id/transfer	admissions:write
POST	/api/admissions/:id/request-discharge	admissions:request-discharge
POST	/api/admissions/:id/discharge	admissions:write
GET	/api/billing/pending	billing:read
GET	/api/billing/unbilled	billing:read
GET	/api/billing/invoices	billing:read
POST	/api/billing/invoices	billing:write
GET	/api/billing/invoices/:id	billing:read
POST	/api/billing/invoices/:id/discount	billing:write
POST	/api/billing/invoices/:id/payments	billing:write
POST	/api/billing/invoices/:id/cancel	billing:write
GET	/api/billing/receipts/:paymentId	billing:read
GET	/api/reports/dashboard	dashboard:read
GET	/api/reports/summary	reports:read
GET	/api/audit	audit:read
GET	/api/audit/entities	audit:read

Hardware interfaces. Standard keyboard/mouse or touch screen; any printer supported by the browser for invoices and receipts. **Communication interfaces.** HTTP(S) on a hospital LAN; TLS termination by a reverse proxy in production.

8.3 Data requirements

- Persistent data is stored in a relational database (SQLite file) with 20 tables: users, sessions, departments, doctors, patients, appointments, consultations, prescriptions, prescription items, medicines, lab tests, lab orders, wards, beds, bed stays, admissions, charges, invoices, payments and audit logs.
- Integrity rules are enforced in the schema: foreign keys, CHECK constraints on every status column, a unique token per doctor per day, at most one active admission per patient and one admission per bed (partial unique indexes), and append-only triggers on the audit log.
- Money is stored in rupees with two decimals and calculated in paise to avoid rounding drift; dates are ISO-8601; appointment dates/times are local (IST).
- Medical and billing records are never deleted; cancellations change status and are audited.

9. Hardware and Software Requirements

9.1 Hardware requirements

Table 27: Hardware requirements

Component	Minimum	Recommended
Application server	Dual-core 2 GHz CPU, 4 GB RAM, 2 GB free disk	Quad-core CPU, 8 GB RAM, SSD with 20 GB free, UPS
Client workstation	Any PC, laptop or tablet able to run a current browser; 1366 × 768 display	1920 × 1080 display at reception and nursing stations; tablet for doctors
Network	100 Mbit/s LAN or Wi-Fi	Gigabit LAN; separate VLAN for clinical systems
Peripherals	Any printer for invoices/receipts	Thermal receipt printer at billing; barcode scanner (future)

9.2 Software requirements

Table 28: Software requirements

Layer	Software (version used)	Purpose
Operating system	Windows 10/11, macOS 12+, Ubuntu 22.04+	Server and development machines
Runtime	Node.js 20 or later (tested on v22.22.0), npm 10	Runs the API server and build tools
Web server / API	Express 4.22.3, jsonwebtoken 9.0.3, bcryptjs 2.4.3, zod 3.25.76	REST API, authentication, password hashing, input validation
Database	SQLite 3 via better-sqlite3 12.11.1	Embedded relational database (single file, WAL mode)
Front end	React 18.3.1, React Router 6.30.6, Tailwind CSS 3.4.19, Recharts 2.15.4, Vite 5.4.21	Single-page user interface and build
Browser	Chrome / Edge 120+, Firefox 120+, Safari 16+	Client access
Testing	Vitest 3.2.4, Supertest 7.3.1, Playwright 1.64.1	Unit, integration, system and acceptance tests
Design & documentation	PlantUML, docx (document generator), LibreOffice	UML diagrams and this report
Version control	Git, GitHub	Source control and collaboration

10. Assumptions and Constraints

10.1 Assumptions

- The system serves a single hospital site; all staff have individual accounts created by the administrator.
- Patients do not access the system directly; the receptionist registers them and books appointments.
- Doctors work a fixed weekly schedule (days and hours) with a constant slot length; leave management is out of scope.
- One consultation is recorded per appointment; in-patient lab tests are ordered against the admission.
- Consultation and in-patient services are treated as tax-exempt, so no GST is computed; all amounts are in Indian rupees (₹).
- Payments are taken at the hospital counter (cash, card or UPI) and only recorded in MediFlow; no money is processed online.
- The server clock is correct and set to the hospital time zone (IST).
- All demonstration data (patients, doctors, bills) is synthetic; no real patient data is used.

10.2 Constraints

- **Schedule:** the project had to be specified, designed, implemented and tested by a group of five students before 31 October 2026.
- **Technology:** only free and open-source software is used; the system must run on a standard student laptop without a separate database server.
- **Platform:** browser-based only; no native mobile app.
- **Regulatory:** patient data is sensitive; access must follow least privilege, passwords must be hashed and every change must be auditable. Full compliance with the Digital Personal Data Protection Act 2023 and NABH accreditation standards would require further work (consent management, data retention policy, backups) beyond this prototype.
- **Scalability:** SQLite supports one application server; a multi-branch deployment would move to a client–server DBMS (e.g. PostgreSQL) without changing the service layer.

10.3 Dependencies

- Availability of npm packages listed in Section 9.2 at build time.
- A modern browser with JavaScript enabled on every client.

PART B – SOFTWARE DESIGN DOCUMENT (SDD)

> The design follows IEEE Std 1016-2009 viewpoints: context and composition (architecture, modules), logical (class diagrams), interaction (sequence diagrams), state dynamics (state diagrams), information (data model) and interface (UI and API). All diagrams are UML 2.5 and were produced with PlantUML from the source files in docs/uml/.

1. System Architecture

MediFlow uses a **three-tier, layered client–server architecture**. The presentation tier is a React single-page application running in the browser; the application tier is a Node.js/Express server exposing a REST API; the data tier is an embedded SQLite database. Inside the server, responsibilities are separated into four layers, and each layer may only call the layer below it.

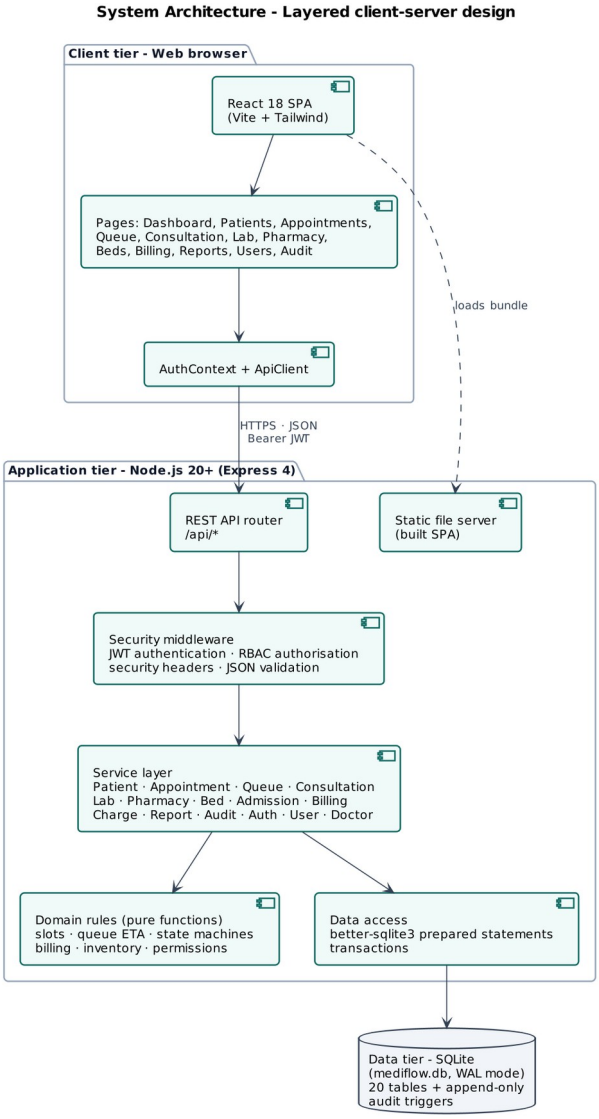


Figure 2: Layered system architecture

Table 29: Architectural layers

Layer	Responsibility	Implementation
Presentation	Screens, navigation by role, forms with client-side validation, charts, printing	client/src – 18 page components, shared UI kit, AuthContext, ApiClient

Layer	Responsibility	Implementation
API / controller	HTTP routing, JSON parsing, authentication, authorisation, error mapping to status codes	server/routes/index.js, server/middleware/auth.js, server/app.js
Service	Use-case logic and transactions; orchestrates rules, data access and audit	server/services/*.js (15 services)
Domain rules	Pure, side-effect-free business rules: slots, conflicts, queue ETA, state machines, billing, inventory, permissions	server/domain/*.js (unit-tested in isolation)
Data access	Connection, schema, prepared statements, transactions, constraints and triggers	server/db/connection.js, schema.sql, seed.js

1.1 Design decisions and rationale

Table 30: Key design decisions

Decision	Rationale
Web SPA + REST API	Works on any device with a browser, no installation at workstations; the API can later serve a mobile app or integrations.
Business rules as pure functions	Rules such as slot validation, ETA and bed-day allocation are deterministic and testable without a database (NFR-06); a frozen clock makes time-based rules reproducible.
SQLite with WAL and transactions	Zero-administration database suitable for one hospital server and for marking/demonstration; ACID transactions guarantee multi-step operations (NFR-04).
Constraints in the schema as well as in code	Defence in depth: unique partial indexes stop double bed allocation and double active admissions even under concurrent requests.
JWT + server-side sessions	Stateless verification with the ability to revoke on logout or deactivation (FR-04, FR-06).
Charges as an intermediate ledger	Clinical modules only record billable events; billing turns them into invoices. This decouples modules and prevents missed charges.
Append-only audit log	Accountability required by O7; enforced by database triggers so even direct SQL cannot alter history.

1.2 Deployment view

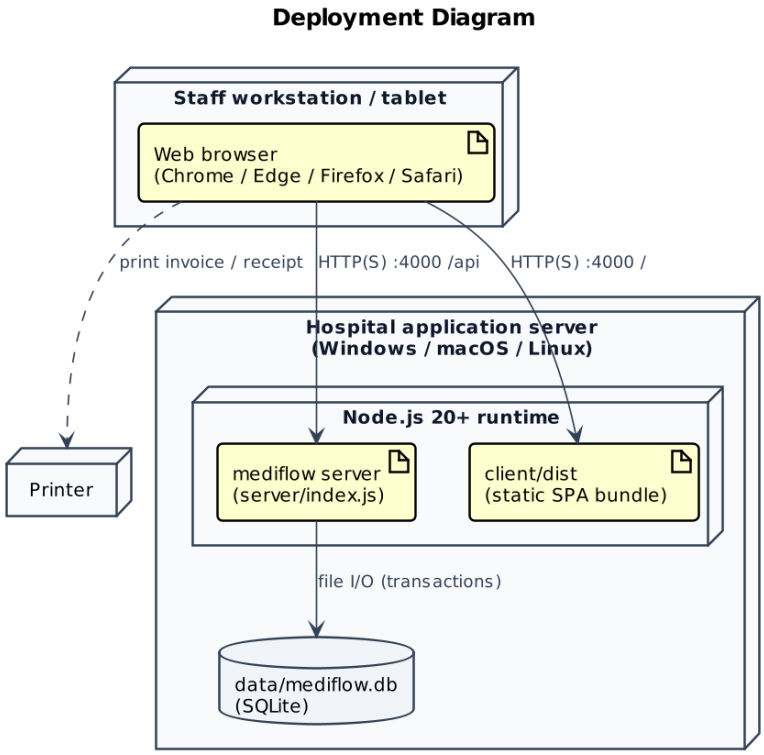


Figure 3: Deployment diagram

The prototype consists of about 3,577 lines of server code in 39 files, 5,497 lines of client code in 29 files and 2,582 lines of automated tests.

2. Module / Component Description

The system is decomposed into thirteen functional modules. Each module owns its screens, service, rules and tables, and communicates with other modules only through service calls and shared identifiers (patient, appointment, admission, charge).

Table 31: Module / component description

Module	Responsibility	UI pages	Services / rules	Tables	FRs
Authentication & Access Control	Login/logout, JWT sessions, account lock-out, role permissions, route guards	Login, Layout (role menu)	AuthService, auth middleware, authRules, permissions	users, sessions	FR-01 – FR-04
User Administration	Create users, (de)activate, reset passwords, role-permission matrix view	Users & Roles	UserService	users	FR-05, FR-06
Patient Management	Registration with unique ID, validation, duplicate detection, search, edit, history	Patients, Patient detail	PatientService, patientRules	patients	FR-07 – FR-12
Doctor & Department	Doctor profiles, schedules, fees, departments, free-slot calculation	Doctors	DoctorService, appointmentRules	doctors, departments	FR-13 – FR-15
Appointments	Book, reschedule, cancel, no-show, filters; 4-step booking wizard	Appointments	AppointmentService, appointmentRules	appointments	FR-15 – FR-20
Smart Queue	Check-in tokens, queue board, ETA, urgent priority, call next	Queue Board	QueueService, queueRules	appointments, consultations	FR-21 – FR-25
Consultation & Records	Vitals, diagnosis, notes, follow-up, prescription builder, lab ordering, history	Consultation	ConsultationService, ChargeService	consultations, prescriptions, prescription_items	FR-26 – FR-30

Module	Responsibility	UI pages	Services / rules	Tables	FRs
Laboratory	Worklist, sample collection, result entry, abnormal flag, cancellation	Laboratory	LabService, labRules	lab*tests*, *lab*orders	FR-31 – FR-34
Pharmacy & Inventory	Medicine master, restock, alerts, dispensing	Pharmacy	PharmacyService, inventoryRules	medicines, prescriptions	FR-35 – FR-37
Admission, Bed & Discharge	Bed board, admit, transfer, bed status, discharge request, clearance, override	Beds & Admissions, Admission detail	BedService, AdmissionService, bedStateMachine, admissionRules	wards, beds, admissions, bed_stays	FR-38 – FR-43
Billing & Payments	Pending charges, invoices, discounts, payments, receipts, printing	Billing, Invoice detail, Print views	BillingService, ChargeService, billingRules	charges, invoices, payments	FR-44 – FR-47
Dashboard & Reports	Role-aware KPIs, 7-day trends, management reports, CSV export	Dashboard, Reports	ReportService	(read-only aggregates)	FR-48, FR-49
Audit Trail	Append-only log of every state change; search & filter	Audit Trail	AuditService	audit_logs (+ triggers)	FR-50

2.1 Cross-cutting components

- **Authentication middleware** verifies the JWT, checks that the session is not revoked or expired and that the account is active, and attaches the user (with role and doctor id) to the request.
- **Authorisation** – `authorize(permission…)` checks the role-permission matrix (Section 8). The UI hides menus the role cannot use, but the server is the authority.
- **Validation** – zod schemas and domain validators return HTTP 400 with field-level messages that the UI shows next to the inputs.
- **Error handling** – a central handler maps errors to consistent JSON: 400 validation, 401 unauthenticated, 403 forbidden, 404 not found, 409 business-rule conflict, 423 account locked, 500 unexpected.
- **ChargeService** – a shared ledger used by Consultation, Laboratory, Pharmacy and Admission modules to create billable events.
- **AuditService** – called by every service after a successful state change; scrubs credential-like fields.
- **Clock** – a single time source that tests can freeze.

3. Database Design (optional)

Database design is not required by the assignment; it is included briefly because the class diagram maps directly to the relational schema. The data model has 20 tables in third normal form; status columns are constrained by CHECK clauses that mirror the state diagrams in Section 7.

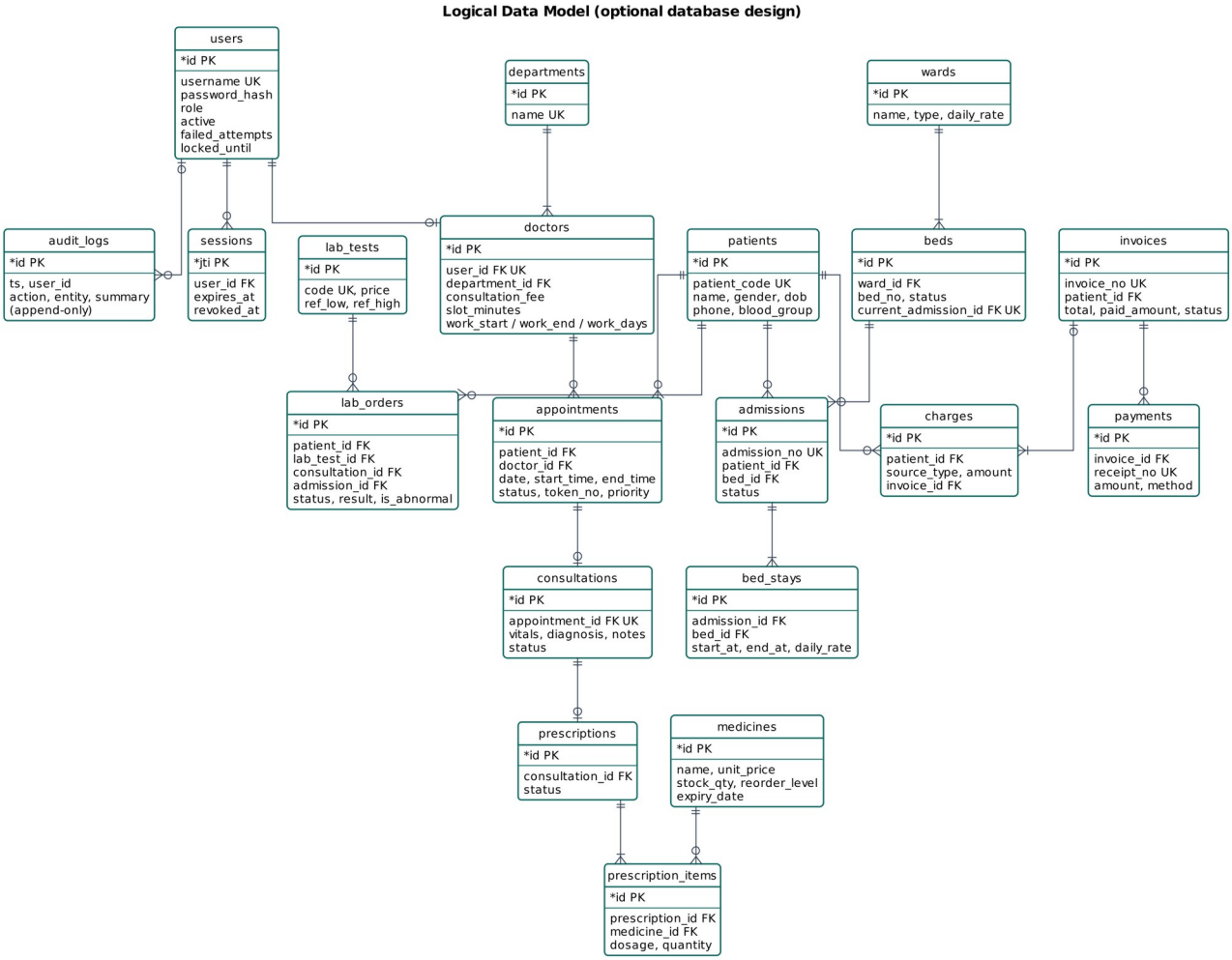


Figure 4: Logical data model (crow's-foot notation)

4. Class Diagram

The **domain class diagram** (Figure below) shows the main entities, their attributes, operations and associations with multiplicities. A patient books many appointments; each appointment has at most one consultation, which may produce one prescription (composed of items referencing medicines) and several lab orders. Wards are composed of beds; an admission is composed of one or more bed stays (one per bed occupied), which is how transfers and bed-day billing are modelled. Charges aggregate into invoices, which own payments.

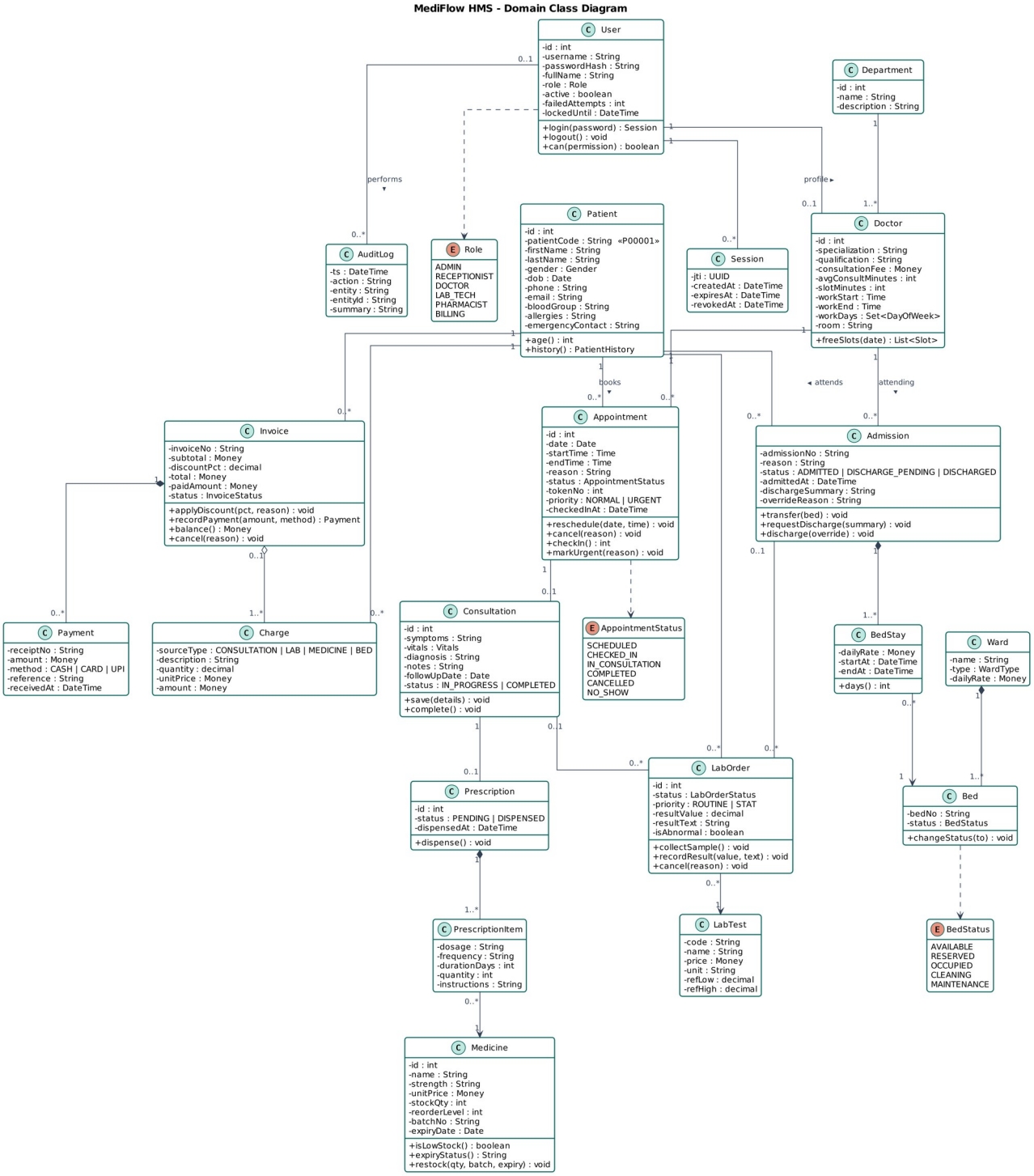


Figure 5: Domain class diagram

The **design class diagram** shows how the implementation realises the domain: controllers delegate to services, services use pure rule modules and the data-access layer, and cross-cutting services (Charge, Audit) are shared.

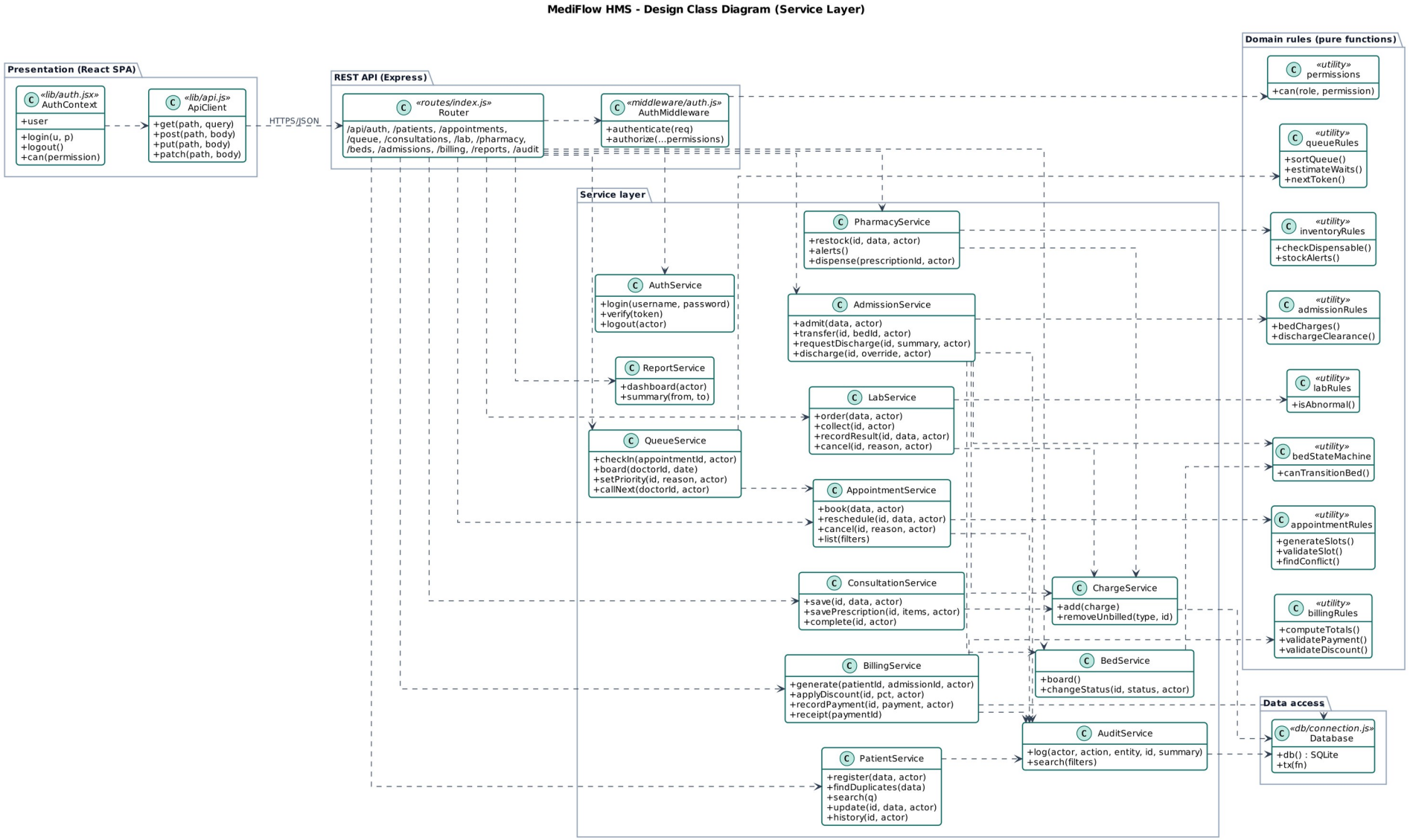


Figure 6: Design class diagram – service layer and dependencies

5. Use Case Diagram

Because a single diagram with six actors and forty use cases is unreadable, the use-case model is split into four diagrams: an overview (Part A, Section 4.3), outpatient care, inpatient care & billing, and administration & access control. «include» marks behaviour that always happens as part of the base use case; «extend» marks optional behaviour.

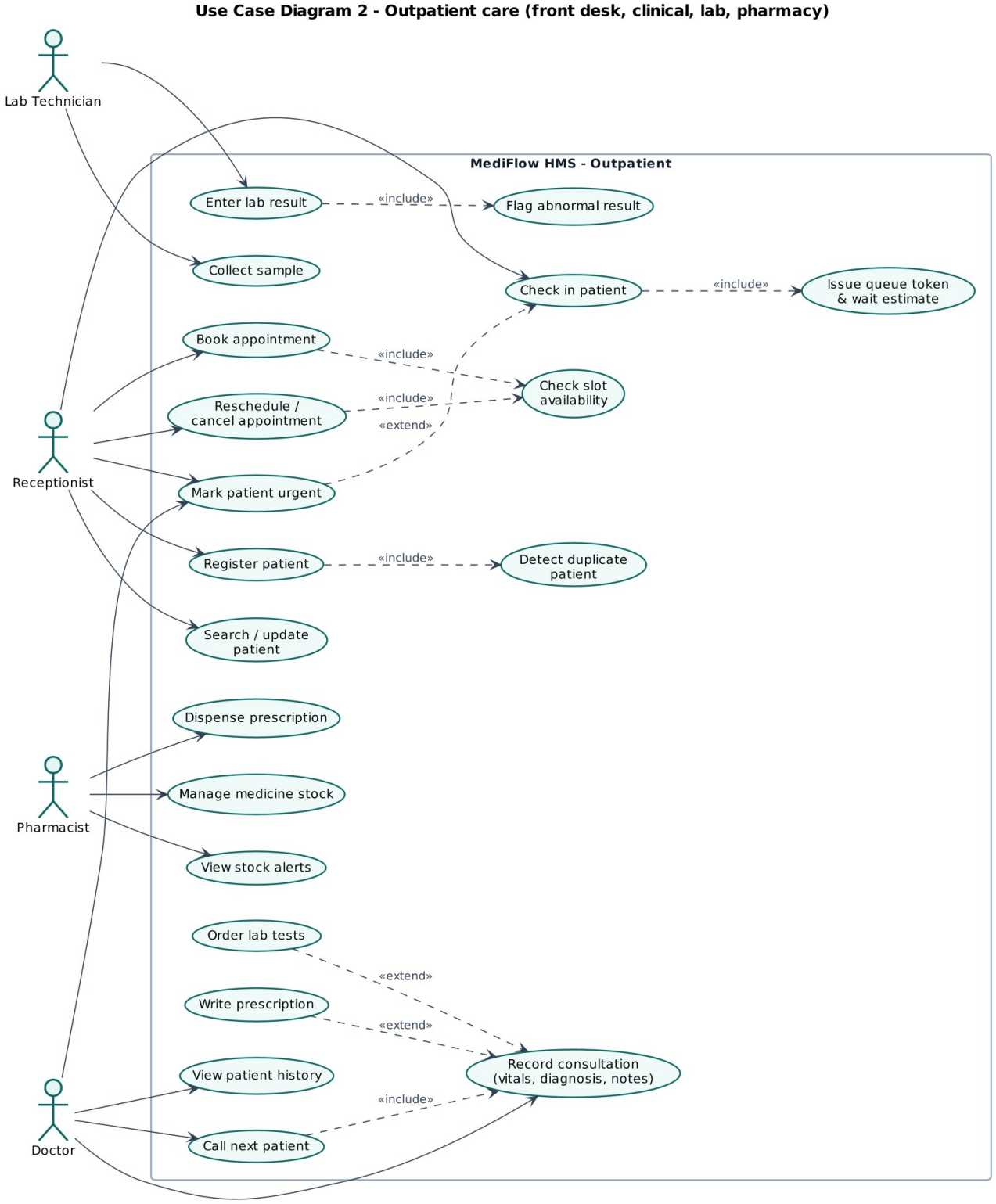


Figure 7: Use case diagram – outpatient care

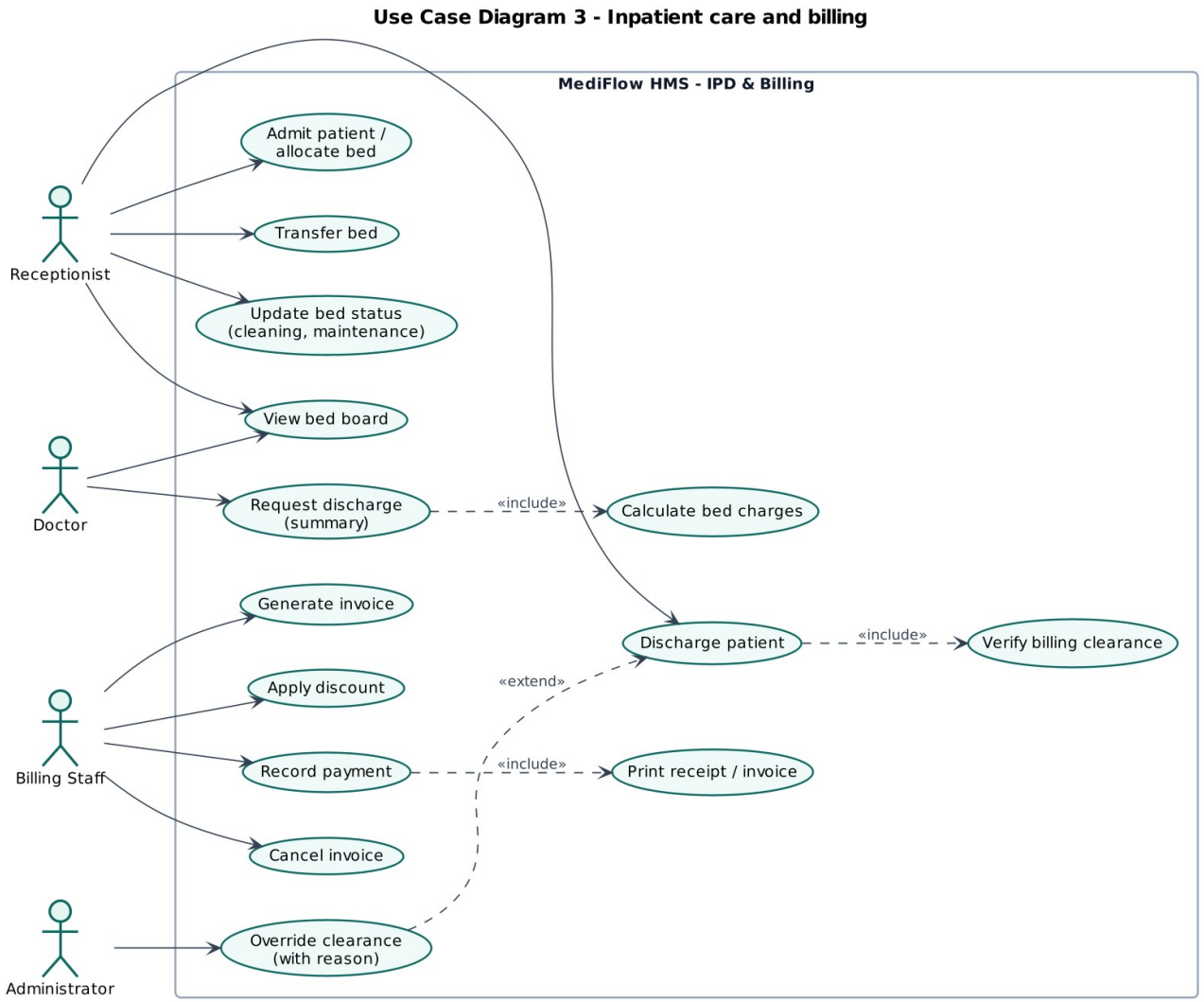


Figure 8: Use case diagram – inpatient care and billing

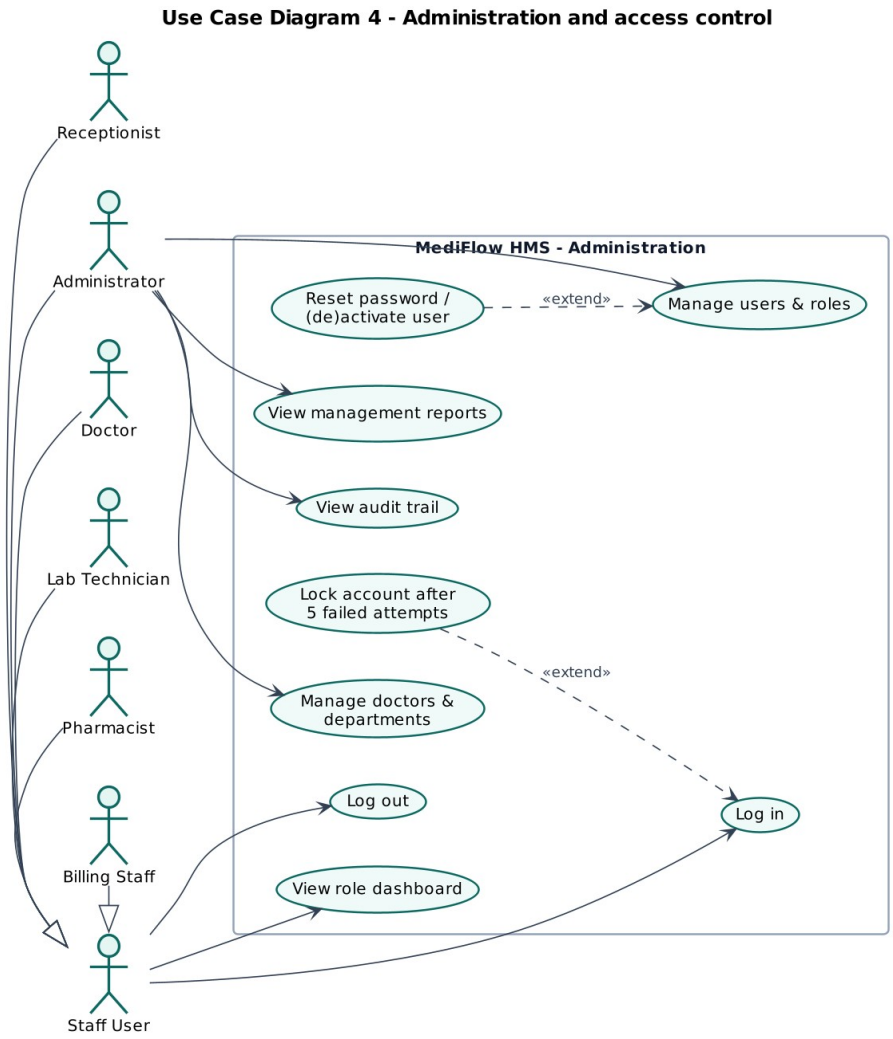


Figure 9: Use case diagram – administration and access control

5.1 Use case specifications

Table 32: Use case specification UC-01 – Book appointment

UC-01	Book appointment
Primary actor(s)	Receptionist
Requirements	FR-15 – FR-17
Pre-condition	Receptionist is logged in; patient is registered; doctor works on the chosen date.
Main success scenario	1. Receptionist opens Appointments → Book appointment. 2. Selects the patient (search by name / ID / phone). 3. Chooses department and doctor; the system shows fee, room and hours. 4. Chooses a date; the system displays all slots marked free, booked or past. 5. Selects a free slot, enters the reason and confirms. 6. System validates the slot, checks doctor and patient overlaps, saves the appointment as SCHEDULED and writes an audit entry.
Alternative flows	5a. Slot was taken meanwhile → HTTP 409 DOCTOR*SLOT*TAKEN; the wizard refreshes the slots. 5b. Patient already has an overlapping appointment → HTTP 409 PATIENT*DOUBLE*BOOKED. 4a. Date is a non-working day → no slots are offered.
Post-condition	Appointment exists in SCHEDULED state and blocks the slot.

Table 33: Use case specification UC-02 – Check in patient and manage queue

UC-02	Check in patient and manage queue
Primary actor(s)	Receptionist (primary), Doctor
Requirements	FR-21 – FR-25
Pre-condition	Appointment is SCHEDULED for today.
Main success scenario	1. Receptionist clicks Check in for the appointment. 2. System assigns the next token for the doctor and sets CHECKED_IN. 3. Queue board shows the patient with position and ETA. 4. Doctor clicks Call next patient. 5. System picks the first patient (URGENT first, then by check-in time), sets IN_CONSULTATION and opens a consultation record.
Alternative flows	2a. Appointment is not for today → HTTP 400. 3a. Patient deteriorates → receptionist/doctor marks URGENT with a mandatory reason (audited); patient moves to the front. 4a. Doctor already has a consultation in progress → HTTP 409.
Post-condition	Patient is in consultation; others' ETA is recalculated.

Table 34: Use case specification UC-03 – Conduct consultation

UC-03	Conduct consultation
Primary actor(s)	Doctor
Requirements	FR-26 – FR-29
Pre-condition	A consultation is IN_PROGRESS for the doctor.
Main success scenario	1. Doctor records vitals, symptoms, diagnosis, notes and follow-up date and saves. 2. Adds medicines (dosage, frequency, duration; quantity auto-calculated) and saves the prescription. 3. Orders lab tests (ROUTINE or STAT); charges are created. 4. Clicks Complete consultation. 5. System checks the diagnosis, sets the consultation and appointment COMPLETED and adds the consultation-fee charge.
Alternative flows	1a. Invalid vitals (e.g. BP “120-80”) → field error. 2a. Quantity ≤ 0 → validation error. 4a. Diagnosis missing → HTTP 400; consultation stays open.
Post-condition	Visit completed; prescription PENDING at pharmacy; lab orders ORDERED; charges waiting for billing.

Table 35: Use case specification UC-04 – Discharge patient

UC-04	Discharge patient
Primary actor(s)	Doctor, Receptionist, Billing Staff, Administrator
Requirements	FR-42, FR-43
Pre-condition	Patient is ADMITTED.
Main success scenario	1. Doctor requests discharge with a summary (≥ 10 characters). 2. System closes the bed stay, calculates bed-days and adds bed charges; status DISCHARGE_PENDING. 3. Billing generates the admission invoice and records payment(s) until PAID. 4. Receptionist completes the discharge. 5. System verifies clearance, sets DISCHARGED and moves the bed to CLEANING.
Alternative flows	4a. Charges not invoiced or balance outstanding → HTTP 409 BILLING*NOT*CLEARED. 4b. Emergency transfer: Administrator overrides with a reason; discharge proceeds and a DISCHARGE_OVERRIDE audit entry records the outstanding amount.
Post-condition	Admission closed; bed in CLEANING until housekeeping marks it AVAILABLE.

Table 36: Use case specification UC-05 – Record payment

UC-05	Record payment
Primary actor(s)	Billing Staff
Requirements	FR-44 – FR-47
Pre-condition	Invoice exists and is UNPAID or PARTIALLY_PAID.
Main success scenario	1. Billing staff opens the invoice and clicks Record payment. 2. Enters amount (defaults to balance), method and reference (for card/UPI). 3. System validates 0 < amount ≤ balance, stores the payment with a receipt number and updates the status. 4. Receipt is printed.
Alternative flows	3a. Amount exceeds balance → HTTP 400 with the outstanding balance. 3b. Card/UPI without reference → HTTP 400. 3c. Invoice already PAID or CANCELLED → HTTP 400.
Post-condition	Invoice PARTIALLY_PAID or PAID; receipt available for printing.

6. Sequence Diagrams

The sequence diagrams show the interaction between actors, the React UI, the API layer, services, rule modules and the database for the four most important scenarios. Messages are numbered; alt/opt fragments show the error and override paths that are also covered by test cases.

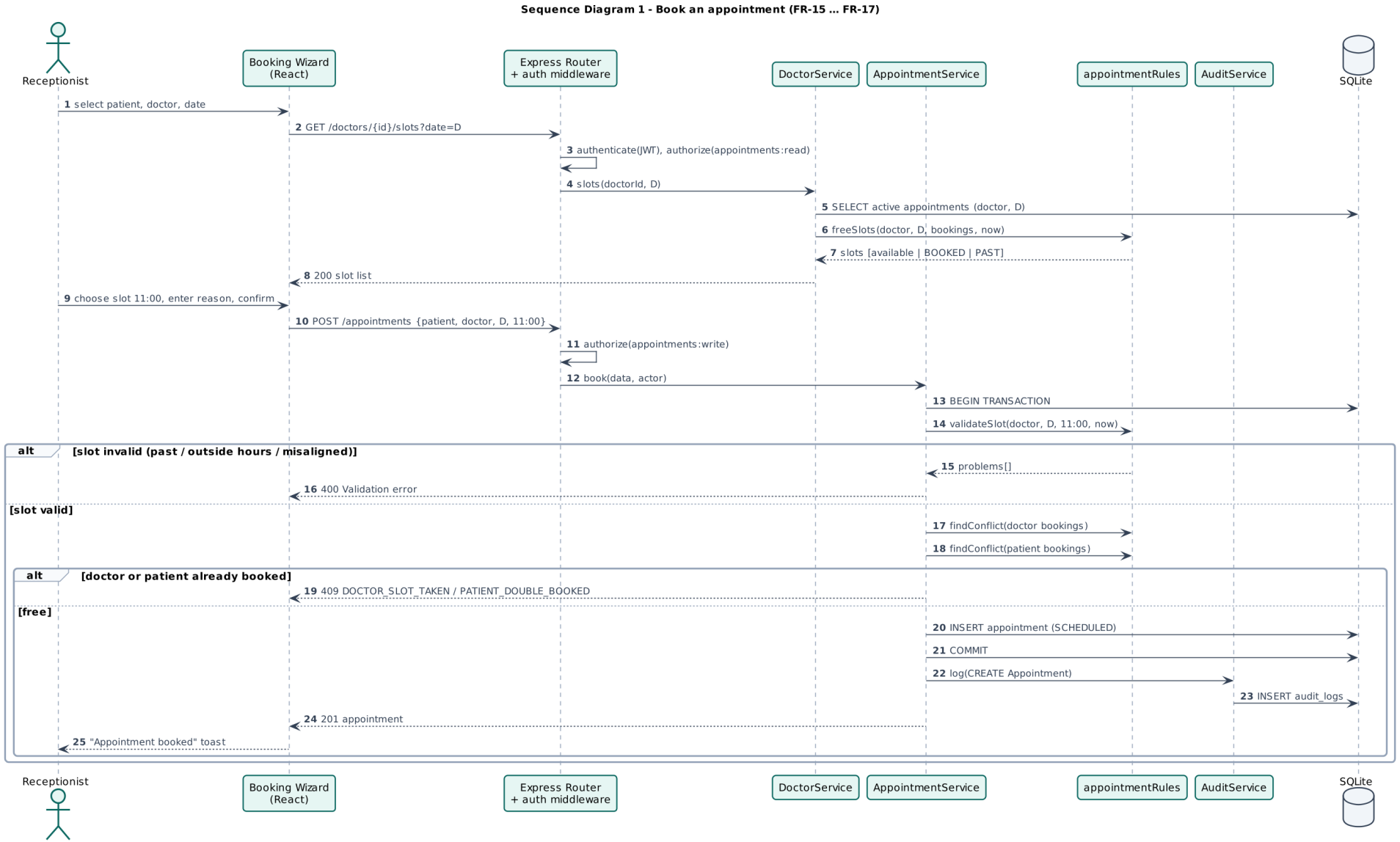


Figure 10: Sequence diagram – book an appointment

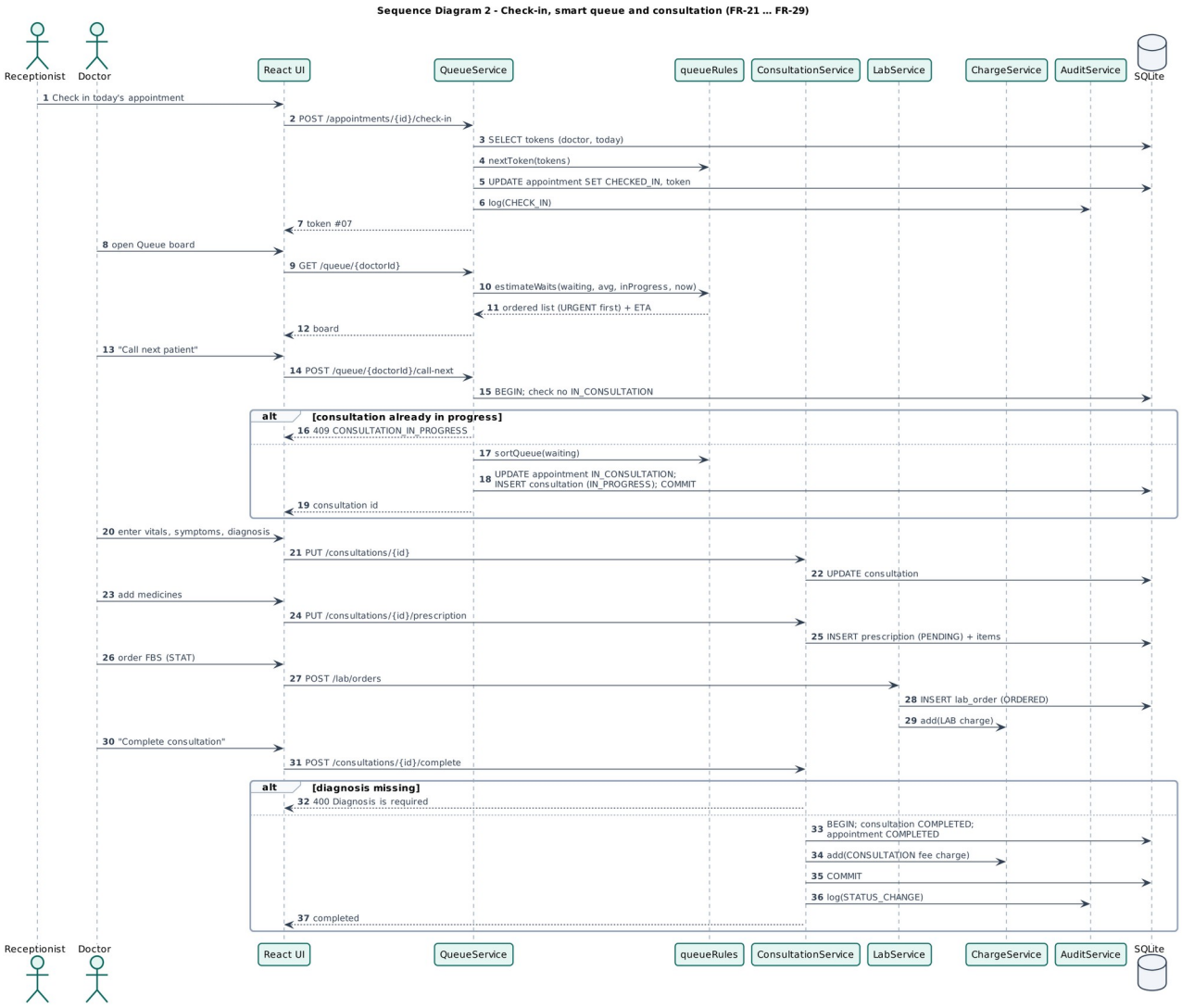


Figure 11: Sequence diagram – check-in, queue and consultation

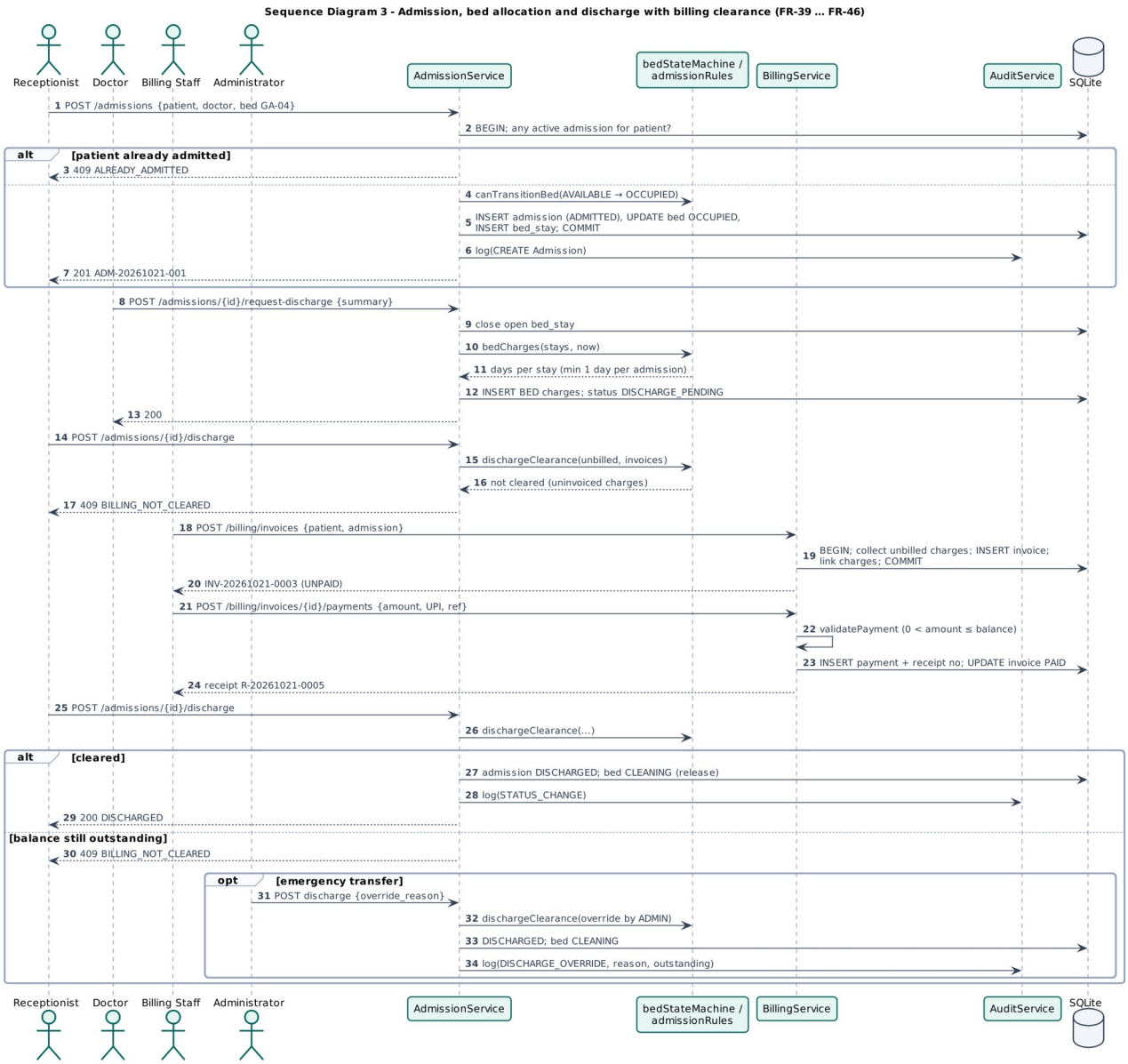


Figure 12: Sequence diagram – admission and discharge with billing clearance

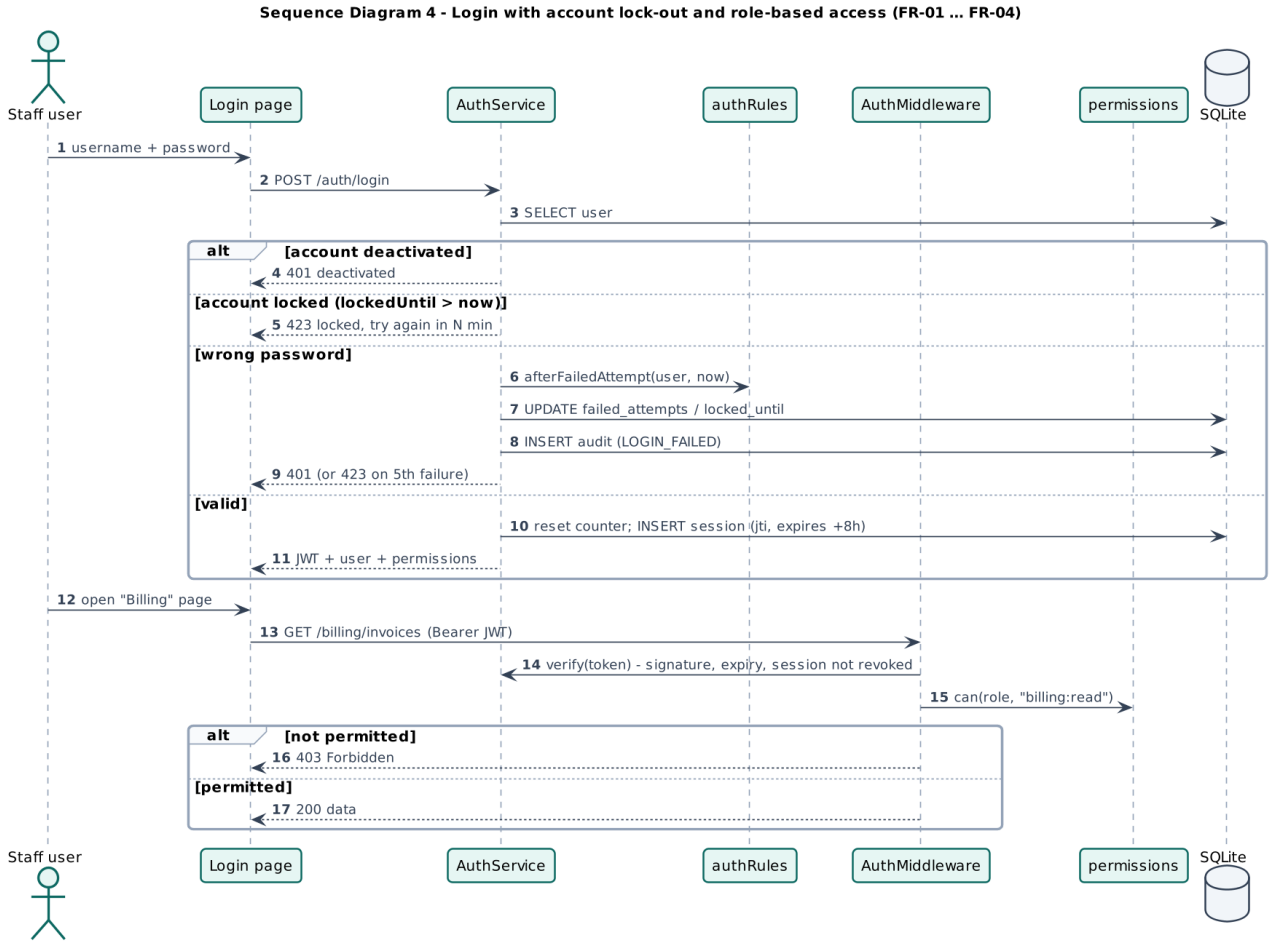


Figure 13: Sequence diagram – login, lock-out and role-based access

7. State Diagrams

State diagrams are applicable to every entity with a life cycle. Transitions not shown are rejected by the system (HTTP 409) – the same transition tables are implemented in `server/domain/*` and verified by unit tests UT-APT-10, UT-IPD-01/02/04, UT-LAB-01 and UT-BIL-06.

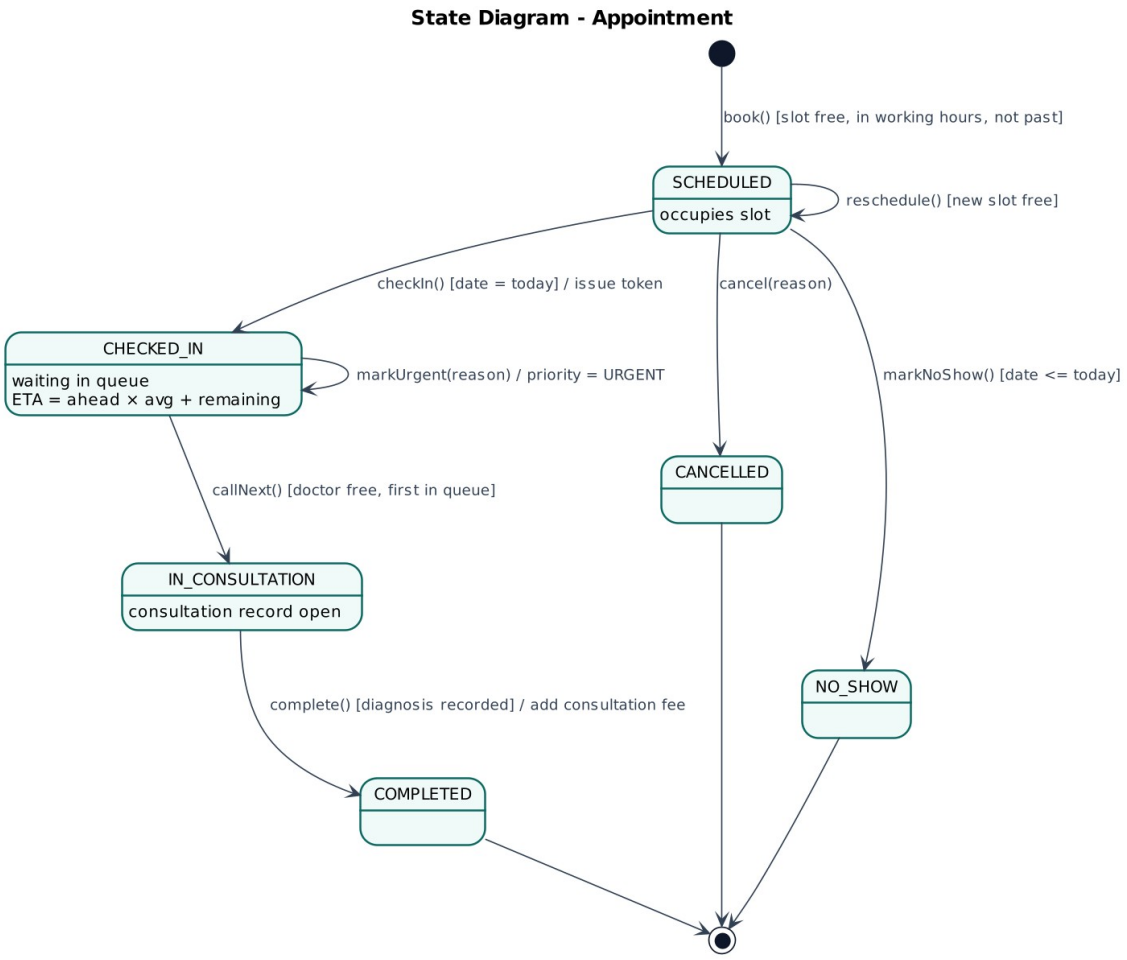


Figure 14: State diagram – Appointment

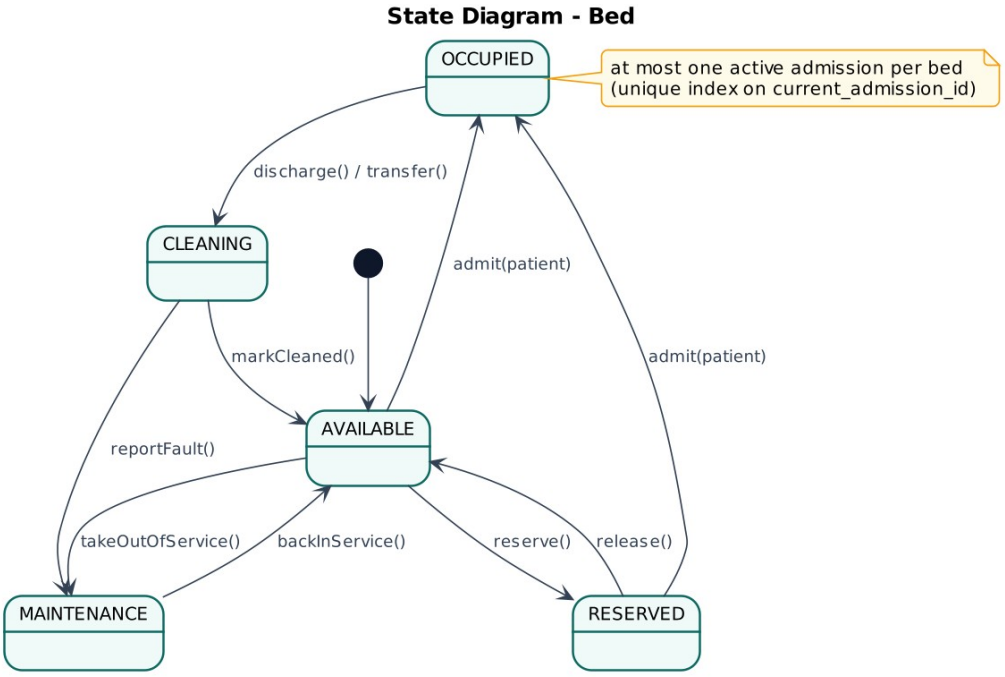


Figure 15: State diagram – Bed

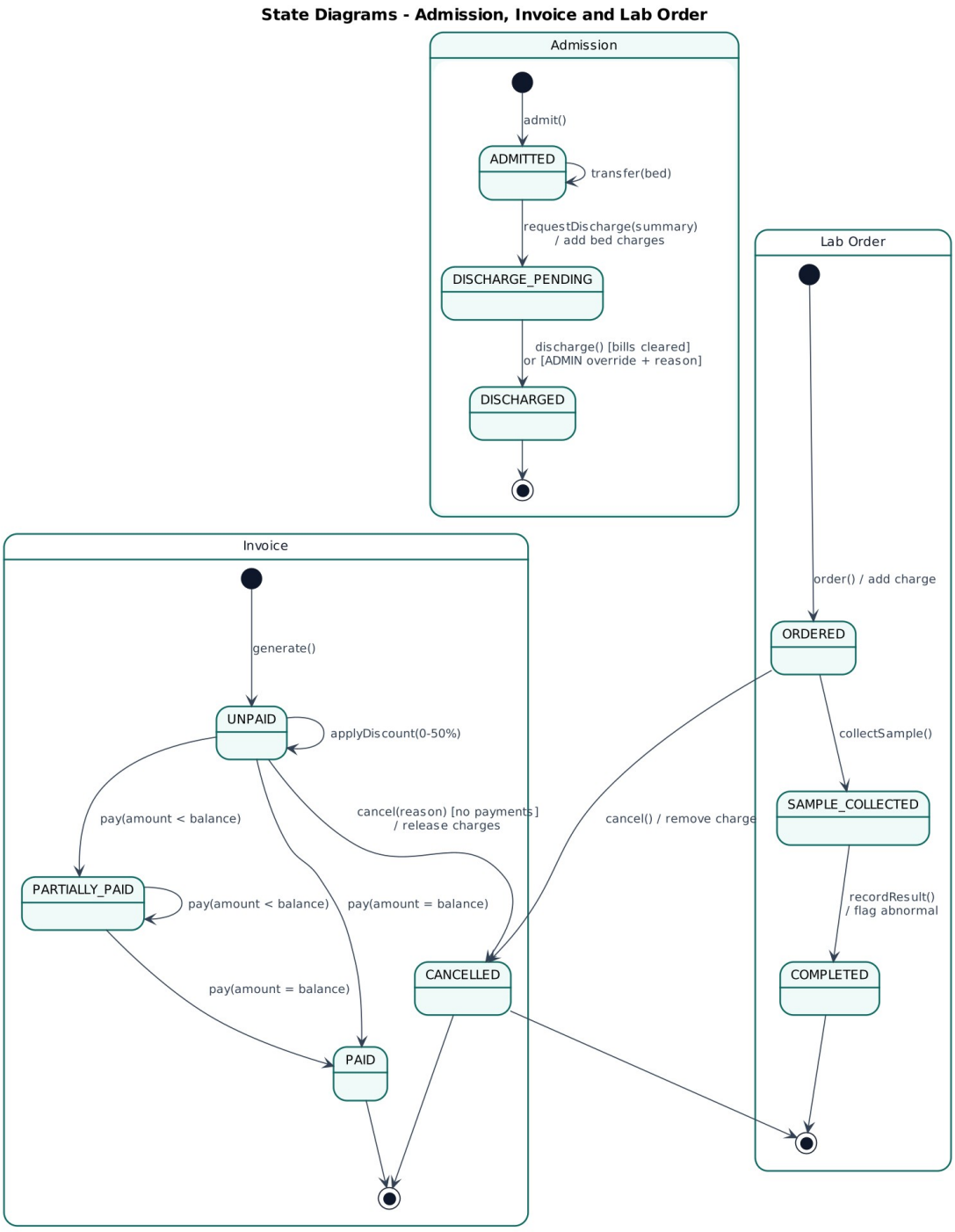


Figure 16: State diagrams – Admission, Invoice and Lab order

7.1 Activity view of the outpatient journey

The activity diagram complements the state diagrams by showing how the responsibility for one patient visit passes between roles (swimlanes).

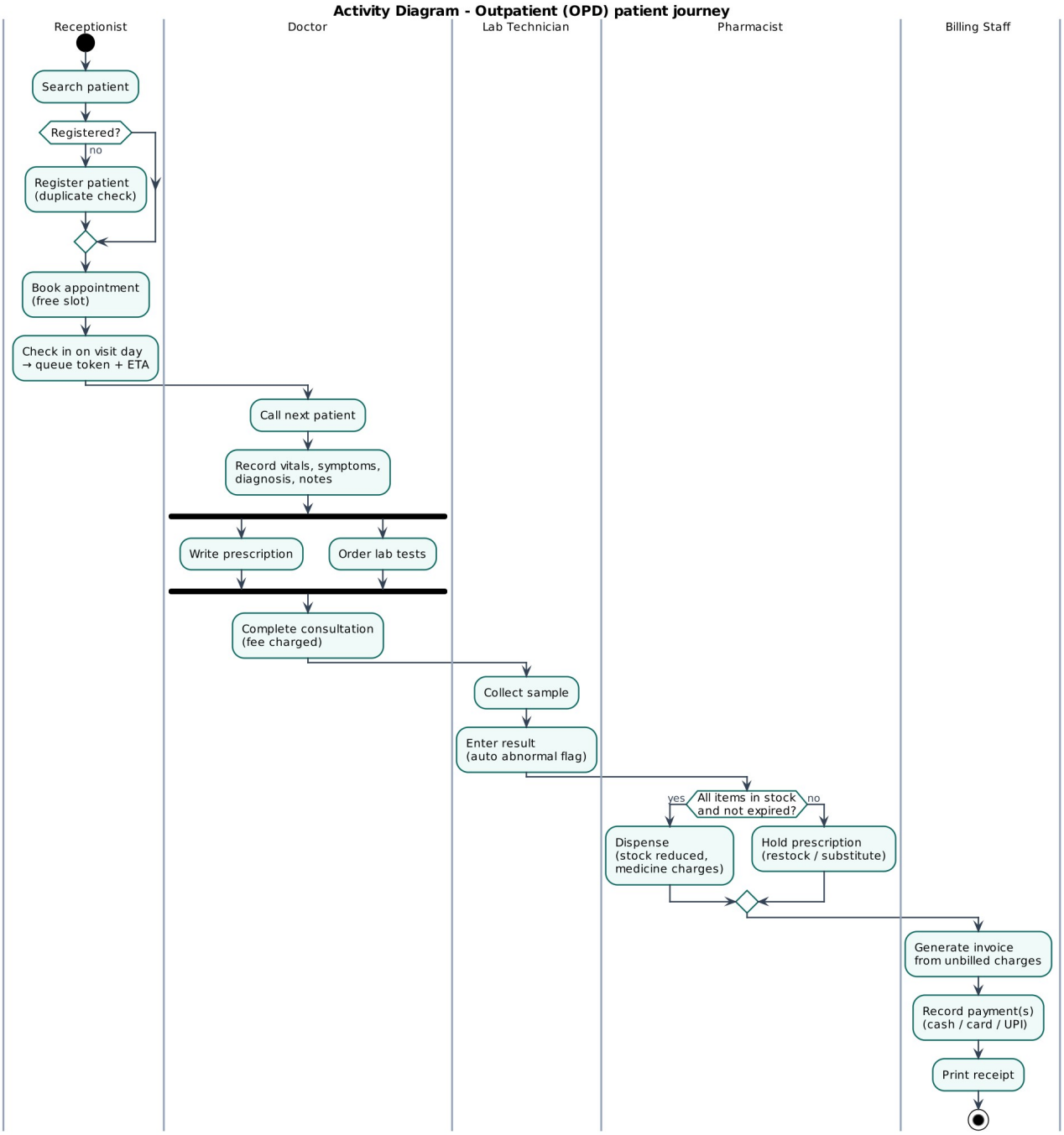


Figure 17: Activity diagram – outpatient journey across roles

8. Security Design

- **Authentication:** bcrypt password hashes (cost 10); JWT signed with HS256 valid 8 h, bound to a server-side session that is revoked on logout or deactivation; 5 failed attempts lock the account for 15 minutes; generic error messages avoid username enumeration.
- **Authorisation:** least-privilege role-permission matrix enforced on every API route; doctors may act only on their own queue and consultations; clinical notes are hidden from non-clinical roles.
- **Input handling:** schema validation on every write; parameterised SQL only (no string concatenation); JSON body limit 1 MB; security headers (nosniff, frame-deny, no-referrer).
- **Integrity and accountability:** transactions for multi-step updates; unique and check constraints; append-only audit log with triggers.

Table 37: Role–permission matrix (generated from server/domain/permissions.js)

Permission	Admin	Recep.	Doctor	Lab	Pharm.	Billing
dashboard:read	✔	✔	✔	✔	✔	✔
users:manage	✔	–	–	–	–	–
doctors:read	✔	✔	✔	✔	✔	✔
doctors:manage	✔	–	–	–	–	–
patients:read	✔	✔	✔	✔	✔	✔
patients:write	✔	✔	–	–	–	–
clinical:read	✔	–	✔	–	–	–
appointments:read	✔	✔	✔	–	–	–
appointments:write	✔	✔	–	–	–	–
queue:read	✔	✔	✔	–	–	–
queue:checkin	✔	✔	–	–	–	–
queue:priority	✔	✔	✔	–	–	–
consultation:write	–	–	✔	–	–	–
lab:read	✔	–	✔	✔	–	–
lab:order	–	–	✔	–	–	–
lab:process	–	–	–	✔	–	–
pharmacy:read	✔	–	✔	–	✔	–
pharmacy:manage	–	–	–	–	✔	–
prescriptions:read	✔	–	✔	–	✔	–
prescriptions:dispense	–	–	–	–	✔	–
beds:read	✔	✔	✔	–	–	–
beds:manage	✔	✔	–	–	–	–
admissions:read	✔	✔	✔	–	–	✔
admissions:write	✔	✔	–	–	–	–
admissions:request-discharge	–	–	✔	–	–	–
admissions:override	✔	–	–	–	–	–
billing:read	✔	–	–	–	–	✔
billing:write	–	–	–	–	–	✔
reports:read	✔	–	–	–	–	–
audit:read	✔	–	–	–	–	–

9. User Interface Design

The interface uses a consistent layout: a role-filtered sidebar, a top bar with global patient search and the user's role, page headers with primary actions, cards, tables and colour-coded status badges (green = available/paid/completed, amber = waiting/pending, red = occupied/unpaid/urgent). Forms validate on the client and show server messages next to the relevant field. The screenshots below were captured automatically by the system tests on the running application.

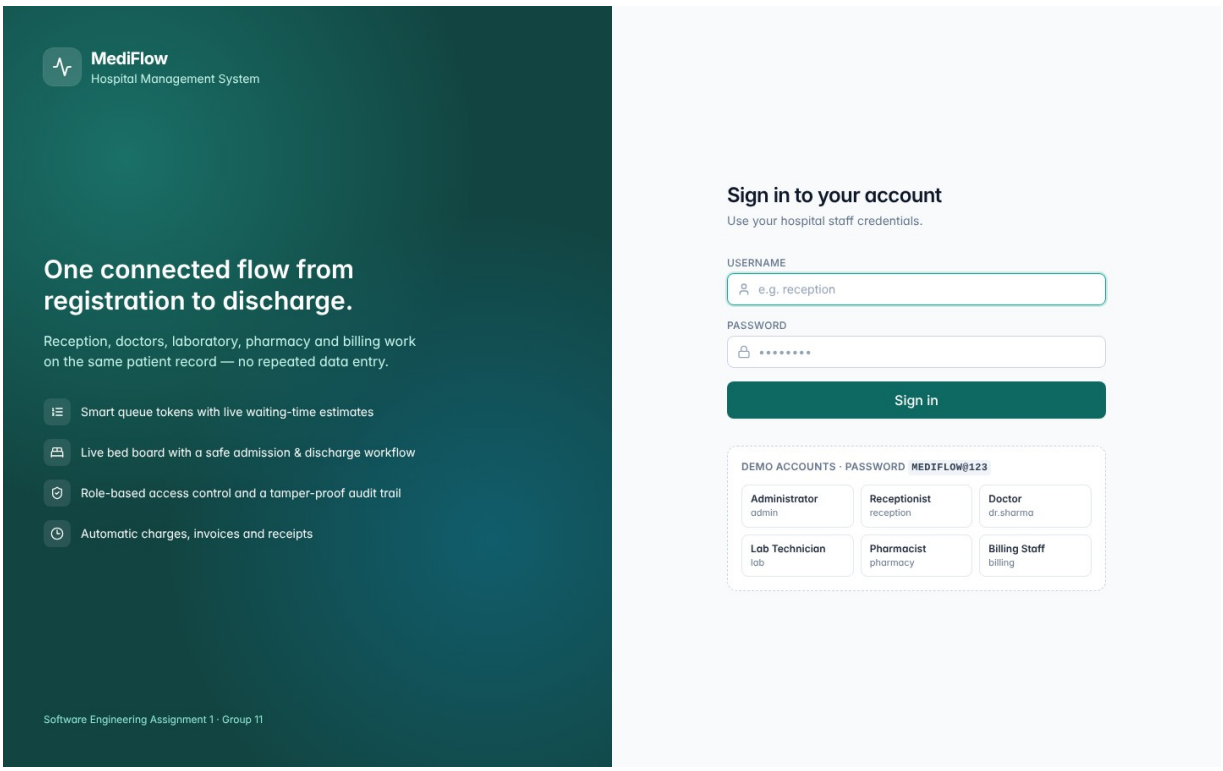


Figure 18: Login screen with demo accounts

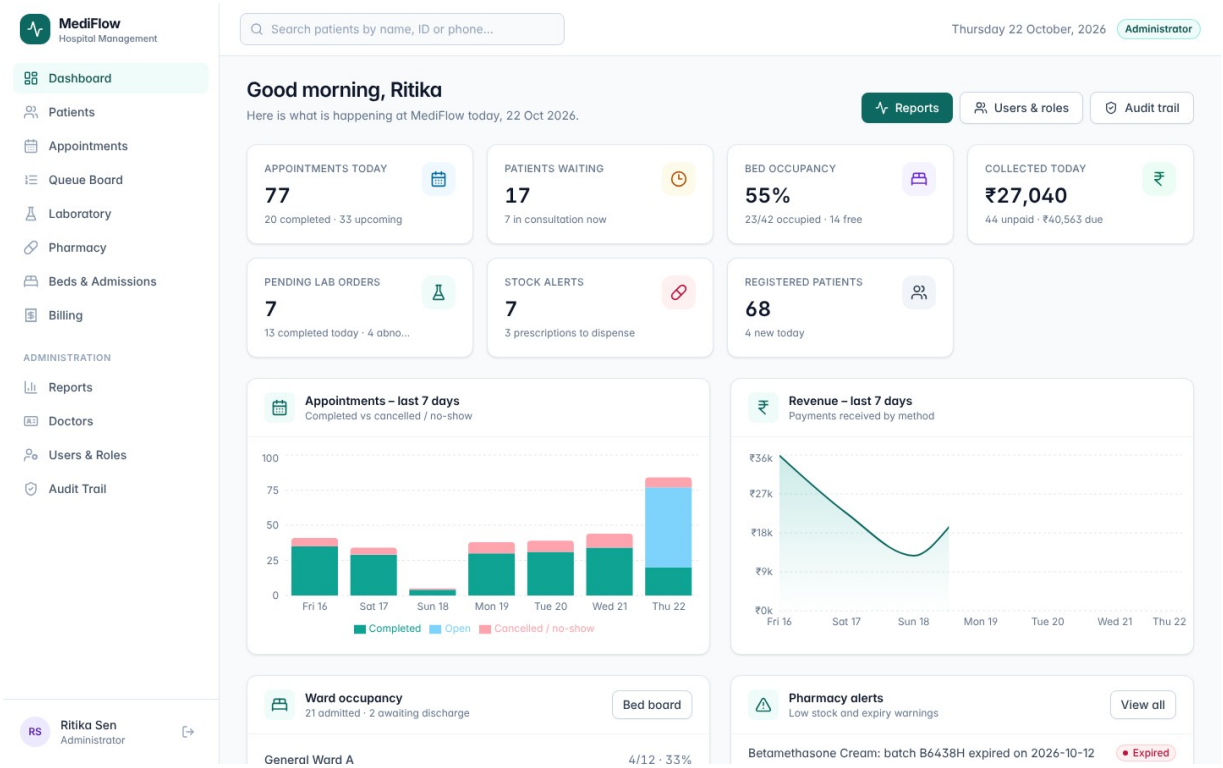


Figure 19: Administrator dashboard – KPIs, 7-day trends, ward occupancy, pharmacy alerts

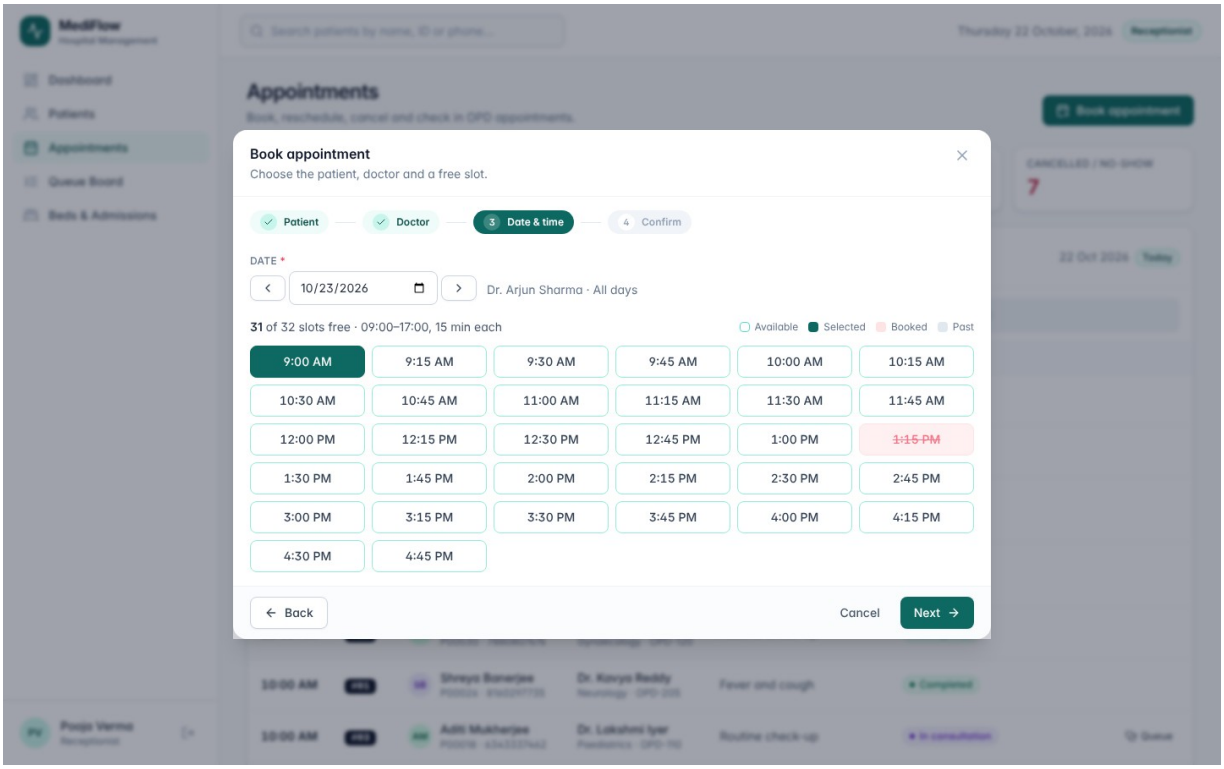


Figure 20: Appointment booking wizard with free / booked / past slots

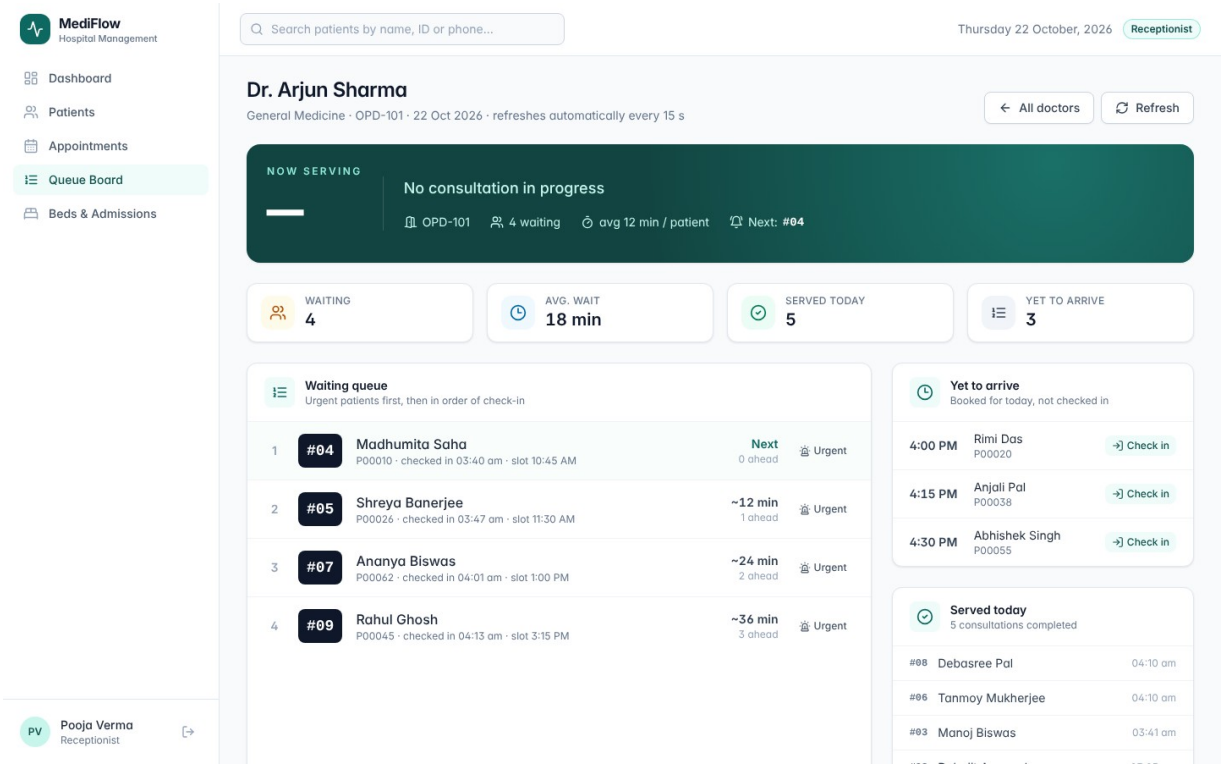


Figure 21: Queue board – now serving, waiting list with tokens, urgent flag and ETA

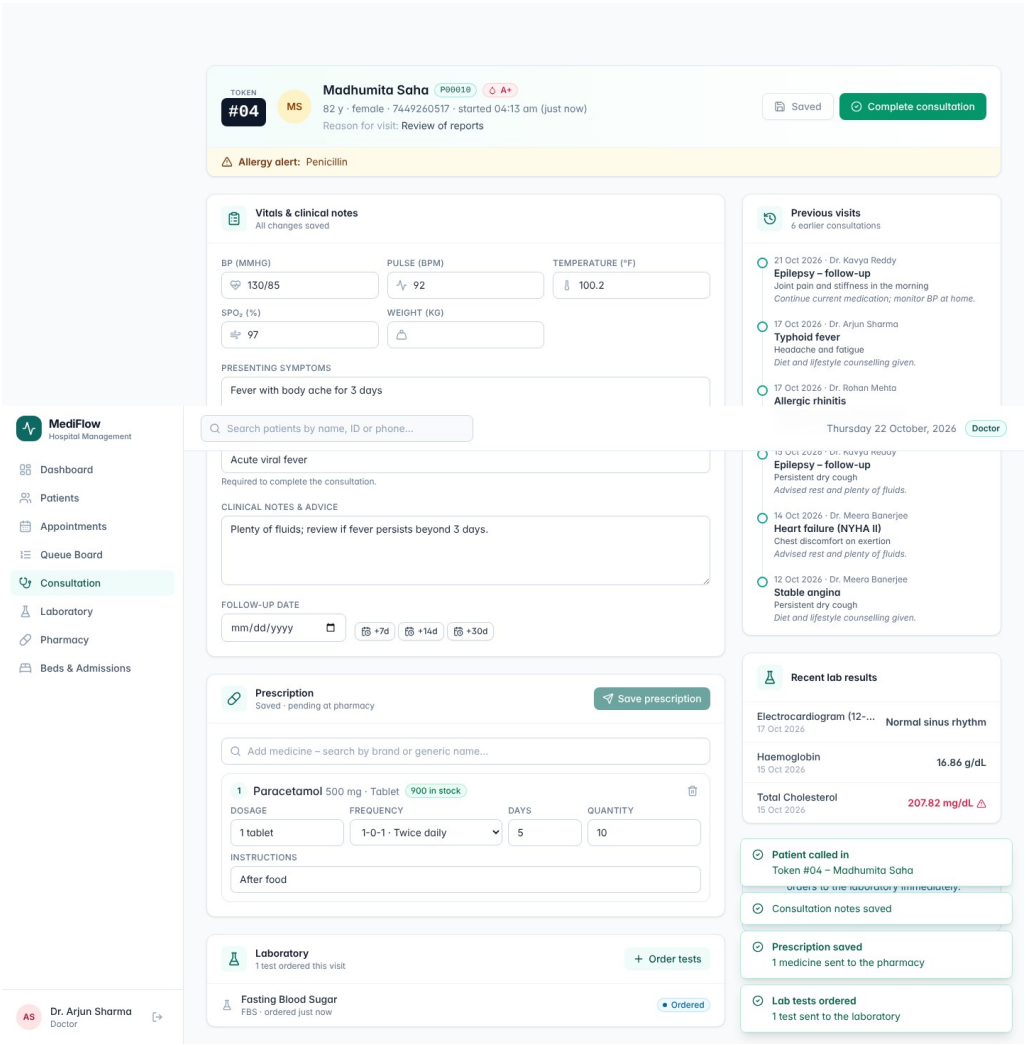


Figure 22: Doctor consultation workspace – vitals, diagnosis, prescription and lab orders

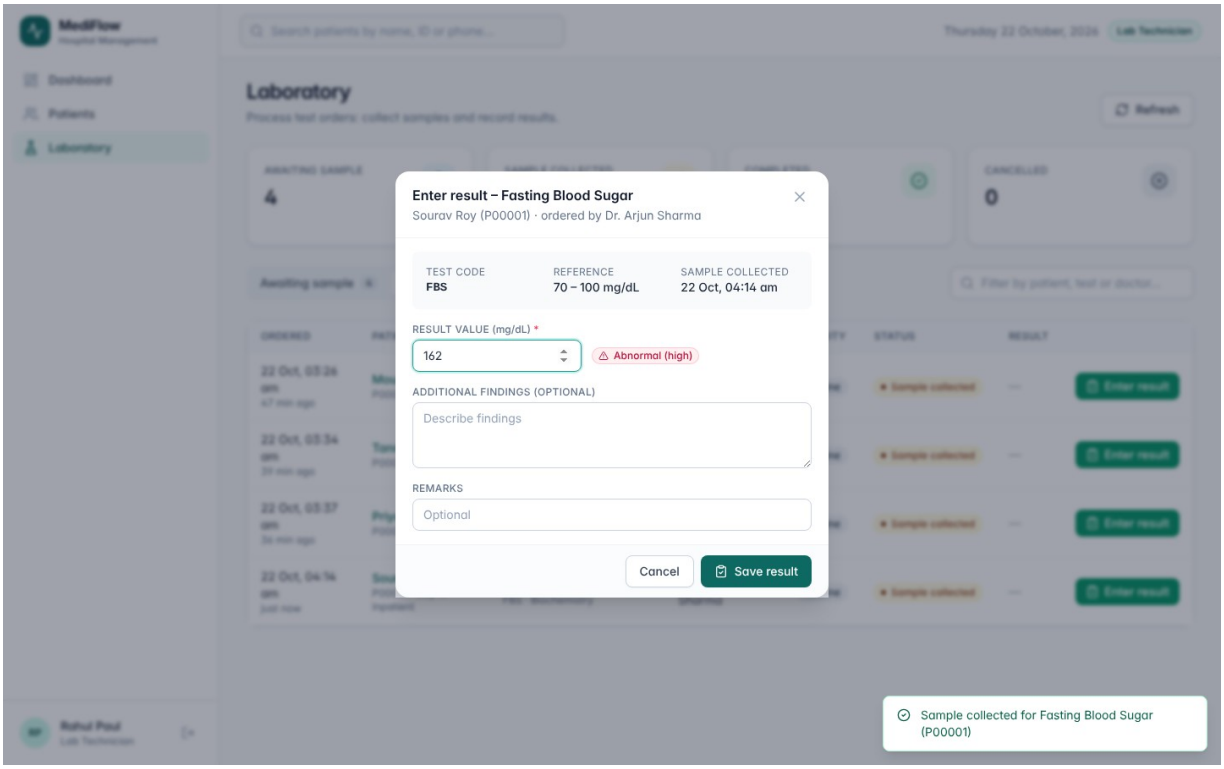


Figure 23: Laboratory result entry with automatic abnormal flag

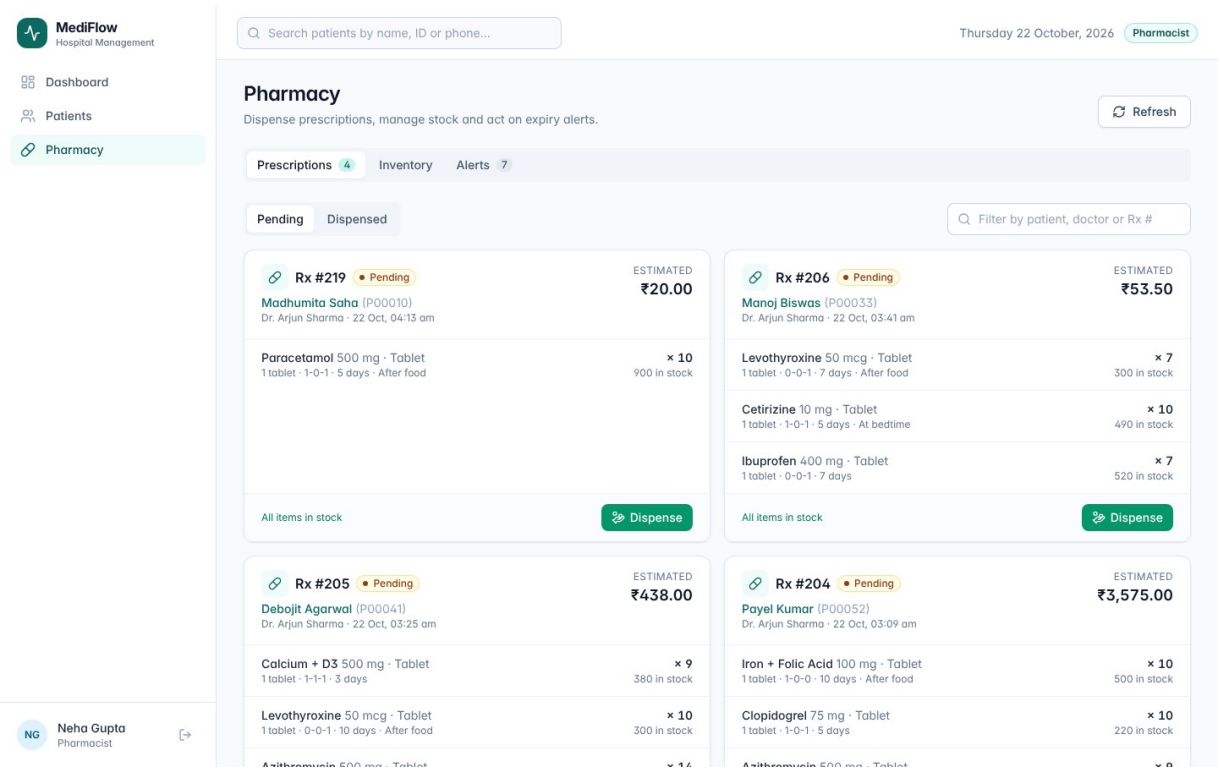


Figure 24: Pharmacy – prescriptions with stock check (undispensable item blocked)

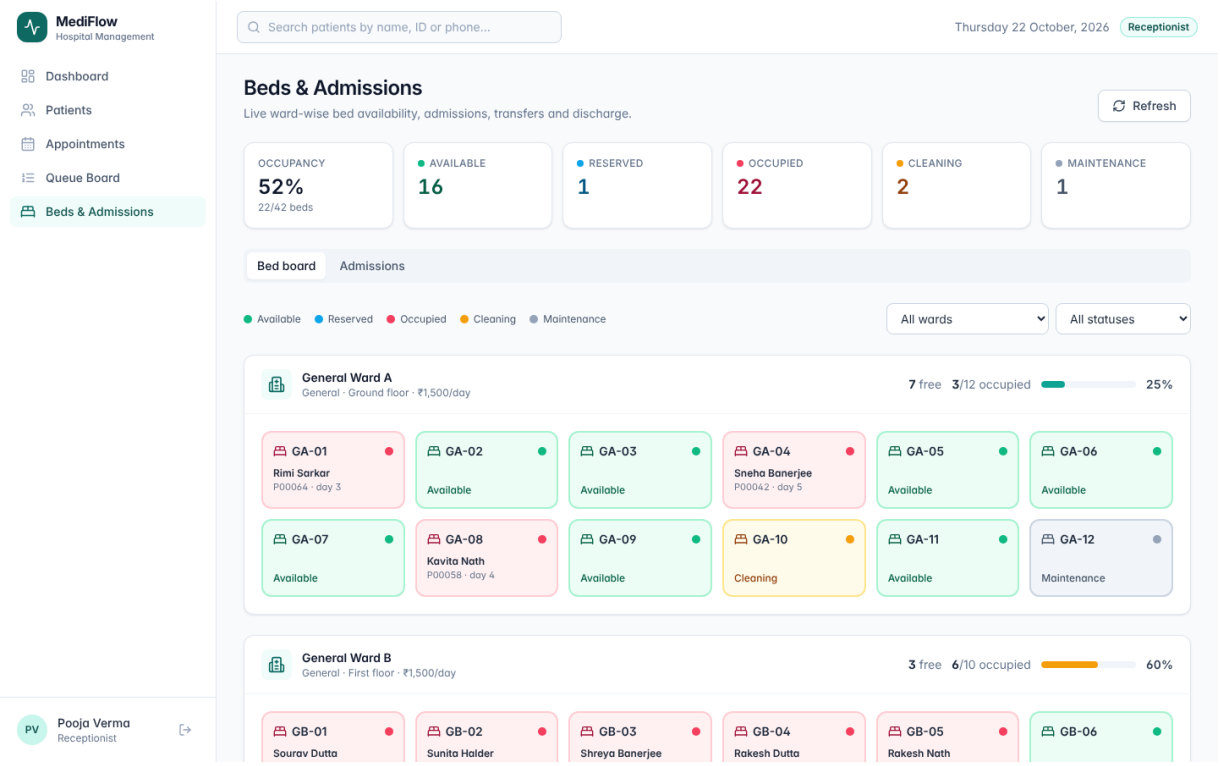


Figure 25: Live bed board colour-coded by status

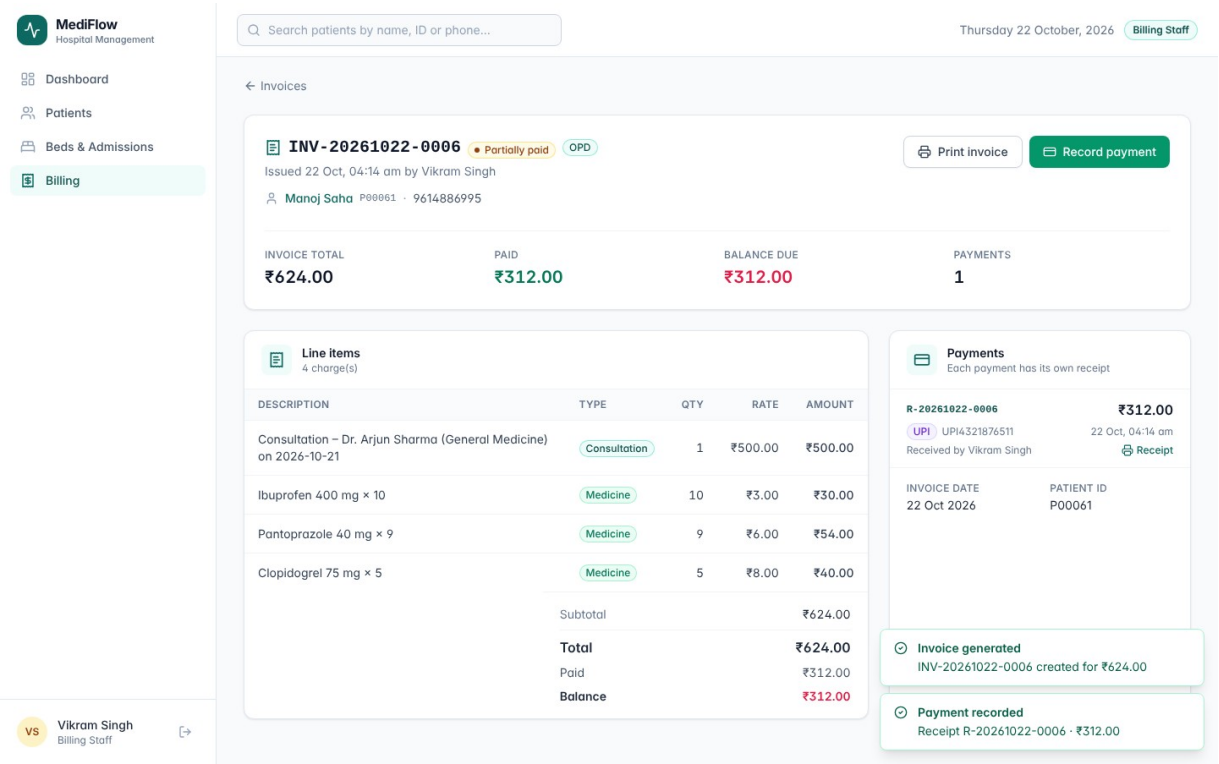


Figure 26: Invoice with line items, discount, payments and balance

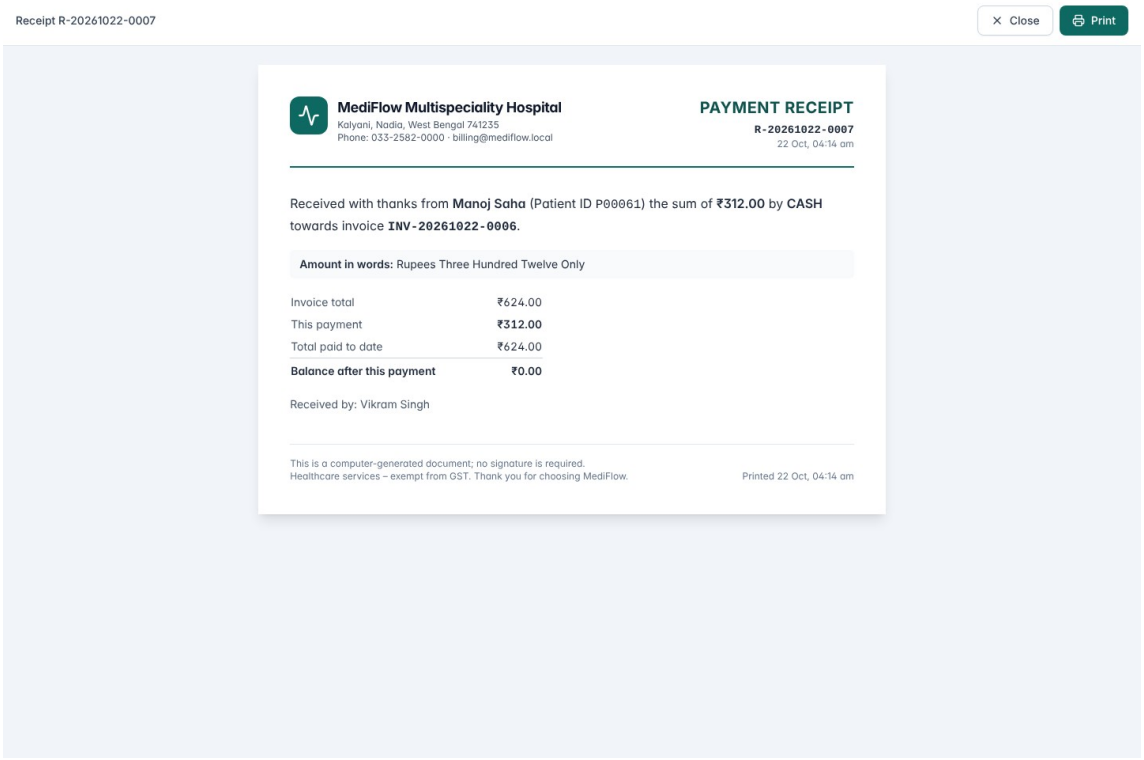


Figure 27: Printable payment receipt

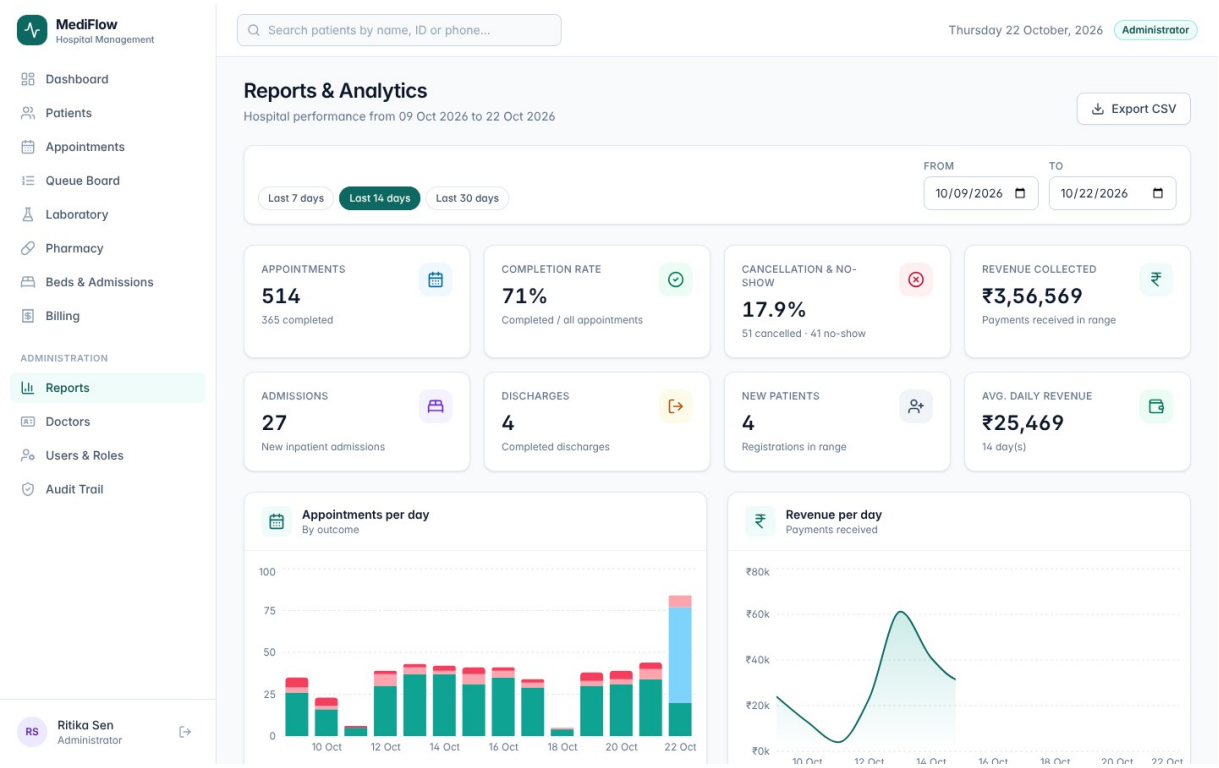


Figure 28: Management reports for a date range

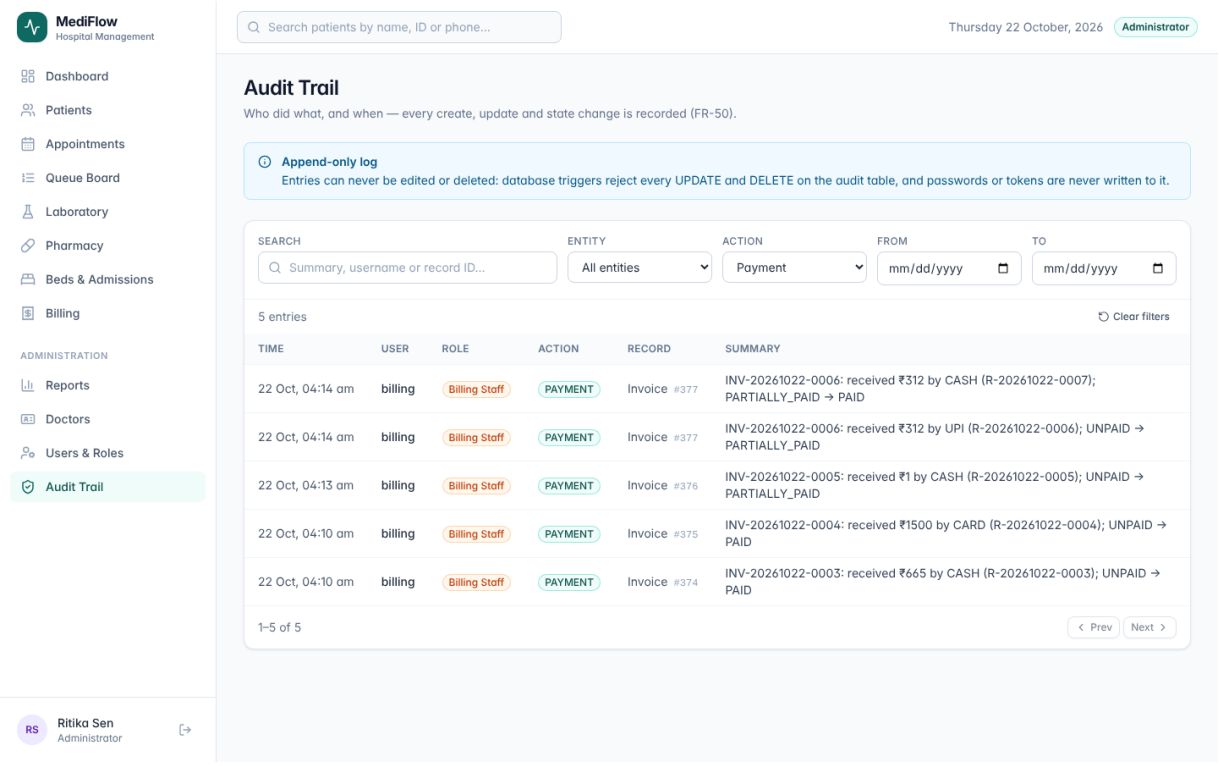


Figure 29: Append-only audit trail

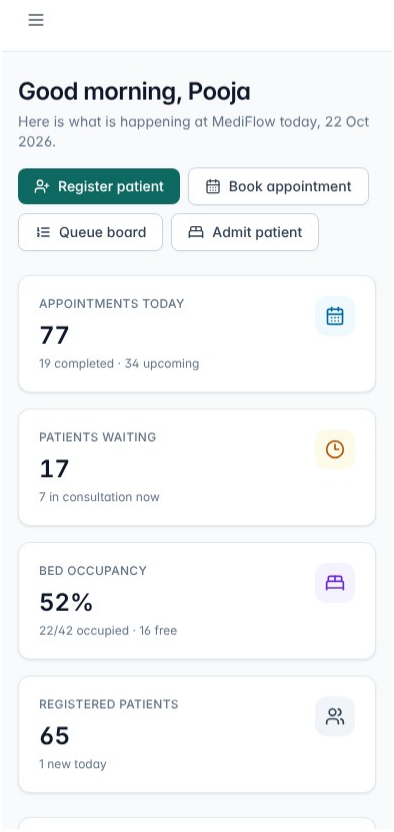


Figure 30: Responsive layout at 390 px (phone)

The module list in this part covers all 50 functional requirements; Appendix A traces each one to its module, screen and test cases.

PART C – SOFTWARE TEST PLAN

> This plan follows the structure of IEEE 829 / ISO/IEC/IEEE 29119-3. Test cases are automated and carry the identifier of the requirement they verify, so the plan, the test report and the traceability matrix are generated from the same source.

1. Testing Objectives

- Verify that each of the 50 functional requirements (FR-01 – FR-50) and the measurable non-functional requirements (NFR-01 – NFR-08) are correctly implemented.
- Verify every business rule (BR-01 – BR-12), especially the boundary conditions: slot limits, 5th failed login, 50% discount, 30-day expiry window, payment equal to balance, minimum one bed-day.
- Verify that invalid input and invalid state transitions are rejected with clear messages and without corrupting data (negative testing).
- Verify role-based access control: every role can perform its tasks and is denied everything else.
- Validate, from the users' point of view, the complete outpatient and inpatient journeys (acceptance).
- Find, record, fix and re-test defects, and provide objective evidence (actual results) for the evaluation of the system.

2. Testing Scope

2.1 In scope

- All thirteen modules listed in Part B, Section 2, through the REST API and through the user interface.
- Business rules and state machines (appointment, bed, admission, lab order, invoice).
- Security: authentication, lock-out, token tampering/expiry, logout revocation, RBAC, SQL-injection resistance, password storage, audit integrity.
- Performance of typical API requests on the demo dataset (NFR-01) and responsive layout at phone width (NFR-05).
- Data integrity of multi-step operations (transactions) and printing views (invoice, receipt).

2.2 Out of scope

- Load and soak testing at production scale (NFR-09, NFR-10) – requires production hardware.
- Automated cross-browser testing on Firefox and Safari (only Chromium is automated; other browsers are a release-candidate activity).
- Penetration testing by an independent party, accessibility audit (WCAG) and usability testing with real hospital staff.
- Third-party integrations, which are out of the product scope (payment gateway, lab analysers, SMS).

3. Testing Strategy

Testing follows the **V-model**: each development artefact has a matching test level, and higher levels are executed only when the lower level passes. All tests are automated so that the whole suite can be re-run after every change (regression testing).

Table 38: Test levels (V-model)

Level	Verifies	Scope and approach	Tool	Cases	Exit criterion
Unit	Detailed design (rules)	Pure domain-rule functions in server/domain (slots, conflicts, queue ETA, state machines, billing, inventory, patient validation, lock-out, permissions)	Vitest 3	50	100% pass
Integration	Architecture & module interfaces	REST API → middleware → services → SQLite (in-memory schema, frozen clock) for every module, NFR security/performance checks and robustness probes with invalid, boundary and malicious input (IT-EDGE-*)	Vitest + Supertest	92	100% pass
System	SRS (FR + NFR)	Complete application in a real browser: built SPA + API + fresh demo database, one test per module workflow, plus a crawl of every screen for all six roles at desktop and phone width (ST-UI-*)	Playwright (Chromium)	14	100% pass
Acceptance	User requirements	End-to-end business scenarios across several roles, validating the user requirements (UR-01 … UR-14)	Playwright (Chromium)	5	100% pass

3.1 Test design techniques

Table 39: Test design techniques

Technique	Where it is applied (examples)
Equivalence partitioning	Valid / invalid mobile numbers, genders, blood groups, payment methods (UT-PAT-02, UT-BIL-05).
Boundary value analysis	Lab reference limits 69/70/100/145 (UT-LAB-02); reorder level 21/20/0 (UT-PHA-01); expiry at 30/31 days (UT-PHA-02); discount 50% vs 60% (UT-BIL-02, IT-BIL-02); payment equal to balance vs +₹0.01 (UT-BIL-04); 4th vs 5th failed login (UT-AUTH-01); 1 h, 24 h, 25 h, 48 h stays (UT-IPD-03).
State-transition testing	Valid and invalid transitions of appointment, bed, admission, lab order and invoice (UT-APT-10, UT-IPD-01/02/04, UT-LAB-01, IT-LAB-02, IT-IPD-05).
Decision tables	Discharge clearance: uninvoiced charges × outstanding balance × role × override reason (UT-IPD-07, IT-IPD-07/08/09).
Use-case / scenario testing	Booking, queue, consultation, dispensing, admission-to-discharge, billing (ST-*, AT-*).
Error guessing & security	Forged/expired tokens, SQL injection strings, double submission, malformed JSON (IT-AUTH-05, IT-NFR-01, IT-NFR-04, IT-PHA-05).

3.2 Test environment

Table 40: Test environment

Item	Configuration
Hardware	Linux x86-64 container, 2 vCPU, 7 GB RAM (comparable to a mid-range laptop)
Software	Node.js v22.22.0, SQLite 3, Chromium 141 (Playwright), Vitest 3.2, Supertest 7
Unit / integration data	Fresh in-memory SQLite database per test file loaded with reference data (8 doctors, 12 lab tests, 26 medicines, 5 wards / 42 beds, 6 patients); clock frozen at Wed 21-Oct-2026 10:00 IST for deterministic results; bcrypt cost lowered to 4 for speed
System / acceptance data	Fresh demo database generated at server start: 64 patients, ~600 appointments over 14 days, ~380 invoices, 20 active admissions; real wall-clock time
Performance data	Demo database (IT-NFR-03) – 200 mixed API requests

3.3 Entry and exit criteria

- **Entry:** requirements baselined with IDs; code builds without errors; database schema creates successfully; test data generator runs.
- **Exit:** 100% of planned test cases executed; all test cases pass; no open defect of severity Critical or High; every FR covered by at least one passing test; domain-rule line coverage ≥ 80%.
- **Suspension:** the build fails or more than 20% of a suite fails because of an environment problem. **Resumption:** the cause is fixed and the full suite is re-run.

3.4 Defect management

Every failure is analysed first: if the test script or test data was wrong, the test is corrected and the incident is recorded in the run log (it is **not** a product defect). Product defects are logged with ID, severity, priority, steps to reproduce, expected and actual behaviour, root cause and resolution, and follow the life cycle **New → Assigned → Fixed → Re-tested → Closed** (or **Deferred** with justification). Each fixed defect is protected by a regression test referenced in the defect log.

Table 41: Defect severity levels (priority P1 = fix immediately … P4 = next release)

Severity	Definition
Critical	System unusable, data loss or security breach; no workaround.
High	A main function produces wrong results (e.g. wrong bill) or is blocked; workaround is difficult.
Medium	Function works with a wrong or confusing detail; workaround exists.
Low	Cosmetic, wording or minor usability issue.

3.5 Roles, schedule and deliverables

Table 42: Test activities and responsibilities

Activity	Responsibility	Output
Test planning and design	Test lead with requirements owner	This test plan; test case catalogue embedded in the test code
Unit & integration test implementation	Developers of each module	tests/unit, tests/integration
System & acceptance test implementation	Test engineer acting as end users of each role	tests/e2e, screenshots
Execution and defect logging	Test engineer	Run log, actual results, defect log
Re-testing and regression	Developers + test engineer	Updated run log
Test summary report	Test lead	Part D of this document

3.6 Risks and mitigations

Table 43: Test risks

Risk	Mitigation
Time-dependent rules (slots in the past, queue ETA, bed-days, session expiry) make tests flaky.	Central clock that tests freeze; E2E tests book only for the next day and use seeded same-day appointments.
Shared database between E2E tests causes order dependence.	Each E2E test creates or looks up its own data through the API before acting in the UI; fresh database for every run.
Tests written by the same team that wrote the code share its blind spots.	Boundary and negative cases derived from the requirements, not from the code; exploratory walkthroughs by members not involved in the module.
Short project schedule.	Full automation: the complete regression suite runs in about two minutes.

4. Unit Testing

Objective: verify each business rule in isolation. **Approach:** white-box tests of the pure functions in `server/domain`, applying boundary-value and state-transition techniques; no database or network is involved, so each test runs in milliseconds. **Coverage:** domain-rule modules 99.2% lines / 85.9% branches; whole server 95.45% lines / 84.15% branches (unit + integration suites, V8 coverage).

Table 44: Unit test coverage by module

Module under test	Rules verified	Cases
appointmentRules, time	Slot generation, working days, slot validation, overlap detection, free slots, appointment transitions	11
queueRules	Priority ordering, token sequence, ETA with and without consultation in progress	5
bedStateMachine, admissionRules	Bed transitions, bed-days, proportional allocation after transfer, discharge clearance	7
billingRules, money	Totals and discount rounding, discount limits, payment validation, invoice status, number formats	8
labRules, inventoryRules	Lab transitions, abnormal flag, low stock, expiry window, dispensability, alerts	7
patientRules, authRules, permissions	ID format, validation, duplicates, age, lock-out, password policy, RBAC matrix	12

5. Integration Testing

Objective: verify that the API, middleware, services, rule modules and database work together, including transactions, constraints and the audit log. **Approach:** grey-box tests that send real HTTP requests to the Express application (Supertest) backed by a fresh in-memory SQLite database with the production schema; users log in through the real login endpoint and act with their own tokens. Integration order is **bottom-up**: rule modules were unit-tested first, then each service was integrated through its API. Each test checks the HTTP status, the response body and, where relevant, side effects such as charges, stock levels, bed status and audit entries.

Table 45: Integration test areas

Area	Interfaces exercised	Cases
Authentication & users	/auth/*, /users/* – login, lock-out, deactivation, token tampering, logout, RBAC, user admin	10
Patients, doctors, appointments	/patients/*, /doctors/*, /appointments/*	15
Smart queue	/appointments/:id/check-in, /queue/*	7
Consultation, laboratory, pharmacy	/consultations/*, /lab/*, /pharmacy/* incl. charge creation and atomic dispensing	17
Admissions, beds, billing	/beds/*, /admissions/*, /billing/* incl. clearance and receipts	18
Reports, audit, NFRs	/reports/*, /audit, SQL injection, password storage, headers, latency	9
Robustness / edge cases	Impossible dates, malformed filters, wildcards, paging limits, over-long or wrong-type input, ownership and duplicate checks, sequence-number uniqueness, time-zone day boundaries	16

6. System Testing

Objective: verify the complete, integrated system against the SRS from the user interface. **Approach:** black-box tests with Playwright driving a real Chromium browser against the production build served by the Node.js server with a freshly generated demo database. Each test logs in through the login form as the relevant role and performs

a complete module workflow (e.g. book → check in → queue; generate invoice → partial payment → overpayment rejected → full payment → print receipt). Non-functional system checks include role-based menus and access-denied pages (security), responsive layout at 390 px (usability, NFR-05) and API latency (performance, NFR-01, measured in IT-NFR-03).

Table 46: System test cases

ID	System test
ST-APT-01	Book an appointment for tomorrow with the 4-step wizard
ST-AUTH-01	Login per role, role-based menus and blocked direct URLs
ST-BIL-01	Generate an invoice, take a partial UPI payment, reject an overpayment, settle and print the receipt
ST-CON-01	Doctor calls the next patient, records the consultation and completes it
ST-IPD-01	Admit a patient from the bed board and transfer to another bed
ST-LAB-01	Lab technician collects a sample and records an out-of-range result
ST-PAT-01	Register a patient through the form with validation feedback
ST-PHA-01	Pharmacist cannot dispense an out-of-stock prescription but dispenses a valid one
ST-QUE-01	Check in today's appointment and see the token on the queue board
ST-RPT-01	Administrator views dashboard, reports, users, doctors and the audit trail
ST-UI-01	Every screen renders for every role at desktop width without errors
ST-UI-02	All screens fit a 390 px phone without horizontal page scrolling
ST-UI-03	Broken or unauthorised deep links show a clear message instead of crashing
ST-UI-04	Double-clicking “Record payment” records only one payment

7. Acceptance Testing

Objective: confirm that the system satisfies the user requirements (Part A, Section 7) and is ready for use. **Approach:** scenario-based acceptance tests that follow a patient through several roles in one continuous flow, written from the acceptance criteria of the user stories. In this academic project the team acted as the hospital users; the scenarios are scripted and automated (Playwright) so that they are repeatable. A real deployment would add a supervised user-acceptance session with hospital staff and formal sign-off.

Table 47: Acceptance test scenarios

ID	Acceptance scenario	User requirements validated
AT-01	OPD patient journey: check-in → consultation → lab → pharmacy → billing	UR-02, UR-04, UR-05, UR-06, UR-07, UR-10
AT-02	IPD journey: admission → discharge request → billing clearance → discharge → bed turnaround	UR-08, UR-09, UR-10
AT-03	Each role sees exactly the menus it is entitled to	UR-13
AT-04	Usability: booking in at most 4 steps and no horizontal scrolling on a 390 px phone	
AT-05	Administrator can trace the journeys in the audit trail	UR-14

Acceptance criteria: all acceptance scenarios pass; no open Critical/High defects; every user requirement UR-01 … UR-14 is validated by at least one acceptance or system test.

PART D – SOFTWARE TEST REPORT

8. Test Cases

This section lists every executed test case with its expected and actual result and pass/fail status. The **actual result** column is not typed by hand: each automated test records what it observed (HTTP status, message, values) while running, and the report is generated from that log. Final execution: 22 Oct 2026, 04:15 am IST.

8.1 Unit test cases

Table 48: Unit test cases (50)

ID	Req.	Test case / input	Expected result	Actual result (recorded during execution)	Status
UT-APT-01	FR-15	Generates slots across the working day Input: Doctor 09:00–17:00, 15-min slots, Wednesday	32 slots, first 09:00–09:15, last 16:45–17:00	32 slots, first 09:00–09:15, last 16:45–17:00	**PASS**
UT-APT-02	FR-15	No slots on a non-working day Input: Doctor works Mon–Sat; date is Sunday 2026-10-25	Empty slot list; isWorkingDay = false	slots=0, isWorkingDay=false	**PASS**
UT-APT-03	FR-16	Rejects a time outside working hours Input: Start 17:00 and 08:45 for a 09:00–17:00 doctor	Validation error "outside the doctor's working hours"	17:00 → "Time is outside the doctor's working hours (09:00–17:00)"; 08:45 → "Time is outside the doctor's working hours (09:00–17:00)"	**PASS**
UT-APT-04	FR-16	Rejects a time that is not on a slot boundary Input: Start 09:07 with 15-minute slots	Validation error "must align to a 15-minute slot"	Time must align to a 15-minute slot	**PASS**
UT-APT-05	FR-16	Rejects booking a slot in the past Input: Now = 21 Oct 10:00; request 21 Oct 09:30	Validation error "Cannot book a slot in the past"	Cannot book a slot in the past	**PASS**
UT-APT-06	FR-16	Accepts a valid future slot Input: 21 Oct 11:15, doctor working	No validation problems	problems = []	**PASS**
UT-APT-07	FR-17	Detects an overlapping active appointment Input: Existing SCHEDULED 10:00–10:15; request 10:00–10:15	Conflict returned (appointment id 7)	conflict id = 7	**PASS**
UT-APT-08	FR-17, FR-18, FR-19	Ignores cancelled bookings and the appointment being rescheduled Input: CANCELLED 10:00 booking; SCHEDULED booking id 9 excluded	No conflict in both cases	cancelled → null, excluded self → null	**PASS**
UT-APT-09	FR-17	Adjacent slots do not overlap (half-open intervals) Input: [10:00,10:15) vs [10:15,10:30) and [10:10,10:25)	false for adjacent, true for overlapping	adjacent=false, overlapping=true	**PASS**
UT-APT-10	FR-19, FR-21, FR-25	Appointment state machine allows only defined transitions Input: SCHEDULED→CHECKED*IN*, *CHECKED*IN→IN*CONSULTATION*, *COMPLETED*→*SCHEDULED*, *CHECKED*IN→CANCELLED	true, true, false, false	true, true, false, false	**PASS**

ID	Req.	Test case / input	Expected result	Actual result (recorded during execution)	Status
UT-APT-11	FR-15	Free-slot list marks booked and past slots unavailable Input: Now 10:00; booking at 10:30	09:45 PAST, 10:30 BOOKED, 10:45 available	09:45=PAST, 10:30=BOOKED, 10:45 available=true	**PASS**
UT-AUTH-01	FR-02	Account locks on the 5th consecutive failed attempt Input: failed_attempts 3 then 4, then one more failure	4th failure: not locked; 5th: locked for 15 min	4th → locked_until=null; 5th → locked for 15 min	**PASS**
UT-AUTH-02	FR-02	Lock expires after 15 minutes and the counter restarts Input: locked_until = now+15 min; check at +14 and +16 min	locked, then unlocked; next failure counts as attempt 1	+14 min locked=true; +16 min locked=false; next attempt count=1	**PASS**
UT-AUTH-03	FR-03, FR-30	Role-permission matrix enforces least privilege Input: clinical:read for RECEPTIONIST/DOCTOR; patients:write for BILLING; billing:write for ADMIN; audit:read for ADMIN	false, true, false, false, true	false, true, false, false, true; LAB_TECH has 5 permissions	**PASS**
UT-AUTH-04	FR-05	Passwords must have at least 8 characters Input: "short" and "Mediflow@123"	Error, then null	Password must be at least 8 characters | ok	**PASS**
UT-BIL-01	FR-44, FR-45	Invoice totals with discount are rounded to paise Input: Lines ₹500, ₹450.50, ₹284; discount 10%	Subtotal ₹1234.50, discount ₹123.45, total ₹1111.05	subtotal 1234.5, discount 123.45, total 1111.05	**PASS**
UT-BIL-02	FR-45	Discount must be 0–50% and only on UNPAID invoices Input: 60% on UNPAID, −5% on UNPAID, 10% on PARTIALLY_PAID, 50% on UNPAID	error, error, error, ok	Discount must be between 0% and 50% | Discount must be between 0% and 50% | Discount can only be applied to an UNPAID invoice | ok	**PASS**
UT-BIL-03	FR-46	Payment amount must be greater than zero Input: Amounts 0 and −50 (CASH)	Error "must be greater than zero"	Payment amount must be greater than zero | Payment amount must be greater than zero	**PASS**
UT-BIL-04	FR-46	Overpayment beyond the outstanding balance is rejected Input: Outstanding ₹800; pay ₹800.01 and ₹800	Error for ₹800.01; ₹800 accepted	800.01 → "Payment exceeds the outstanding balance of ₹800.00"; 800 → null	**PASS**
UT-BIL-05	FR-46	Rejects unknown methods and payments on PAID or CANCELLED invoices Input: Method CHEQUE; PAID invoice; CANCELLED invoice	Three distinct errors	Payment method must be one of CASH, CARD, UPI | Invoice is already fully paid | Cannot pay a cancelled invoice	**PASS**
UT-BIL-06	FR-46	Invoice status follows payments Input: Total ₹900 with paid ₹0, ₹300, ₹900	UNPAID, PARTIALLY_PAID, PAID	UNPAID, PARTIALLY_PAID, PAID	**PASS**
UT-BIL-07	FR-44, FR-47	Invoice and receipt numbers follow the defined format Input: Date 20261021, sequence 7	INV-20261021-0007 and R-20261021-0007	INV-20261021-0007, R-20261021-0007	**PASS**
UT-BIL-08	FR-44	Money arithmetic avoids floating-point drift Input: sum(0.1, 0.2) and round2(1.005)	0.3 and 1.01	0.3, 1.01	**PASS**
UT-IPD-01	FR-41	Bed state machine accepts every defined transition Input: AVAILABLE→RESERVED→OCCUPIED→CLEANING→AVAILABLE, AVAILABLE ↔ MAINTENANCE, RESERVED→AVAILABLE	All transitions allowed	9/9 allowed	**PASS**
UT-IPD-02	FR-41	Bed state machine rejects undefined transitions Input: OCCUPIED→AVAILABLE, MAINTENANCE→OCCUPIED, CLEANING→OCCUPIED; manual AVAILABLE→OCCUPIED	All rejected	false, false, false, false	**PASS**

ID	Req.	Test case / input	Expected result	Actual result (recorded during execution)	Status
UT-IPD-03	FR-42	Bed days are rounded up with a 1-day minimum Input: Stays of 1 h, 24 h, 25 h, 48 h	1, 1, 2, 2 days	1, 1, 2, 2	**PASS**
UT-IPD-04	FR-42, FR-43	Admission life cycle allows only ADMITTED → DISCHARGE_PENDING → DISCHARGED Input: ADMITTED→DISCHARGE*PENDING*, *DISCHARGE*PENDING→DISCHARGED, ADMITTED→DISCHARGED	true, true, false	true, true, false	**PASS**
UT-IPD-05	FR-40, FR-42	Same-day transfer does not create an extra minimum bed-day (DEF-05 regression) Input: Ward A ₹1500 for 10 min, then Ward B ₹1500 for 5 h	Total 1 bed-day (₹1,500), not 2	days = [0, 1], total ₹1500	**PASS**
UT-IPD-06	FR-42	Bed-days are shared between stays in proportion to time spent Input: ICU ₹8000 for 30 h then General ₹1500 for 20 h (50 h total)	3 days total: ICU 2, General 1 → ₹17,500	days = [2, 1], total ₹17500	**PASS**
UT-IPD-07	FR-43	Discharge clearance requires settled bills unless an administrator overrides with a reason Input: (a) ₹500 outstanding (b) all paid (c) admin override "Patient transferred to AIIMS" (d) receptionist override	blocked, allowed, allowed+overridden, blocked	a=false (outstanding ₹500), b=true, c=true/true, d=false	**PASS**
UT-LAB-01	FR-32	Lab order life cycle allows only defined transitions Input: ORDERED→SAMPLE*COLLECTED*, *SAMPLE*COLLECTED→COMPLETED, ORDERED→COMPLETED, COMPLETED→CANCELLED	true, true, false, false	true, true, false, false	**PASS**
UT-LAB-02	FR-33	Numeric results outside the reference range are flagged abnormal (bounds inclusive) Input: FBS 70–100 mg/dL with values 69, 70, 100, 145	true, false, false, true	true, false, false, true	**PASS**
UT-LAB-03	FR-33	Text-only tests and non-numeric values are not auto-flagged Input: Chest X-ray (no range); value "abc" for FBS	null (not applicable) in both cases	[null,null]	**PASS**
UT-PAT-01	FR-07	Patient IDs use the P + 5 digits format Input: 1, 42, 12345	P00001, P00042, P12345	P00001, P00042, P12345	**PASS**
UT-PAT-02	FR-08	Required fields and a valid 10-digit mobile number are enforced Input: Empty names, phone "12345", gender "X"	Errors for first*name*, *last*name, phone, gender, dob	first*name*, *last*name, phone, gender, dob	**PASS**
UT-PAT-03	FR-08	Date of birth in the future is rejected Input: DOB 2026-12-01 when today is 2026-10-21	Error "Date of birth cannot be in the future"	Date of birth cannot be in the future	**PASS**
UT-PAT-04	FR-09	Duplicate detection matches phone+DOB or name+DOB (case-insensitive) Input: Existing Riya Ghosh 1990-05-01 9876543210; candidates with same phone, same name in caps, different DOB	Match, match, no match	1, 1, 0	**PASS**

ID	Req.	Test case / input	Expected result	Actual result (recorded during execution)	Status
UT-PAT-05	FR-12	Age is computed correctly around the birthday Input: DOB 2000-10-21 and 2000-10-22 on 2026-10-21	26 and 25	26, 25	**PASS**
UT-PAT-06	FR-08	Only real calendar dates are accepted Input: 2026-02-28, 2028-02-29 (leap), 2026-02-30, 2026-13-01, 0000-00-00	true, true, false, false, false	true, true, false, false, false	**PASS**
UT-PAT-07	FR-08	Mobile numbers typed in common formats are normalised to 10 digits Input: "+91 98765 43210", "098765-43210", "(98765) 43210", "12345"	9876543210 ×3, "12345" unchanged (later rejected)	9876543210, 9876543210, 9876543210, 12345	**PASS**
UT-PAT-08	FR-10	Paging values are clamped and search wildcards are escaped Input: clampInt(-5,50,1,500), clampInt("abc",50,1,500), clampInt(99999,50,1,500); likeTerm("50%_off")	1, 50, 500; "%50\%_off%"	1, 50, 500; %50\%_off%	**PASS**
UT-PHA-01	FR-36	Low stock when quantity is at or below the reorder level Input: Reorder level 20 with stock 21, 20, 0	false, true, true	false, true, true	**PASS**
UT-PHA-02	FR-36	Expiry status uses a 30-day warning window Input: Expiry 2026-10-20, 2026-11-20 (30 d), 2026-11-21 (31 d)	EXPIRED, NEAR_EXPIRY, OK	EXPIRED, NEAR_EXPIRY, OK	**PASS**
UT-PHA-03	FR-37	Dispensing is refused for insufficient stock, expired batches and invalid quantities Input: Stock 3 & qty 5; expired batch; qty 0	Three errors; valid case returns null	Insufficient stock for Insulin Glargine: 3 available, 5 required | Insulin Glargine is expired and cannot be dispensed | Quantity must be a positive whole number | ok	**PASS**
UT-PHA-04	FR-36	Stock alerts list out-of-stock, low-stock, near-expiry and expired items Input: 4 medicines in different states	Alert types OUT*OF*STOCK, LOW*STOCK*, *NEAR*EXPIRY, EXPIRED	EXPIRED, LOW*STOCK*, *NEAR*EXPIRY, OUT*OF*STOCK	**PASS**
UT-QUE-01	FR-22, FR-24	Queue orders URGENT first, then by check-in time Input: A normal 09:01, B normal 09:05, C urgent 09:10	Order C, A, B	C, A, B	**PASS**
UT-QUE-02	FR-21	Token numbers are sequential per doctor per day Input: Existing tokens [] and [1,2,5]	1 and 6	1 and 6	**PASS**
UT-QUE-03	FR-23	Wait estimate without a consultation in progress Input: 3 waiting, average 15 min, nobody in consultation	ETA 0, 15, 30 minutes	0, 15, 30	**PASS**
UT-QUE-04	FR-23	Wait estimate includes remaining time of current consultation Input: 2 waiting, average 15 min, current consultation started 5 min ago	ETA 10 and 25 minutes	10, 25	**PASS**
UT-QUE-05	FR-23	Remaining time never drops below 1 minute when a consultation overruns Input: Average 15 min, consultation running for 40 min	Remaining = 1	remaining = 1	**PASS**

8.2 Integration test cases

Table 49: Integration test cases (92)

ID	Req.	Test case / input	Expected result	Actual result (recorded during execution)	Status
IT-APT-01	FR-16	A valid appointment is booked in the SCHEDULED state Input: Dr. Sharma, 2026-10-22 11:00	201, status SCHEDULED, end time 11:15	201 SCHEDULED 11:00–11:15	**PASS**
IT-APT-02	FR-16	Bookings in the past or outside working hours are rejected Input: Today 09:00 (now 10:00); tomorrow 18:00	400 "Cannot book a slot in the past"; 400 "outside the doctor's working hours"	400 "Cannot book a slot in the past"; 400 "Time is outside the doctor's working hours (09:00–17:00)"	**PASS**
IT-APT-03	FR-17	Double-booking a doctor's slot is rejected with a clear message Input: Another patient books Dr. Sharma 2026-10-22 11:00	409 DOCTOR*SLOT*TAKEN "Dr. Arjun Sharma already has an appointment at 11:00…" (no "Dr. Dr.")	409 DOCTOR*SLOT*TAKEN "Dr. Arjun Sharma already has an appointment at 11:00 on 2026-10-22"	**PASS**
IT-APT-04	FR-17	A patient cannot be booked with two doctors at the same time Input: Same patient, Dr. Banerjee 2026-10-22 11:00 (overlaps Sharma 11:00–11:15)	409 PATIENT*DOUBLE*BOOKED	409 PATIENT*DOUBLE*BOOKED "The patient already has an appointment at 11:00 on 2026-10-22"	**PASS**
IT-APT-05	FR-18	Rescheduling moves the booking and frees the old slot Input: PATCH reschedule to 2026-10-22 12:30	200 new time 12:30; slot 11:00 available again	200 → 12:30; 11:00 available=true	**PASS**
IT-APT-06	FR-19	Cancellation needs a reason and only applies to SCHEDULED appointments Input: Cancel without reason; cancel with reason "Patient travelling"; cancel again	400; 200 CANCELLED; 409 INVALID_STATE	400 "reason: Cancellation reason is required"; 200 CANCELLED; 409 INVALID_STATE	**PASS**
IT-APT-07	FR-20	Appointments can be filtered by date, doctor and status Input: GET /appointments?date=2026-10-22&doctorId=sharma&status=CANCELLED	Only the cancelled appointment is returned	1 row(s): #1 CANCELLED	**PASS**
IT-AUD-01	FR-50, NFR-03	State changes are audited with user and role, and passwords never appear Input: Register patient, create user with password "Secret@999", then search the audit log	CREATE Patient by reception (RECEPTIONIST); CREATE User by admin; no entry contains "Secret@999"	patient entry: CREATE by reception (RECEPTIONIST); user entry: "Created LAB_TECH account "audit.user""; password leaked=false	**PASS**
IT-AUD-02	FR-50	The audit log is append-only at database level Input: Direct SQL UPDATE and DELETE on audit_logs	Both rejected with "audit log is append-only"	UPDATE → "audit log is append-only"; DELETE → "audit log is append-only"	**PASS**
IT-AUD-03	FR-50, FR-03	Only administrators can read the audit trail Input: GET /audit as billing and doctor	403, 403	403, 403	**PASS**
IT-AUTH-01	FR-01	Valid credentials return a session token and the role permissions Input: POST /auth/login reception / Mediflow@123	200 with JWT, role RECEPTIONIST and permissions list	HTTP 200, role RECEPTIONIST, 13 permissions, token length 228	**PASS**
IT-AUTH-02	FR-01	Wrong password is rejected with a generic message Input: POST /auth/login billing / wrongpass	401 "Invalid username or password" (no hint which part was wrong)	HTTP 401 "Invalid username or password"; unknown user HTTP 401 "Invalid username or password"	**PASS**

ID	Req.	Test case / input	Expected result	Actual result (recorded during execution)	Status
IT-AUTH-03	FR-02	Five consecutive failures lock the account, even for the correct password Input: 5 × wrong password for "reception2", then correct password	5th attempt → 423 locked; correct password → 423 locked	attempts → 401, 401, 401, 401, 423; correct password → 423 "Account locked after repeated failed attempts. Try again in 15 minute(s)."	**PASS**
IT-AUTH-04	FR-02, FR-06	Deactivated accounts cannot log in and their sessions end Input: Admin deactivates "pharmacy"; pharmacy uses old token, then logs in	Old token → 401; login → 401 "deactivated"	deactivate 200; old token 401; login 401 "This account has been deactivated. Contact the administrator."	**PASS**
IT-AUTH-05	FR-04	Missing, tampered and expired tokens are rejected Input: GET /patients with no token, a token signed with another secret, and an expired token	401 in all three cases	no token 401, forged 401, expired 401	**PASS**
IT-AUTH-06	FR-04	Logout revokes the session token Input: Login as lab, POST /auth/logout, reuse token	Logout 200; reused token → 401	logout 200; reuse 401 "Session has ended. Please log in again."	**PASS**
IT-AUTH-07	FR-03	Role-based access control blocks operations outside each role Input: Receptionist GET /users; Billing POST /patients; Lab GET /billing/invoices; Pharmacist POST /appointments	403 for all four	403, 403, 403, 403	**PASS**
IT-BIL-01	FR-44	Invoice is generated from all unbilled OPD charges; nothing left to bill afterwards Input: POST /billing/invoices for Sohini Pal (consultation ₹500 + cholesterol ₹300); repeat	201 total ₹800, 2 lines, UNPAID; repeat → 400 "no unbilled charges"	201 INV-20261021-0002 ₹800 (2 lines, UNPAID); repeat 400 "There are no unbilled charges for this patient"	**PASS**
IT-BIL-02	FR-45	Discounts above 50% are rejected; a 10% discount recalculates the total Input: discount_pct 60 then 10 with reason "Senior citizen"	400; 200 total ₹720, discount ₹80	400 "Discount must be between 0% and 50%"; 200 total ₹720, discount ₹80	**PASS**
IT-BIL-03	FR-46	A partial payment moves the invoice to PARTIALLY_PAID Input: Pay ₹200 CASH	201 receipt; status PARTIALLY_PAID; balance ₹520	201 R-20261021-0001; PARTIALLY_PAID; balance ₹520	**PASS**
IT-BIL-04	FR-46	Overpayments and card/UPI payments without a reference are rejected Input: Pay ₹600 (balance ₹520); pay ₹100 UPI without reference	400 "exceeds the outstanding balance of ₹520.00"; 400 "reference is required"	400 "Payment exceeds the outstanding balance of ₹520.00"; 400 "A transaction reference is required for UPI payments"	**PASS**
IT-BIL-05	FR-46	Paying the balance marks the invoice PAID; no further payments or discounts Input: Pay ₹520 UPI ref UPI881234; then pay ₹1; then discount 5%	PAID; 400 "already fully paid"; 400 discount only on UNPAID	PAID; 400 "Invoice is already fully paid"; 400 "Discount can only be applied to an UNPAID invoice"	**PASS**
IT-BIL-06	FR-47	Every payment gets a unique, sequential receipt number Input: Receipts of the two payments	R-20261021-NNNN numbers, strictly increasing and unique	R-20261021-0001, R-20261021-0002	**PASS**
IT-BIL-07	FR-47	A receipt shows the balance right after that payment (DEF-02 regression) Input: GET /billing/receipts/{first payment}	Amount ₹200, balance after ₹520 (not ₹0)	R-20261021-0001: ₹200, balance_after ₹520	**PASS**

ID	Req.	Test case / input	Expected result	Actual result (recorded during execution)	Status
IT-BIL-08	FR-44	Cancelling an unpaid invoice releases its charges; paid invoices cannot be cancelled Input: Cancel the paid invoice; create an invoice for another visit and cancel it	409 for the paid invoice; 200 CANCELLED and charges billable again	paid invoice cancel 409; INV-20261021-0003 → CANCELLED; 1 charge(s) billable again	**PASS**
IT-BIL-09	FR-03	Only billing staff can create invoices or take payments Input: Receptionist POST /billing/invoices; Admin POST payment	403, 403	403, 403	**PASS**
IT-CON-01	FR-26	Doctor saves vitals, symptoms and diagnosis; malformed vitals are rejected Input: PUT /consultations/{id} with BP "120-80"; then BP 128/84, pulse 88, diagnosis "Acute viral fever"	400 "BP must look like 120/80"; then 200 with saved values	400 "bp: BP must look like 120/80"; 200 BP 128/84, diagnosis "Acute viral fever"	**PASS**
IT-CON-02	FR-26	A consultation cannot be completed without a diagnosis Input: Second patient: call next, then complete with empty diagnosis	400 "Diagnosis is required to complete the consultation"	400 "Diagnosis is required to complete the consultation"	**PASS**
IT-CON-03	FR-27	Prescriptions need a positive quantity; a valid prescription is saved Input: Items: Paracetamol qty 0; then Paracetamol 1-0-1 × 5 days qty 10 and Pantoprazole 1-0-0 × 5 qty 5	400 "Quantity must be greater than 0"; then 200 with 2 PENDING items	400 "items.0.quantity: Quantity must be greater than 0"; 200 prescription #1 PENDING with 2 items	**PASS**
IT-CON-04	FR-28, FR-34	Ordering lab tests during a consultation creates orders and unbilled charges Input: POST /lab/orders FBS + HB (STAT) for the open consultation	201 with 2 ORDERED orders; 2 LAB charges (₹100 + ₹150) unbilled	201: FBS ORDERED, HB ORDERED; charges 100 + 150	**PASS**
IT-CON-05	FR-29	Completing a consultation closes the visit and charges the consultation fee Input: Save diagnosis "Type 2 diabetes mellitus", POST complete	Appointment COMPLETED; CONSULTATION charge ₹500 unbilled; further edits rejected (409)	200 consultation COMPLETED; appointment COMPLETED; fee ₹500; late edit 409	**PASS**
IT-CON-06	FR-30	Clinical notes are hidden from non-clinical roles Input: Receptionist and Billing GET /consultations/{id}; Receptionist GET /patients/{id}/history	403, 403; history without consultations/prescriptions	403, 403; receptionist history has consultations=false; doctor sees 1 consultation(s)	**PASS**
IT-DOC-01	FR-13	Administrator adds a doctor with a login; invalid hours are rejected Input: POST /doctors (Dermatology, 10:00–14:00); again with end 09:00 before start 10:00	201 with department name; 400 "Work end time must be after work start time"	201 Dr. Ishan Paul (Dermatology); 400 "Work end time must be after work start time"	**PASS**
IT-DOC-02	FR-15	Free-slot listing marks the booked slot unavailable Input: GET /doctors/{sharma}/slots?date=2026-10-22	11:00 → BOOKED; 11:15 → available	11:00 BOOKED; 11:15 available=true; 32 slots	**PASS**
IT-DOC-03	FR-14	Departments are listed with their doctors Input: GET /departments; GET /doctors?departmentId={Cardiology}	8+ departments with doctor counts; only Cardiology doctors returned	8 departments (Cardiology has 1 doctor(s)); filter → Dr. Meera Banerjee (Cardiology)	**PASS**

ID	Req.	Test case / input	Expected result	Actual result (recorded during execution)	Status
IT-EDGE-01	FR-08, FR-15, FR-16, FR-35	Impossible calendar dates are rejected everywhere Input: Book on 2027-02-30; slots for 2026-13-01; DOB 1990-02-30; restock expiry 2027-02-31	400 "Not a valid calendar date" (or DOB equivalent) in all four cases; nothing stored	booking 400 "date: Not a valid calendar date"; slots 400; DOB 400 "Date of birth is not a valid calendar date"; restock 400 "expiry_date: Not a valid calendar date"	**PASS**
IT-EDGE-02	FR-20, FR-22, FR-49, FR-50	Malformed date filters return 400 instead of a server error Input: reports ?from=abc&to=xyz; invoices ?from=abc; audit ?from=2026-99-99; queue ?date=xyz; appointments ?date=2026-02-31	400 with "must be a valid date" for every request (previously the report and list endpoints crashed with 500)	400, 400, 400, 400, 400	**PASS**
IT-EDGE-03	FR-49	Report period must be ordered and at most 366 days Input: from=2026-10-30&to=2026-10-01; from=2024-01-01&to=2026-10-01; from=2026-10-01&to=2026-10-21	400 "from … must not be after …"; 400 "cannot exceed 366 days"; 200 with 21 daily rows	400 ""from" date must not be after "to" date"; 400 "The report period cannot exceed 366 days"; 200 21 days	**PASS**
IT-EDGE-04	FR-08	Patient fields must be text and within length limits Input: first*name as array ["a"]*; *first*name of 5,000 characters	400 "Must be text"; 400 "Must be at most 60 characters"	400 "Must be text"; 400 "Must be at most 60 characters"	**PASS**
IT-EDGE-05	FR-08, FR-10	Mobile numbers typed with +91, spaces or a leading 0 are normalised Input: Register with phone "+91 98765-43210"; search "9876543210"	201 with phone stored as 9876543210; search finds the patient	201 phone=9876543210; search → P00008	**PASS**
IT-EDGE-06	FR-10, FR-50	Search terms containing % or _ are matched literally Input: GET /patients?q=% and /audit?q=%	No rows (wildcards no longer match everything)	patients 0, audit 0	**PASS**
IT-EDGE-07	FR-10, FR-20	Negative or non-numeric limit / offset values are clamped Input: /patients?limit=-5; /appointments?limit=-1; /audit?limit=abc&offset=-1	200; limit=-5 returns at most 1 row (SQLite treats a negative LIMIT as "no limit")	patients 200 rows=1; appointments 200 rows=0; audit 200 rows=9	**PASS**
IT-EDGE-08	FR-39	Non-text and over-long free-text values are rejected Input: Admission reason = 42; reason of 2,000 characters	400 "must be text"; 400 "at most 1000 characters"	400 "reason: Reason for admission must be text"; 400 "reason: Reason for admission must be at most 1000 characters"	**PASS**
IT-EDGE-09	FR-19	A no-show cannot be recorded before the appointment has started Input: Today 15:00 appointment, clock 10:00: POST no-show	400 "can only be marked as no-show after its start time"; status stays SCHEDULED	400 "The appointment starts at 15:00; it can only be marked as no-show after its start time"; status SCHEDULED	**PASS**
IT-EDGE-10	FR-13, FR-16	Appointments cannot be booked with a deactivated doctor Input: Admin deactivates a newly added doctor; reception books tomorrow 10:00	409 DOCTOR_INACTIVE	409 DOCTOR_INACTIVE "Dr. On Leave is not currently available for appointments (account inactive)"	**PASS**
IT-EDGE-11	FR-28, FR-03	Lab orders: only on the doctor's own consultation, duplicates ordered once Input: Dr. Banerjee orders on Dr. Sharma's consultation; Dr. Sharma orders [FBS, FBS]; then FBS again	403; 201 with one FBS order; 409 DUPLICATE*LAB*ORDER	other doctor 403; [FBS,FBS] → 201 1 order(s); again 409 DUPLICATE*LAB*ORDER	**PASS**
IT-EDGE-12	FR-27	The same medicine cannot appear twice in one prescription Input: Prescription with two Paracetamol lines	400 "Each medicine can appear only once…"	400 "Each medicine can appear only once in a prescription – adjust the quantity instead"	**PASS**
IT-EDGE-13	FR-44, FR-46	A zero-value invoice is settled automatically instead of staying UNPAID Input: Free-camp doctor (fee ₹0) completes a consultation; billing generates the invoice	Invoice total ₹0 with status PAID (a ₹0 invoice could never be paid otherwise)	201 INV-20261021-0001 total ₹0 status PAID	**PASS**

ID	Req.	Test case / input	Expected result	Actual result (recorded during execution)	Status
IT-EDGE-14	FR-39, FR-44, FR-47	Admission, invoice and receipt numbers stay unique when older numbers are out of sequence Input: Database already holds ADM-20261021-005 (discharged), INV-20261021-0009 and R-20261021-0009 from an earlier import; create one more of each	New numbers ADM-…-006, INV-…-0010, R-…-0010 – continuing after the highest existing number (a COUNT-based counter gives 002/0002 here and collides with an existing number once the count catches up)	201 ADM-20261021-006; 201 INV-20261021-0010; 201 R-20261021-0010	**PASS**
IT-EDGE-15	FR-43, FR-44	Cancelling the admission invoice re-blocks discharge Input: Discharge-pending admission whose invoice is cancelled; reception tries to discharge	409 BILLING*NOT*CLEARED because the released charges are unbilled again	409 BILLING*NOT*CLEARED "Discharge blocked: 1 charge(s) have not been invoiced yet"	**PASS**
IT-EDGE-16	FR-31	Lab worklist date filter uses the local (IST) day, not the UTC day Input: Clock 22-Oct-2026 01:00 IST (21-Oct 19:30 UTC); doctor orders TSH; filter date=2026-10-22 and date=2026-10-21	Order listed for 2026-10-22, not for 2026-10-21	order 3 created_at 2026-10-21T19:30:00.000Z; in 22-Oct list=true, in 21-Oct list=false	**PASS**
IT-IPD-01	FR-38	Bed board shows every ward with bed status counts Input: GET /beds on reference data (5 wards, 42 beds)	5 wards, 42 beds, all AVAILABLE, occupancy 0%	5 wards, 42 beds, AVAILABLE 42, occupancy 0%	**PASS**
IT-IPD-02	FR-39	Admitting a patient occupies the bed; a second active admission is refused Input: Admit Kamal Nath to GA-01; admit again to GA-02	201 ADMITTED, GA-01 OCCUPIED; 409 ALREADY_ADMITTED	201 ADM-20261021-001 ADMITTED; GA-01 OCCUPIED; again 409 ALREADY_ADMITTED	**PASS**
IT-IPD-03	FR-39	A bed that is occupied or under maintenance cannot be assigned Input: Admit another patient to GA-01 (occupied) and to GA-03 after marking it MAINTENANCE	409 BED*NOT*AVAILABLE twice	occupied → 409 BED*NOT*AVAILABLE; maintenance → 409 BED*NOT*AVAILABLE	**PASS**
IT-IPD-04	FR-40	Transfer moves the patient and sends the old bed for cleaning Input: 10 minutes after admission, transfer to GB-01	Admission bed GB-01; GA-01 CLEANING; GB-01 OCCUPIED; 2 bed stays	200 now General Ward B GB-01; GA-01 CLEANING; GB-01 OCCUPIED; stays 2	**PASS**
IT-IPD-05	FR-41	Manual bed status changes follow the state machine Input: GA-01 CLEANING → OCCUPIED (manual); GA-01 CLEANING → AVAILABLE	409 INVALID*BED*TRANSITION; 200 AVAILABLE	409 INVALID*BED*TRANSITION; 200 AVAILABLE	**PASS**
IT-IPD-06	FR-42	Discharge request adds one bed-day after a same-day transfer (DEF-05 regression) Input: 5 hours later Dr. Sharma requests discharge with summary	DISCHARGE_PENDING; BED charges total ₹1,500 (1 day), no zero-amount lines	short summary 400; 200 DISCHARGE_PENDING; bed lines: Bed charges – General Ward B GB-01 (1 day @ ₹1500); total ₹1500	**PASS**
IT-IPD-07	FR-43	Discharge is blocked until all admission charges are invoiced and paid Input: Reception discharges before invoicing; Billing invoices; Reception discharges with balance due	409 BILLING*NOT*CLEARED (uninvoiced), then 409 (outstanding ₹1,500)	409 "Discharge blocked: 1 charge(s) have not been invoiced yet"; invoice INV-20261021-0001 ₹1500; 409 "Discharge blocked: outstanding balance of ₹1500.00 must be paid"	**PASS**
IT-IPD-08	FR-43	Only an administrator can override billing clearance, and must give a reason Input: Receptionist sends override_reason; Admin overrides with "Transferred to SSKM for surgery"	403; 200 DISCHARGED with DISCHARGE_OVERRIDE audit entry and bed CLEANING	receptionist 403; admin 200 DISCHARGED; audit "DISCHARGE_PENDING → DISCHARGED (ADM-20261021-001); override: Transferred to SSKM for surgery; outstanding ₹1500; bed General Ward B GB-01 → CLEANING"; GB-01 CLEANING	**PASS**

ID	Req.	Test case / input	Expected result	Actual result (recorded during execution)	Status
IT-IPD-09	FR-43	With bills fully paid, discharge completes without override Input: New admission → discharge request → invoice → full payment → discharge	200 DISCHARGED, clearance.allowed = true	invoice ₹9000 (2 days × ₹4500); discharge 200 DISCHARGED	**PASS**
IT-LAB-01	FR-31	Lab worklist can be filtered by status Input: Lab tech GET /lab/orders?status=ORDERED	The two new orders, STAT first	2 order(s): FBS/STAT, HB/STAT	**PASS**
IT-LAB-02	FR-32	Result entry is refused before the sample is collected Input: POST /lab/orders/{FBS}/result directly from ORDERED	409 INVALID_STATE	409 INVALID_STATE "Cannot enter a result for an order that is ORDERED"	**PASS**
IT-LAB-03	FR-32, FR-33	Collect sample then record result; out-of-range values are flagged abnormal Input: FBS: collect, result 162 mg/dL (ref 70–100)	SAMPLE*COLLECTED then COMPLETED with i*sabnormal = 1	collect SAMPLE_COLLECTED; result COMPLETED, value 162 mg/dL, abnormal=1	**PASS**
IT-LAB-04	FR-32	Only lab technicians can process samples Input: Doctor POST /lab/orders/{HB}/collect	403	403	**PASS**
IT-LAB-05	FR-34	Cancelling an ORDERED test removes its unbilled charge Input: Cancel HB order with reason "Duplicate order"	Status CANCELLED; LAB charge for HB no longer unbilled	CANCELLED; remaining LAB charges: Lab test – Fasting Blood Sugar (FBS)	**PASS**
IT-LAB-06	FR-31	Lab status counters can be limited to one doctor (DEF-07 regression) Input: GET /lab/stats?doctorId={sharma} vs {banerjee}	Sharma: 1 COMPLETED + 1 CANCELLED; Banerjee: no orders	sharma {"CANCELLED":1,"COMPLETED":1}; banerjee {}	**PASS**
IT-NFR-01	NFR-02	SQL injection attempts are treated as plain text Input: GET /patients?q=' OR 1=1 -- and q=x'; DROP TABLE patients; --	200 with 0 matches; patients table intact	200 0 rows; 200 0 rows; patients in DB 65	**PASS**
IT-NFR-02	NFR-02	Passwords are stored only as bcrypt hashes and never returned by the API Input: Inspect users.password_hash; GET /users as admin	Hashes start with $2; API rows have no password field	15 users, all bcrypt=true; API exposes password_hash=false	**PASS**
IT-NFR-03	NFR-01	95% of typical API requests finish within 500 ms on the demo dataset Input: 200 mixed requests (dashboard, queue, bed board, patient search, invoices, reports) on ~600 appointments / ~380 invoices	p95 latency < 500 ms	p50 3.9 ms, p95 5.1 ms, max 8.1 ms over 200 requests	**PASS**
IT-NFR-04	NFR-02	Security headers are sent and malformed JSON is handled Input: GET /api/health; POST /auth/login with body "{bad json"	X-Content-Type-Options nosniff, X-Frame-Options DENY; 400 BAD_JSON	nosniff=nosniff, frame=DENY; bad JSON → 400 undefined	**PASS**
IT-PAT-01	FR-07	Registering a patient generates the next sequential patient ID Input: POST /patients with valid details (6 patients already exist)	201, patient_code P00007, then P00008	201 P00007, 201 P00008	**PASS**
IT-PAT-02	FR-08	Invalid patient data is rejected with field-level messages Input: phone "98765", dob 2030-01-01, gender missing	400 with details for phone, dob and gender	400; fields: phone, gender, dob	**PASS**
IT-PAT-03	FR-09	Possible duplicate registration is flagged and can be confirmed Input: Register Arjun Bose again with same DOB; then resend with confirmDuplicate=true	409 POSSIBLE_DUPLICATE listing P00007; confirmed → 201	409 POSSIBLE_DUPLICATE → P00007; confirmed 201 P00009	**PASS**

ID	Req.	Test case / input	Expected result	Actual result (recorded during execution)	Status
IT-PAT-04	FR-10	Patients can be searched by ID, name and mobile number Input: q = "P00008", "mitali", phone of P00007	Each search returns the matching patient first	code → P00007; name → P00008; phone → P00007	**PASS**
IT-PAT-05	FR-11	Patient details can be updated but the patient ID cannot change Input: PATCH address; PATCH patient_code = P99999	200 with new address; 400 "Patient ID cannot be changed"	200 address="12 Lake Road, Kalyani"; 400 "Patient ID cannot be changed"	**PASS**
IT-PHA-01	FR-35	Pharmacist adds a medicine and restocks it; expired batches are refused Input: Add "Doxycycline 100 mg" stock 50; restock +100 batch DX2027 exp 2027-08-31; restock with expiry 2026-10-01	201 stock 50; 200 stock 150; 400	201 stock 50; 200 stock 150; 400 "Cannot stock a batch that is already expired"	**PASS**
IT-PHA-02	FR-36	Stock alerts report low-stock, out-of-stock and expiring medicines Input: GET /pharmacy/alerts on reference data	Includes OUTOFSTOCK Montelukast, LOW*STOCK Insulin Glargine*, *EXPIRED Betamethasone*, *NEAR*EXPIRY Cough Syrup DX	7 alerts; true, true, true, true	**PASS**
IT-PHA-03	FR-37, NFR-04	Dispensing is all-or-nothing when one item is out of stock Input: Prescription: Paracetamol 10 + Insulin Glargine 5 (only 3 in stock)	409 CANNOT_DISPENSE; Paracetamol stock unchanged; prescription still PENDING	409 CANNOT_DISPENSE "Insufficient stock for Insulin Glargine: 3 available, 5 required"; paracetamol 900 → 900; prescription PENDING	**PASS**
IT-PHA-04	FR-37	Dispensing a valid prescription reduces stock and adds medicine charges Input: Dispense the prescription from IT-CON-03 (Paracetamol 10, Pantoprazole 5)	DISPENSED; stock −10 and −5; MEDICINE charges ₹20 and ₹30	200 DISPENSED; paracetamol 900→890, pantoprazole 600→595; charges 20, 30	**PASS**
IT-PHA-05	FR-37, FR-03	Only pharmacists can dispense; a prescription cannot be dispensed twice Input: Doctor dispenses; pharmacist dispenses the same prescription again	403; 409	403; 409 "Prescription is already DISPENSED"	**PASS**
IT-QUE-01	FR-21	Check-in issues sequential tokens per doctor per day Input: Check in three of Dr. Sharma's appointments for today	Tokens 1, 2, 3 and status CHECKED_IN	1:CHECKED*IN*, *2:CHECKED*IN, 3:CHECKED_IN	**PASS**
IT-QUE-02	FR-21	Only today's SCHEDULED appointments can be checked in Input: Check in a booking for tomorrow; check in token 1 again	400 (not today); 409 (already checked in)	400 "Only today's appointments can be checked in (this one is on 2026-10-22)"; 409 "Cannot check in an appointment that is CHECKED_IN"	**PASS**
IT-QUE-03	FR-22, FR-23	Queue board lists waiting patients in order with wait estimates Input: GET /queue/{sharma} with 3 waiting, average 12 min, no consultation in progress	Tokens 1,2,3 with ETA 0, 12, 24 min	#1 ETA 0, #2 ETA 12, #3 ETA 24	**PASS**
IT-QUE-04	FR-24, FR-50	Marking a patient URGENT needs a reason, moves them to the front and is audited Input: Mark token 3 urgent without reason, then with "Chest pain"	400; then token order 3,1,2 and an audit entry PRIORITY_URGENT	400; 200 URGENT; order 3,1,2; audit "Marked P00013 (token 3) URGENT: Chest pain"	**PASS**
IT-QUE-05	FR-25	Doctor calls the next patient; a second call while busy is refused Input: Dr. Sharma POST /queue/{id}/call-next twice	200 token 3 → IN*CONSULTATION; then 409 CONSULTATION*IN_PROGRESS	200 token 3 IN*CONSULTATION; 409 CONSULTATION*IN_PROGRESS	**PASS**

ID	Req.	Test case / input	Expected result	Actual result (recorded during execution)	Status
IT-QUE-06	FR-25, FR-03	A doctor cannot call patients from another doctor's queue Input: Dr. Banerjee POST /queue/{sharma}/call-next	403	403 "Doctors can only call patients from their own queue"	**PASS**
IT-QUE-07	FR-23	Wait estimates account for the consultation in progress Input: 5 minutes after the call, board for Dr. Sharma (avg 12)	Remaining waiting tokens 1,2 with ETA 7 and 19 min	in progress token 3; #1 ETA 7, #2 ETA 19	**PASS**
IT-RPT-01	FR-48	Dashboard KPIs are tailored to the role Input: GET /reports/dashboard as admin, receptionist, doctor, pharmacist	Admin: all sections; receptionist: no billing; doctor: own appointments only; pharmacist: pharmacy only	admin keys [date,role,appointments,appointmentTrend,newPatientsToday, totalPatients,beds,wards,admissions,lab,pharmacy,billing,revenueTrend,users,recentActivity]; reception billing=false; doctor today=12 vs all=78; pharmacist appointments=false, pharmacy=true	**PASS**
IT-RPT-02	FR-49	Management report aggregates a date range; restricted to administrators Input: GET /reports/summary?from=2026-10-08&to=2026-10-21 as admin and as billing	14 daily rows, revenue = sum of daily revenue, top diagnoses present; billing → 403	14 days; totals: 510 appointments, revenue ₹317130.37 (daily sum ₹317130.37); top diagnosis "Chronic sinusitis"; billing → 403	**PASS**
IT-USR-01	FR-05	Administrator creates users; duplicate usernames and short passwords are rejected Input: Create "nurse.anita" (BILLING); repeat; create with password "abc"	201, 409, 400	201, 409 "Username "nurse.anita" is already taken", 400 "password: Password must be at least 8 characters"	**PASS**
IT-USR-02	FR-06	An administrator cannot deactivate their own account Input: Admin POST /users/{self}/deactivate	403 "You cannot deactivate your own account"	403 "You cannot deactivate your own account"	**PASS**
IT-USR-03	FR-06	Password reset lets the user log in with the new password only Input: Admin resets "nurse.anita" to NewPass@2026	Old password 401, new password 200	old 401, new 200	**PASS**

8.3 System test cases

Table 50: System test cases (14)

ID	Req.	Test case / input	Expected result	Actual result (recorded during execution)	Status
ST-APT-01	FR-15, FR-16, FR-20	Book an appointment for tomorrow with the 4-step wizard Input: New patient; Dr. Arjun Sharma; tomorrow; first free slot; reason "Persistent cough"	Toast "Appointment booked"; appointment listed for tomorrow as Scheduled	booked P00067 with Dr. Arjun Sharma on 2026-10-23 at 09:00; list shows Scheduled	**PASS**
ST-AUTH-01	FR-01, FR-03, FR-04	Login per role, role-based menus and blocked direct URLs Input: Wrong password for reception; reception and billing log in; reception opens /users directly	Error on wrong password; receptionist menu has no Billing/Reports/Users; billing menu has no Queue Board; /users shows "Access denied"	wrong password → "Invalid username or password"; receptionist menu: Dashboard, Patients, Appointments, Queue Board, Beds & Admissions; receptionist → /users shows "Access denied"; billing menu: Dashboard, Patients, Beds & Admissions, Billing; logout returns to /login	**PASS**

ID	Req.	Test case / input	Expected result	Actual result (recorded during execution)	Status
ST-BIL-01	FR-44, FR-46, FR-47	Generate an invoice, take a partial UPI payment, reject an overpayment, settle and print the receipt Input: First OPD patient in "To bill"; pay ~50% by UPI with reference; then balance + ₹100 in cash; then the exact balance	Invoice UNPAID → PARTIALLY_PAID; overpayment rejected with the outstanding balance; → PAID; receipt page shows "Payment Receipt"	INV-20261022-0006 ₹624 for P00061: paid ₹312 UPI (R-20261022-0006) → Partially paid; ₹412 rejected ("Payment not accepted Payment exceeds the outstanding balance of ₹312.00"); paid ₹312.00 (R-20261022-0007) → Paid; receipt page rendered	**PASS**
ST-CON-01	FR-25, FR-26, FR-27, FR-28, FR-29	Doctor calls the next patient, records the consultation and completes it Input: Dr. Sharma: Call next → vitals 130/85, pulse 92, temp 100.2 → diagnosis "Acute viral fever" → Paracetamol → FBS test → Complete	Workspace opens for the called token; prescription saved; lab order shown; visit completed and queue screen returns	called and treated Madhumita Saha; notes, prescription (Paracetamol) and FBS order saved; consultation completed	**PASS**
ST-IPD-01	FR-38, FR-39, FR-40, FR-41	Admit a patient from the bed board and transfer to another bed Input: New patient; first AVAILABLE bed; attending Dr. Sharma; then Transfer bed to the first bed offered	Admission detail opens with status Admitted; after transfer the new bed is Occupied and the old bed is Cleaning	P00068 admitted to GA-02 (ADM-20261022-002, status ADMITTED); transferred to GA-03; GA-02 → CLEANING, GA-03 → OCCUPIED	**PASS**
ST-LAB-01	FR-31, FR-32, FR-33	Lab technician collects a sample and records an out-of-range result Input: FBS order for an inpatient; collect sample; result 162 mg/dL (reference 70–100)	Status moves ORDERED → SAMPLE_COLLECTED → COMPLETED; live preview and saved result show ABNORMAL	order #218 for P00001: collected → result 162 mg/dL; preview "Abnormal (high)"; saved status COMPLETED, is_abnormal=1	**PASS**
ST-PAT-01	FR-07, FR-08, FR-12	Register a patient through the form with validation feedback Input: Register dialog: first with mobile "12345", then with a valid 10-digit mobile	Inline error "Enter a valid 10-digit mobile number"; then patient detail page with a new P-code	invalid mobile → inline error "Enter a valid 10-digit mobile number"; registered Moumita Senipihxw → P00066; detail page shows allergy alert	**PASS**
ST-PHA-01	FR-36, FR-37	Pharmacist cannot dispense an out-of-stock prescription but dispenses a valid one Input: Pending prescriptions: one includes Insulin Glargine ×5 (3 in stock); another fully in stock	Insulin prescription shows the stock problem and a disabled Dispense button; the other is dispensed and stock decreases	Rx #204 blocked ("Insufficient stock for Insulin Glargine: 3 available, 5 required"); Rx #219 → DISPENSED; Paracetamol stock 900 → 890	**PASS**
ST-QUE-01	FR-21, FR-22, FR-23	Check in today's appointment and see the token on the queue board Input: A SCHEDULED appointment of Dr. Sharma for today; Check in from the Appointments list	Toast with token number; patient appears in Dr. Sharma's waiting list with token and ETA	checked in P00045 → "Rahul Ghosh checked in · token #09"; queue board: token #09, ETA "~36 min", 3 ahead	**PASS**
ST-RPT-01	FR-48, FR-49, FR-50	Administrator views dashboard, reports, users, doctors and the audit trail Input: Admin opens Dashboard, Reports (last 14 days), Users, Doctors, Audit trail filtered by action PAYMENT	KPI cards and charts render; audit trail lists the payments recorded by billing staff	dashboard KPIs + charts visible; reports page renders 15 charts; users permission matrix and doctors list shown; audit PAYMENT entry: "22 Oct, 04:14 am billing Billing Staff PAYMENT Invoice #377 INV-20261022-0006: received ₹312 by CASH (R-20261022-0007); PARTIALLY_PAID → PAI"	**PASS**
ST-UI-01	FR-03, FR-48, NFR-05	Every screen renders for every role at desktop width without errors Input: 6 roles × 20 routes at 1440 px (allowed pages and access-denied pages)	No uncaught exception or console error; each page shows content, an access-denied message or a not-found message (never a blank screen)	120 page visits (64 access-denied pages shown as designed); problems: none	**PASS**
ST-UI-02	NFR-05	All screens fit a 390 px phone without horizontal page scrolling Input: 6 roles × 20 routes at 390 × 844 px	document scroll width ≤ viewport width on every page (wide tables scroll inside their card)	120 page visits at 390 px; widest document 390px; problems: none	**PASS**

ID	Req.	Test case / input	Expected result	Actual result (recorded during execution)	Status
ST-UI-03	FR-03, FR-12	Broken or unauthorised deep links show a clear message instead of crashing Input: Administrator opens /patients/99999, /patients/abc, /admissions/abc, /admissions/99999, /billing/invoices/0, /billing/invoices/99999, /print/receipt/xyz, /print/invoice/99999, /no-such-page; receptionist opens /billing/invoices/1 and /users	"Record not found" / "Page not found" / "Access denied" message with a way back; no page errors	/patients/99999 → Record not found; /patients/abc → Record not found; /admissions/abc → Record not found; /admissions/99999 → Record not found; /billing/invoices/0 → Record not found; /billing/invoices/99999 → Record not found; /print/receipt/xyz → Record not found; /print/invoice/99999 → Record not found; /no-such-page → Page not found; reception /billing/invoices/1 → Access denied; reception /users → Access denied; problems: none	**PASS**
ST-UI-04	FR-46, NFR-04	Double-clicking "Record payment" records only one payment Input: Billing staff generates an OPD invoice, enters ₹1 in Record payment and double-clicks the submit button	Exactly one payment of ₹1 and one receipt; the button is disabled while the request is in progress	INV-20261022-0005: 1 payment(s), paid ₹1, receipt R-20261022-0005	**PASS**

8.4 Acceptance test cases

Table 51: Acceptance test cases (5)

ID	Req.	Test case / input	Expected result	Actual result (recorded during execution)	Status
AT-01	FR-12, FR-21, FR-24, FR-25, FR-26, FR-27, FR-28, FR-29, FR-32, FR-37, FR-44, FR-46	OPD patient journey: check-in → consultation → lab → pharmacy → billing Input: Today's scheduled patient of Dr. Sharma; reception marks urgent (fever 103°F); doctor prescribes Cetirizine and orders Haemoglobin; lab result 13.5 g/dL; pharmacy dispenses; billing settles in cash	Each role completes its step from its own screen; patient history shows the visit COMPLETED and the invoice PAID	reception checked in P00028 and marked urgent; doctor called next (after 1 served ahead) and completed the visit with diagnosis, prescription and HB order; lab recorded HB 13.5 g/dL (within range); pharmacist dispensed Rx #218; billing raised INV-20261022-0003 with 3 line(s) and settled it in cash; patient history shows the visit Completed (appointment #543 status COMPLETED) and the invoice Paid	**PASS**
AT-02	FR-38, FR-39, FR-41, FR-42, FR-43, FR-44, FR-46	IPD journey: admission → discharge request → billing clearance → discharge → bed turnaround Input: New patient admitted by reception; Dr. Sharma requests discharge; reception tries to discharge; billing invoices and settles; reception discharges; housekeeping marks bed cleaned	Discharge blocked until billing is cleared; then DISCHARGED; bed goes OCCUPIED → CLEANING → AVAILABLE	reception admitted P00065 to GA-02 (ADM-20261022-001); doctor requested discharge with summary; reception discharge blocked: "Discharge blocked by billing Discharge blocked: 1 charge(s) have not been invoiced yet"; billing raised INV-20261022-0004 (bed charges) and settled it by card; reception completed the discharge; bed GA-02: CLEANING → AVAILABLE after "Mark cleaned"	**PASS**
AT-03	FR-03, NFR-03	Each role sees exactly the menus it is entitled to Input: Log in as admin, reception, dr.sharma, lab, pharmacy and billing	Sidebar menus match the role-permission matrix for all 6 roles	admin: 12 items; reception: 5 items; dr.sharma: 8 items; lab: 3 items; pharmacy: 3 items; billing: 4 items	**PASS**
AT-04	NFR-05	Usability: booking in at most 4 steps and no horizontal scrolling on a 390 px phone Input: Receptionist opens the booking wizard; then views Dashboard, Queue and Patients at 390×844	Wizard has ≤ 4 steps; page scrollWidth ≤ viewport width on all three pages	booking wizard steps = 4; scrollWidth/innerWidth at 390 px: / 390/390px, /queue 390/390px, /patients 390/390px	**PASS**
AT-05	FR-50, NFR-08	Administrator can trace the journeys in the audit trail Input: Admin filters the audit trail by entity Admission and by the invoice numbers from AT-01/AT-02	Admission, discharge, payment and dispensing actions are listed with user and role	Admission: "22 Oct, 04:10 am reception Receptionist STATUS*CHANGE Admission #31 DISCHARGE*PENDING → DISCHARGED (ADM-202610"; INV-20261022-0003: PAYMENT by billing; INV-20261022-0004: PAYMENT by billing; DISPENSE entries by pharmacy	**PASS**

9. Expected Results

Expected results were defined **before** execution from the requirements and business rules (not from the observed behaviour) and are stored with each test case. They specify the exact HTTP status code, error code or message, state transition, calculated amount or UI element that must appear – for example “409 DOCTOR*SLOT*TAKEN”, “ETA 0, 12, 24 min”, “total ₹1,111.05” or “bed GA-01 CLEANING”. The expected result of every case is shown in the tables of Section 8.

10. Actual Results

Actual results were captured automatically at run time. The final regression run executed the complete suite on a clean build:

Table 52: Test execution summary (final run)

Test level	Planned	Executed	Passed	Failed	Pass rate
Unit	50	50	50	0	100%
Integration	92	92	92	0	100%
System	14	14	14	0	100%
Acceptance	5	5	5	0	100%
Total	161	161	161	0	100%

10.1 Execution history

The suite was executed several times during development. Failures in earlier runs and their causes are recorded honestly below; failures caused by mistakes in test scripts are distinguished from product defects (Section 12).

Table 53: Test run log

Run	Date / time (IST)	Scope	Result	Notes
R1	2026-10-21 23:09	Unit (47)	47 passed	First execution of unit suite.
R2	2026-10-21 23:13	Integration (75)	73 passed, 2 failed	IT-IPD-09: test advanced the simulated clock by 26 h, so the 8-hour sessions had expired (correct product behaviour, FR-01) → test now logs in again after the jump. IT-BIL-08: test relied on an OPD charge that did not exist in its fixture → test now creates its own visit. Both were test-script errors, not product defects.
R3	2026-10-21 23:16	Unit + Integration (122)	122 passed	Regression run after correcting the two test scripts.
R4	2026-10-21 23:20	System + Acceptance E2E (15, Playwright/Chromium)	15 passed	First execution of the browser suite against a freshly seeded demo database (10 system ST-*, 5 acceptance AT-*). No product defects found; report screenshots captured in docs/screenshots/.
R5	2026-10-21 23:24	System + Acceptance E2E (15)	15 passed	Confirmation run on a new database to check the suite is repeatable (no flaky tests); E2E entries in actuals.jsonl are from this run.
R6	2026-10-22 03:00–03:35	Robustness / edge-case probes (API)	16 issues found	Exploratory boundary and invalid-input probes against every API (impossible dates, wrong types, extreme values, wildcards, ownership, sequence numbers, time-zone boundaries). Product defects DEF-09 … DEF-24 logged and fixed; 16 regression tests IT-EDGE-01 … 16 added.
R7	2026-10-22 03:35–04:05	Robustness / edge-case crawl (UI)	9 issues found	Every screen for all 6 roles at 1440 px, 390 px and 360 px, bad deep links, empty database, double-clicks. Product defects DEF-25 … DEF-33 logged and fixed; crawl automated as ST-UI-01 … 04.

Run	Date / time (IST)	Scope	Result	Notes
R9	2026-10-22 04:14	System + Acceptance E2E (19, Playwright/Chromium)	19 passed	Final regression run on a freshly generated demo database (10 ST functional, 4 ST-UI, 5 AT) in 3.9 min. Report screenshots re-captured in this run.
R10	2026-10-22 04:20	Unit + Integration (142), with coverage	142 passed	Final regression run after all fixes (includes IT-EDGE-*, UT-PAT-06 … 08). Traceability review showed FR-14 had no test, so IT-DOC-03 was added before this run. Together with R9 this is the final execution reported in Part D.

10.2 Code coverage

Table 54: Code coverage of the server by unit + integration tests (V8)

Scope	Lines	Branches	Functions	Statements
Domain rules (server/domain)	99.2%	85.9%	98.3%	99.2%
Services (server/services)	95.2%	78.6%	86.8%	95.2%
Whole server	95.45%	84.15%	90.58%	95.45%

10.3 Non-functional results

Table 55: Non-functional requirement results

NFR	Measured / observed	Verdict
NFR-01	p50 3.9 ms, p95 5.1 ms, max 8.1 ms over 200 requests	**PASS**
NFR-02	15 users, all bcrypt=true; API exposes password_hash=false; 200 0 rows; 200 0 rows; patients in DB 65; forged/expired tokens rejected (IT-AUTH-05)	**PASS**
NFR-03	Clinical data denied to non-clinical roles (IT-CON-06); passwords never in audit log (IT-AUD-01)	**PASS**
NFR-04	Dispensing with one unavailable item leaves stock and prescription unchanged (IT-PHA-03); admission + bed update in one transaction	**PASS**
NFR-05	booking wizard steps = 4; scrollWidth/innerWidth at 390 px: / 390/390px, /queue 390/390px, /patients 390/390px	**PASS**
NFR-06	Layered code; domain rules 99.2% line coverage (target ≥ 80%)	**PASS**
NFR-07	Built and fully tested on Linux with Node.js 22 and Chromium; all dependencies are cross-platform (Node.js 20+, prebuilt SQLite binaries for Windows/macOS/Linux). Windows/macOS and Firefox/Safari runs are not yet verified	**PASS (partial – see scope)**
NFR-08	patient entry: CREATE by reception (RECEPTIONIST); user entry: "Created LAB_TECH account "audit.user""; password leaked=false	**PASS**

11. Pass/Fail Status

Overall status: PASS. 161 of 161 test cases passed (100%) in the final run. 50 of 50 functional requirements are covered by at least one passing test (see the traceability matrix in Appendix A). 32 defects were found and fixed during testing and 1 low-severity enhancement was deferred; no Critical or High defect is open.

Table 56: Exit criteria evaluation

Exit criterion (Part C, 3.3)	Result	Met?
100% of planned test cases executed	161/161 executed	**PASS**
All test cases pass	161/161 passed	**PASS**
No open Critical/High defects	0 open	**PASS**
Every FR covered by a passing test	50/50	**PASS**
Domain-rule line coverage ≥ 80%	99.2%	**PASS**

12. Defect Reporting

33 defects were reported during system testing and code review: 2 high, 12 medium, 19 low severity. Each fixed defect was re-tested and is protected by the regression test listed in the log.

Table 57: Defect log

ID	Title	Module	Severity / priority	Found in	Status	Verified by
DEF-01	Doctor name printed as "Dr. Dr. ..." in conflict errors and audit entries	APT / QUE	Low / P3	System testing (UI walkthrough of booking a taken slot)	**Fixed**	IT-APT-03 (asserts the conflict message)
DEF-02	Payment receipt shows the invoice's current balance instead of the balance after that payment	BIL	Medium / P2	System testing (reprinting the first receipt of an invoice paid in two instalments)	**Fixed**	IT-BIL-07
DEF-03	Invoice list ordered by internal id rather than invoice date	BIL	Low / P3	System testing (Billing → Invoices tab)	**Fixed**	Manual re-test
DEF-04	Clicking a form label toggles the first payment-method / working-day button	UI (Billing, Doctors)	Medium / P2	System testing (Record payment dialog)	**Fixed**	ST-BIL-01 (E2E payment flow)
DEF-05	Transferring a patient on the day of admission bills two minimum bed-days	IPD / BIL	High / P1	System testing (admit → transfer after a few minutes → request discharge)	**Fixed**	UT-IPD-05, UT-IPD-06, IT-IPD-06
DEF-06	Demo data gives one patient two tokens with the same doctor on the same day	Seed data	Low / P3	System testing (Queue board shows P00017 with tokens #04 and #05)	**Fixed**	Manual re-test of queue board
DEF-07	Laboratory status counters ignore the doctor's "My orders" filter	LAB	Low / P3	System testing (doctor view of Laboratory page)	**Fixed**	IT-LAB-06
DEF-08	Patient picker in the Admit dialog does not flag patients who are already admitted	IPD (UI)	Low / P4	System testing (admit dialog)	**Deferred**	IT-IPD-02 (server-side rejection)
DEF-09	Impossible calendar dates (e.g. 30 February) are accepted	APT / PAT / PHA	Medium / P2	Robustness / edge-case testing (boundary and invalid-input probes against the API)	**Fixed**	IT-EDGE-01, UT-PAT-06
DEF-10	Malformed date filters cause an internal server error	RPT / BIL / AUD / QUE	Medium / P2	Robustness / edge-case testing (boundary and invalid-input probes against the API)	**Fixed**	IT-EDGE-02
DEF-11	Report period is not validated (from after to, multi-year ranges)	RPT	Low / P3	Robustness / edge-case testing (boundary and invalid-input probes against the API)	**Fixed**	IT-EDGE-03
DEF-12	Non-text and over-long patient fields are stored	PAT	Medium / P2	Robustness / edge-case testing (boundary and invalid-input probes against the API)	**Fixed**	IT-EDGE-04
DEF-13	Mobile numbers typed with +91, spaces or a leading 0 are rejected	PAT	Low / P3	Robustness / edge-case testing (boundary and invalid-input probes against the API)	**Fixed**	IT-EDGE-05, UT-PAT-07
DEF-14	Search terms containing % or _ match every record	PAT / AUD / APT / BIL / PHA	Low / P3	Robustness / edge-case testing (boundary and invalid-input probes against the API)	**Fixed**	IT-EDGE-06, UT-PAT-08
DEF-15	Negative limit returns every row	PAT / APT / AUD / BIL / LAB	Low / P3	Robustness / edge-case testing (boundary and invalid-input probes against the API)	**Fixed**	IT-EDGE-07, UT-PAT-08

ID	Title	Module	Severity / priority	Found in	Status	Verified by
DEF-16	Free-text reasons accept numbers and unlimited length	IPD / APT / USR	Low / P3	Robustness / edge-case testing (boundary and invalid-input probes against the API)	**Fixed**	IT-EDGE-08
DEF-17	No-show can be recorded before the appointment time	APT	Medium / P2	Robustness / edge-case testing (boundary and invalid-input probes against the API)	**Fixed**	IT-EDGE-09
DEF-18	Appointments can be booked with a deactivated doctor	APT / DOC	Medium / P2	Robustness / edge-case testing (boundary and invalid-input probes against the API)	**Fixed**	IT-EDGE-10
DEF-19	A doctor can order tests on another doctor's consultation; duplicate tests are ordered twice	LAB	Medium / P2	Robustness / edge-case testing (boundary and invalid-input probes against the API)	**Fixed**	IT-EDGE-11
DEF-20	The same medicine can appear twice in one prescription	CON	Low / P3	Robustness / edge-case testing (boundary and invalid-input probes against the API)	**Fixed**	IT-EDGE-12
DEF-21	A zero-value invoice stays UNPAID and cannot be settled	BIL	Medium / P2	Robustness / edge-case testing (boundary and invalid-input probes against the API)	**Fixed**	IT-EDGE-13
DEF-22	Admission / invoice / receipt numbers can collide with existing numbers	IPD / BIL	High / P1	Robustness / edge-case testing (boundary and invalid-input probes against the API)	**Fixed**	IT-EDGE-14
DEF-23	Lab worklist date filter uses the UTC day instead of the local day	LAB	Medium / P2	Code review during robustness testing	**Fixed**	IT-EDGE-16
DEF-24	Some database constraint errors are reported as HTTP 500	API	Low / P3	Code review during robustness testing	**Fixed**	Code review re-test (no API path triggers it)
DEF-25	Pages scroll sideways on phones	UI (Dashboard, Queue, Admission, Invoice, Reports)	Medium / P2	Robustness / edge-case testing of the UI (role × page crawl at 1440 px and 390 px, bad deep links)	**Fixed**	ST-UI-02
DEF-26	Long unbroken text overflows the patient header	PAT (UI)	Low / P3	Robustness / edge-case testing of the UI (role × page crawl at 1440 px and 390 px, bad deep links)	**Fixed**	Manual re-test
DEF-27	Create-user dialog keeps the previous user's data, including the password	USR (UI)	Medium / P2	Code review of dialogs during robustness testing	**Fixed**	Manual re-test
DEF-28	Missing or forbidden records show a generic “Could not load data – Retry” message	UI-wide	Low / P3	Robustness / edge-case testing of the UI (role × page crawl at 1440 px and 390 px, bad deep links)	**Fixed**	ST-UI-03
DEF-29	Prescription and medicine forms accept fractional or absurd numbers	CON / PHA (UI)	Low / P3	Robustness / edge-case testing of the UI (role × page crawl at 1440 px and 390 px, bad deep links)	**Fixed**	Manual re-test
DEF-30	Session expiry and server outages show unclear messages	AUTH / API client (UI)	Low / P3	Robustness / edge-case testing of the UI (role × page crawl at 1440 px and 390 px, bad deep links)	**Fixed**	Manual re-test

ID	Title	Module	Severity / priority	Found in	Status	Verified by
DEF-31	Disabled “Sign in” button has low contrast	AUTH (UI)	Low / P4	Robustness / edge-case testing of the UI (role × page crawl at 1440 px and 390 px, bad deep links)	Fixed	Manual re-test
DEF-32	React Router future-flag warnings on every page in development mode	UI	Low / P4	Robustness / edge-case testing of the UI (role × page crawl at 1440 px and 390 px, bad deep links)	Fixed	ST-UI-01
DEF-33	Bed-stay note and lab units display incorrectly	IPD / LAB (UI)	Low / P4	Robustness / edge-case testing of the UI (role × page crawl at 1440 px and 390 px, bad deep links)	Fixed	Manual re-test

12.1 Detailed defect reports

DEF-01	Doctor name printed as "Dr. Dr. ..." in conflict errors and audit entries
Module / severity	APT / QUE · Low · P3
Detected during	System testing (UI walkthrough of booking a taken slot)
Steps to reproduce	Book an appointment in a slot that is already taken for Dr. Arjun Sharma.
Expected	Message: "Dr. Arjun Sharma already has an appointment at ..."
Actual (before fix)	Message: "Dr. Dr. Arjun Sharma already has an appointment at ..." (same in audit log summaries).
Root cause	Service prefixed "Dr." to full_name, which already contains the title.
Resolution	Removed the extra prefix in appointmentService and queueService.
Status / verification	Fixed – IT-APT-03 (asserts the conflict message)

DEF-02	Payment receipt shows the invoice's current balance instead of the balance after that payment
Module / severity	BIL · Medium · P2
Detected during	System testing (reprinting the first receipt of an invoice paid in two instalments)
Steps to reproduce	Pay ₹100 of an ₹900 invoice, then pay the remaining ₹800, then print the first receipt.
Expected	First receipt shows balance after payment ₹800.00.
Actual (before fix)	First receipt shows balance ₹0.00 (uses the invoice's latest paid amount).
Root cause	BillingService.receipt() computed total − invoice.paid_amount.
Resolution	Balance is now total − SUM(payments with id ≤ this payment).
Status / verification	Fixed – IT-BIL-07

DEF-03	Invoice list ordered by internal id rather than invoice date
Module / severity	BIL · Low · P3
Detected during	System testing (Billing → Invoices tab)
Steps to reproduce	Open Billing → Invoices after loading demo data.
Expected	Newest invoices first.
Actual (before fix)	Older invoices created later in the seed appeared first.
Root cause	ORDER BY i.id DESC.
Resolution	ORDER BY i.created_at DESC, i.id DESC.
Status / verification	Fixed – Manual re-test

DEF-04	Clicking a form label toggles the first payment-method / working-day button
Module / severity	UI (Billing, Doctors) · Medium · P2
Detected during	System testing (Record payment dialog)
Steps to reproduce	Open Record payment, select UPI, then click on the "Payment method" label text.
Expected	Selection stays on UPI.
Actual (before fix)	Selection jumps to CASH because the buttons were nested inside a <label> element.
Root cause	Button group wrapped by the shared Field component, which renders a <label>.
Resolution	Button groups use a plain caption and role=radio / aria-pressed buttons.
Status / verification	Fixed – ST-BIL-01 (E2E payment flow)

DEF-05	Transferring a patient on the day of admission bills two minimum bed-days
Module / severity	IPD / BIL · High · P1
Detected during	System testing (admit → transfer after a few minutes → request discharge)
Steps to reproduce	Admit to General Ward A (₹1,500/day), transfer to General Ward B after 10 minutes, request discharge 5 hours later.
Expected	1 bed-day charged in total (₹1,500).
Actual (before fix)	2 bed-days charged (₹3,000): each bed stay applied its own 1-day minimum.
Root cause	bedCharges() applied max(1, ceil(h/24)) per stay instead of per admission.
Resolution	Admission is billed max(1, ceil(totalHours/24)) days, shared across stays by largest-remainder allocation; zero-day stays are not charged.
Status / verification	Fixed – UT-IPD-05, UT-IPD-06, IT-IPD-06

DEF-06	Demo data gives one patient two tokens with the same doctor on the same day
Module / severity	Seed data · Low · P3
Detected during	System testing (Queue board shows P00017 with tokens #04 and #05)
Steps to reproduce	Load demo data, open Dr. Sharma's queue board.
Expected	A patient appears at most once per doctor per day.
Actual (before fix)	The same patient appeared twice in the queue.
Root cause	Seeder only prevented identical time slots, not repeat visits to the same doctor.
Resolution	Seeder tracks patient + doctor + date.
Status / verification	Fixed – Manual re-test of queue board

DEF-07	Laboratory status counters ignore the doctor's "My orders" filter
Module / severity	LAB · Low · P3
Detected during	System testing (doctor view of Laboratory page)
Steps to reproduce	Log in as a doctor, open Laboratory, keep "My orders" selected.
Expected	Tab counts reflect only the doctor's orders.
Actual (before fix)	Tab counts showed hospital-wide totals.
Root cause	/lab/stats had no doctor filter.
Resolution	/lab/stats accepts doctorId; page passes it when "My orders" is selected.
Status / verification	Fixed – IT-LAB-06

DEF-08	Patient picker in the Admit dialog does not flag patients who are already admitted
Module / severity	IPD (UI) · Low · P4
Detected during	System testing (admit dialog)
Steps to reproduce	Open Admit patient and search for a patient who is currently admitted.
Expected	Picker marks the patient as already admitted.
Actual (before fix)	Patient can be selected; the server then correctly rejects the admission with "already admitted" (FR-39 is still enforced).
Root cause	Picker does not query active admissions (usability enhancement).
Resolution	Deferred to next release – business rule is enforced server-side and covered by IT-IPD-02.
Status / verification	Deferred – IT-IPD-02 (server-side rejection)

DEF-09	Impossible calendar dates (e.g. 30 February) are accepted
Module / severity	APT / PAT / PHA · Medium · P2
Detected during	Robustness / edge-case testing (boundary and invalid-input probes against the API)
Steps to reproduce	Book an appointment on 2027-02-30; register a patient with DOB 1990-02-30; restock with expiry 2027-02-31.
Expected	HTTP 400 "Not a valid calendar date".
Actual (before fix)	Accepted (201/200); JavaScript rolled the date over to March and the invalid value was stored.
Root cause	Date fields were validated only by the YYYY-MM-DD regular expression.
Resolution	isRealDate() added to the domain layer and used by every date validator.
Status / verification	Fixed – IT-EDGE-01, UT-PAT-06

DEF-10	Malformed date filters cause an internal server error
Module / severity	RPT / BIL / AUD / QUE · Medium · P2
Detected during	Robustness / edge-case testing (boundary and invalid-input probes against the API)
Steps to reproduce	GET /api/reports/summary?from=abc, /api/billing/invoices?from=abc, /api/audit?from=2026-99-99.
Expected	HTTP 400 with a field message.
Actual (before fix)	HTTP 500 Internal server error (RangeError: Invalid time value).
Root cause	new Date('abcT00:00').toISOString() throws; query dates were not validated.
Resolution	optionalDate() validator applied to all date query parameters.
Status / verification	Fixed – IT-EDGE-02

DEF-11	Report period is not validated (from after to, multi-year ranges)
Module / severity	RPT · Low · P3
Detected during	Robustness / edge-case testing (boundary and invalid-input probes against the API)
Steps to reproduce	GET /api/reports/summary?from=2026-10-30&to=2026-10-01.
Expected	HTTP 400.
Actual (before fix)	HTTP 200 with an empty or very large report.
Root cause	No ordering or range checks.
Resolution	Reject from > to and ranges longer than 366 days; the UI also keeps from ≤ to.
Status / verification	Fixed – IT-EDGE-03

DEF-12	Non-text and over-long patient fields are stored
Module / severity	PAT · Medium · P2
Detected during	Robustness / edge-case testing (boundary and invalid-input probes against the API)
Steps to reproduce	Register a patient with first_name ["a"] or a 5,000-character name.
Expected	HTTP 400 with field errors.
Actual (before fix)	HTTP 201 – the array/huge string was stored.
Root cause	No type or length checks on patient fields.
Resolution	Type checks and maximum lengths (names 60, email 120, address/allergies 300).
Status / verification	Fixed – IT-EDGE-04

DEF-13	Mobile numbers typed with +91, spaces or a leading 0 are rejected
Module / severity	PAT · Low · P3
Detected during	Robustness / edge-case testing (boundary and invalid-input probes against the API)
Steps to reproduce	Register a patient with phone "+91 98765 43210".
Expected	Number normalised to 9876543210 and accepted.
Actual (before fix)	HTTP 400 "Enter a valid 10-digit mobile number".
Root cause	Strict 10-digit regular expression without normalisation.
Resolution	normalizePhone() on server and client (spaces, dashes, brackets, +91/91/0 prefixes).
Status / verification	Fixed – IT-EDGE-05, UT-PAT-07

DEF-14	Search terms containing % or _ match every record
Module / severity	PAT / AUD / APT / BIL / PHA · Low · P3
Detected during	Robustness / edge-case testing (boundary and invalid-input probes against the API)
Steps to reproduce	GET /api/patients?q=%
Expected	Only records literally containing "%".
Actual (before fix)	All patients returned.
Root cause	User input inserted into LIKE patterns without escaping wildcards.
Resolution	likeTerm() escapes % and _ (LIKE … ESCAPE '\').
Status / verification	Fixed – IT-EDGE-06, UT-PAT-08

DEF-15	Negative limit returns every row
Module / severity	PAT / APT / AUD / BIL / LAB · Low · P3
Detected during	Robustness / edge-case testing (boundary and invalid-input probes against the API)
Steps to reproduce	GET /api/appointments?limit=-1
Expected	Bounded page size.
Actual (before fix)	All rows (SQLite treats a negative LIMIT as unlimited); NaN offsets passed through.
Root cause	Math.min(Number(limit) || default, max) keeps negative values.
Resolution	clampInt() clamps limit/offset into a valid range.
Status / verification	Fixed – IT-EDGE-07, UT-PAT-08

DEF-16	Free-text reasons accept numbers and unlimited length
Module / severity	IPD / APT / USR · Low · P3

DEF-16	Free-text reasons accept numbers and unlimited length
Detected during	Robustness / edge-case testing (boundary and invalid-input probes against the API)
Steps to reproduce	Admit a patient with reason 42 or a 2,000-character reason.
Expected	HTTP 400 "must be text" / "at most 1000 characters".
Actual (before fix)	Generic error for a number; very long text accepted.
Root cause	nonEmpty() validator had no type message or maximum length.
Resolution	nonEmpty(label, max) with type and length checks; discharge summary ≤ 5,000 characters.
Status / verification	Fixed – IT-EDGE-08

DEF-17	No-show can be recorded before the appointment time
Module / severity	APT · Medium · P2
Detected during	Robustness / edge-case testing (boundary and invalid-input probes against the API)
Steps to reproduce	At 10:00, mark today's 15:00 appointment as no-show.
Expected	HTTP 400 – the appointment has not started yet.
Actual (before fix)	HTTP 200, status NO_SHOW.
Root cause	Only future dates were checked, not the time on the current day.
Resolution	For today's appointments the start time must have passed.
Status / verification	Fixed – IT-EDGE-09

DEF-18	Appointments can be booked with a deactivated doctor
Module / severity	APT / DOC · Medium · P2
Detected during	Robustness / edge-case testing (boundary and invalid-input probes against the API)
Steps to reproduce	Administrator deactivates a doctor's account; receptionist books that doctor.
Expected	HTTP 409 DOCTOR_INACTIVE.
Actual (before fix)	HTTP 201 – booking created.
Root cause	Doctor's user account status not checked when booking.
Resolution	assertBookable() rejects inactive doctors (booking and rescheduling).
Status / verification	Fixed – IT-EDGE-10

DEF-19	A doctor can order tests on another doctor's consultation; duplicate tests are ordered twice
Module / severity	LAB · Medium · P2
Detected during	Robustness / edge-case testing (boundary and invalid-input probes against the API)
Steps to reproduce	Dr. Banerjee orders FBS on Dr. Sharma's open consultation; order test_ids [FBS, FBS].
Expected	HTTP 403; a duplicate test is ordered once.
Actual (before fix)	HTTP 201; two identical orders and two charges.
Root cause	No ownership check and no de-duplication of test ids.
Resolution	Ownership check (403), de-duplication, and 409 DUPLICATE*LAB*ORDER if the same test is still pending for the visit.
Status / verification	Fixed – IT-EDGE-11

DEF-20	The same medicine can appear twice in one prescription
Module / severity	CON · Low · P3

DEF-20	The same medicine can appear twice in one prescription
Detected during	Robustness / edge-case testing (boundary and invalid-input probes against the API)
Steps to reproduce	Save a prescription with two Paracetamol lines.
Expected	HTTP 400 "Each medicine can appear only once".
Actual (before fix)	Accepted; dispensing and billing showed duplicate lines.
Root cause	No duplicate check on prescription items.
Resolution	Duplicate medicines rejected; maximum 30 items.
Status / verification	Fixed – IT-EDGE-12

DEF-21	A zero-value invoice stays UNPAID and cannot be settled
Module / severity	BIL · Medium · P2
Detected during	Robustness / edge-case testing (boundary and invalid-input probes against the API)
Steps to reproduce	Doctor with a ₹0 (free camp) fee completes a visit; billing generates the invoice.
Expected	Invoice status PAID (nothing to collect).
Actual (before fix)	Status UNPAID; a ₹0 payment is rejected, so the invoice could never be closed and blocked discharge.
Root cause	Status was always initialised to UNPAID.
Resolution	Invoices with total 0 are created as PAID.
Status / verification	Fixed – IT-EDGE-13

DEF-22	Admission / invoice / receipt numbers can collide with existing numbers
Module / severity	IPD / BIL · High · P1
Detected during	Robustness / edge-case testing (boundary and invalid-input probes against the API)
Steps to reproduce	With demo data containing ADM-20261021-019, admit about 18 patients on the same day.
Expected	Every new number is unique.
Actual (before fix)	The 18th admission of the day would generate an existing number and fail with a database constraint error.
Root cause	Sequence computed as COUNT(*)+1 for the day, which is wrong when numbers have gaps or come from imported data.
Resolution	nextSequence() uses the highest existing suffix + 1 for admissions, invoices and receipts; seed numbers admissions per day.
Status / verification	Fixed – IT-EDGE-14

DEF-23	Lab worklist date filter uses the UTC day instead of the local day
Module / severity	LAB · Medium · P2
Detected during	Code review during robustness testing
Steps to reproduce	Order a test at 01:00 IST on 22-Oct; filter the worklist by 2026-10-22.
Expected	The order is listed under 22-Oct.
Actual (before fix)	Listed under 21-Oct (UTC date of the timestamp).
Root cause	substr(created_at, 1, 10) compares the UTC date part of an ISO timestamp.
Resolution	Filter uses the local day converted to a UTC time range.
Status / verification	Fixed – IT-EDGE-16

DEF-24	Some database constraint errors are reported as HTTP 500
Module / severity	API · Low · P3

DEF-24	Some database constraint errors are reported as HTTP 500
Detected during	Code review during robustness testing
Steps to reproduce	Trigger a NOT NULL / trigger constraint error through an unexpected path.
Expected	HTTP 409 with a clear message.
Actual (before fix)	HTTP 500 Internal server error.
Root cause	Error handler mapped only three SQLite constraint codes.
Resolution	Every SQLITE_CONSTRAINT* code maps to HTTP 409.
Status / verification	Fixed – Code review re-test (no API path triggers it)

DEF-25	Pages scroll sideways on phones
Module / severity	UI (Dashboard, Queue, Admission, Invoice, Reports) · Medium · P2
Detected during	Robustness / edge-case testing of the UI (role × page crawl at 1440 px and 390 px, bad deep links)
Steps to reproduce	Open /, /queue (doctor), /admissions/1, /billing/invoices/1 or /reports at 390 px width.
Expected	Page fits the screen; wide tables scroll inside their card.
Actual (before fix)	Page became 444–641 px wide (horizontal page scroll).
Root cause	Cards inside CSS grid columns kept min-width:auto, so wide tables stretched the column.
Resolution	Cards get min-width 0 so tables scroll inside the card.
Status / verification	Fixed – ST-UI-02

DEF-26	Long unbroken text overflows the patient header
Module / severity	PAT (UI) · Low · P3
Detected during	Robustness / edge-case testing of the UI (role × page crawl at 1440 px and 390 px, bad deep links)
Steps to reproduce	Register a patient with a 250-character unbroken address; open the record at 390 px.
Expected	Text wraps.
Actual (before fix)	Page became 1,648 px wide.
Root cause	Inline elements without word wrapping or min-width.
Resolution	Name, e-mail and address wrap anywhere.
Status / verification	Fixed – Manual re-test

DEF-27	Create-user dialog keeps the previous user's data, including the password
Module / severity	USR (UI) · Medium · P2
Detected during	Code review of dialogs during robustness testing
Steps to reproduce	Create a user, then open “Create user” again.
Expected	Empty form.
Actual (before fix)	Previous username, name and password still filled in.
Root cause	Form state initialised only when the component first mounted.
Resolution	Form resets each time the dialog opens (also Add department).
Status / verification	Fixed – Manual re-test

DEF-28	Missing or forbidden records show a generic “Could not load data – Retry” message
Module / severity	UI-wide · Low · P3
Detected during	Robustness / edge-case testing of the UI (role × page crawl at 1440 px and 390 px, bad deep links)

DEF-28	Missing or forbidden records show a generic “Could not load data – Retry” message
Steps to reproduce	Open /patients/99999 or /admissions/abc.
Expected	“Record not found” (or “Access denied”) with a way back.
Actual (before fix)	Generic error with a Retry button that cannot help.
Root cause	ErrorState treated every error the same.
Resolution	404 → “Record not found”, 403 → “Access denied”, both with “Go back”.
Status / verification	Fixed – ST-UI-03

DEF-29	Prescription and medicine forms accept fractional or absurd numbers
Module / severity	CON / PHA (UI) · Low · P3
Detected during	Robustness / edge-case testing of the UI (role × page crawl at 1440 px and 390 px, bad deep links)
Steps to reproduce	Enter quantity 2.5, duration 0.5 days, price "abc" or reorder level −3.
Expected	Field-level error before saving.
Actual (before fix)	Client checks passed; the server rejected the request with a less readable error.
Root cause	Checks only tested ≥ 1 / ≥ 0 (NaN comparisons are false).
Resolution	Whole-number and finiteness checks with sensible maxima.
Status / verification	Fixed – Manual re-test

DEF-30	Session expiry and server outages show unclear messages
Module / severity	AUTH / API client (UI) · Low · P3
Detected during	Robustness / edge-case testing of the UI (role × page crawl at 1440 px and 390 px, bad deep links)
Steps to reproduce	Revoke a session token, then navigate; or stop the API server.
Expected	Login page explains that the session ended; outage shows a readable message.
Actual (before fix)	Silent redirect; raw “Unexpected token <” / “Failed to fetch”.
Root cause	API client did not map network or non-JSON failures.
Resolution	Readable network/server messages; “Your session has ended” notice after involuntary logout.
Status / verification	Fixed – Manual re-test

DEF-31	Disabled “Sign in” button has low contrast
Module / severity	AUTH (UI) · Low · P4
Detected during	Robustness / edge-case testing of the UI (role × page crawl at 1440 px and 390 px, bad deep links)
Steps to reproduce	Open the login page with empty fields.
Expected	Normal-contrast button; empty fields reported on submit.
Actual (before fix)	Button faded (60% opacity) until both fields were filled.
Root cause	Disabled state used for input validation.
Resolution	Button always enabled; submit validates the fields.
Status / verification	Fixed – Manual re-test

DEF-32	React Router future-flag warnings on every page in development mode
Module / severity	UI · Low · P4
Detected during	Robustness / edge-case testing of the UI (role × page crawl at 1440 px and 390 px, bad deep links)
Steps to reproduce	npm run dev; open any page and check the console.

DEF-32	React Router future-flag warnings on every page in development mode
Expected	Clean console.
Actual (before fix)	Two warnings per navigation.
Root cause	Router v7 behaviour flags not opted into.
Resolution	Enabled v7*startTransition and* v7relativeSplatPath.
Status / verification	Fixed – ST-UI-01

DEF-33	Bed-stay note and lab units display incorrectly
Module / severity	IPD / LAB (UI) · Low · P4
Detected during	Robustness / edge-case testing of the UI (role × page crawl at 1440 px and 390 px, bad deep links)
Steps to reproduce	Open an admission; open the lab result form for a numeric test.
Expected	Bed note describes the per-admission bed-day rule; units shown as “mg/dL”.
Actual (before fix)	Bed note still said “minimum 1 day” per stay (outdated after DEF-05); unit shown as “MG/DL”.
Root cause	Text not updated after DEF-05; unit placed inside an uppercase label style.
Resolution	Updated text; units rendered in their real case.
Status / verification	Fixed – Manual re-test

12.2 Defect summary and analysis

Table 58: Defects by severity and status

Severity	Reported	Fixed	Deferred / open
High	2	2	0
Medium	12	12	0
Low	19	18	1

Analysis. Most defects (25 of 33) were found by the dedicated robustness / edge-case pass, which probed every API with impossible dates, wrong data types, extreme values and malicious strings and crawled every screen for all six roles at desktop and phone width. The two high-severity defects were both data-integrity problems that only appear in longer scenarios: DEF-05 (bed-day billing after a same-day transfer) and DEF-22 (sequence numbers colliding with existing numbers) – confirming the value of scenario-based and data-driven testing in addition to unit tests. No defect caused data loss and no security vulnerability was found; the one deferred item (DEF-08) is a usability enhancement whose rule is already enforced by the server.

13. Conclusion

MediFlow implements all 50 functional requirements of the SRS. The final regression run passed 161 of 161 automated test cases across unit, integration, system and acceptance levels, every exit criterion of the test plan is met, and all defects of medium or higher severity are fixed and covered by regression tests. The system is therefore considered **ready for demonstration and user acceptance by hospital staff**. Recommended next steps before a production pilot are: cross-browser and accessibility testing, load testing on the target server, nightly database backups, HTTPS deployment behind a reverse proxy and a supervised user-acceptance session.

APPENDIX A – REQUIREMENTS TRACEABILITY MATRIX

Each requirement is traced forward to the design module (service) and user-interface screen that implement it and to the test cases that verify it. The matrix is generated from the requirement identifiers embedded in the automated tests.

Table 59: Requirements traceability matrix

Requirement	Design module / service	UI screen	Verified by test cases	Status
FR-01	AUTH – AuthService, permissions	Login, menus	IT-AUTH-01, IT-AUTH-02, ST-AUTH-01	PASS
FR-02	AUTH – AuthService, permissions	Login, menus	UT-AUTH-01, UT-AUTH-02, IT-AUTH-03, IT-AUTH-04	PASS
FR-03	AUTH – AuthService, permissions	Login, menus	UT-AUTH-03, IT-AUD-03, IT-AUTH-07, IT-BIL-09, IT-EDGE-11, IT-PHA-05, IT-QUE-06, ST-AUTH-01, ST-UI-01, ST-UI-03, AT-03	PASS
FR-04	AUTH – AuthService, permissions	Login, menus	IT-AUTH-05, IT-AUTH-06, ST-AUTH-01	PASS
FR-05	USR – UserService	Users & Roles	UT-AUTH-04, IT-USR-01	PASS
FR-06	USR – UserService	Users & Roles	IT-AUTH-04, IT-USR-02, IT-USR-03	PASS
FR-07	PAT – PatientService	Patients	UT-PAT-01, IT-PAT-01, ST-PAT-01	PASS
FR-08	PAT – PatientService	Patients	UT-PAT-02, UT-PAT-03, UT-PAT-06, UT-PAT-07, IT-EDGE-01, IT-EDGE-04, IT-EDGE-05, IT-PAT-02, ST-PAT-01	PASS
FR-09	PAT – PatientService	Patients	UT-PAT-04, IT-PAT-03	PASS
FR-10	PAT – PatientService	Patients	UT-PAT-08, IT-EDGE-05, IT-EDGE-06, IT-EDGE-07, IT-PAT-04	PASS
FR-11	PAT – PatientService	Patients	IT-PAT-05	PASS
FR-12	PAT – PatientService	Patients	UT-PAT-05, ST-PAT-01, ST-UI-03, AT-01	PASS
FR-13	DOC – DoctorService	Doctors	IT-DOC-01, IT-EDGE-10	PASS
FR-14	DOC – DoctorService	Doctors	IT-DOC-03	PASS
FR-15	APT – AppointmentService	Appointments	UT-APT-01, UT-APT-02, UT-APT-11, IT-DOC-02, IT-EDGE-01, ST-APT-01	PASS
FR-16	APT – AppointmentService	Appointments	UT-APT-03, UT-APT-04, UT-APT-05, UT-APT-06, IT-APT-01, IT-APT-02, IT-EDGE-01, IT-EDGE-10, ST-APT-01	PASS
FR-17	APT – AppointmentService	Appointments	UT-APT-07, UT-APT-08, UT-APT-09, IT-APT-03, IT-APT-04	PASS
FR-18	APT – AppointmentService	Appointments	UT-APT-08, IT-APT-05	PASS
FR-19	APT – AppointmentService	Appointments	UT-APT-08, UT-APT-10, IT-APT-06, IT-EDGE-09	PASS
FR-20	APT – AppointmentService	Appointments	IT-APT-07, IT-EDGE-02, IT-EDGE-07, ST-APT-01	PASS
FR-21	QUE – QueueService	Queue Board	UT-APT-10, UT-QUE-02, IT-QUE-01, IT-QUE-02, ST-QUE-01, AT-01	PASS
FR-22	QUE – QueueService	Queue Board	UT-QUE-01, IT-EDGE-02, IT-QUE-03, ST-QUE-01	PASS
FR-23	QUE – QueueService	Queue Board	UT-QUE-03, UT-QUE-04, UT-QUE-05, IT-QUE-03, IT-QUE-07, ST-QUE-01	PASS

Requirement	Design module / service	UI screen	Verified by test cases	Status
FR-24	QUE – QueueService	Queue Board	UT-QUE-01, IT-QUE-04, AT-01	**PASS**
FR-25	QUE – QueueService	Queue Board	UT-APT-10, IT-QUE-05, IT-QUE-06, ST-CON-01, AT-01	**PASS**
FR-26	CON – ConsultationService	Consultation	IT-CON-01, IT-CON-02, ST-CON-01, AT-01	**PASS**
FR-27	CON – ConsultationService	Consultation	IT-CON-03, IT-EDGE-12, ST-CON-01, AT-01	**PASS**
FR-28	CON – ConsultationService	Consultation	IT-CON-04, IT-EDGE-11, ST-CON-01, AT-01	**PASS**
FR-29	CON – ConsultationService	Consultation	IT-CON-05, ST-CON-01, AT-01	**PASS**
FR-30	CON – ConsultationService	Consultation	UT-AUTH-03, IT-CON-06	**PASS**
FR-31	LAB – LabService	Laboratory	IT-EDGE-16, IT-LAB-01, IT-LAB-06, ST-LAB-01	**PASS**
FR-32	LAB – LabService	Laboratory	UT-LAB-01, IT-LAB-02, IT-LAB-03, IT-LAB-04, ST-LAB-01, AT-01	**PASS**
FR-33	LAB – LabService	Laboratory	UT-LAB-02, UT-LAB-03, IT-LAB-03, ST-LAB-01	**PASS**
FR-34	LAB – LabService	Laboratory	IT-CON-04, IT-LAB-05	**PASS**
FR-35	PHA – PharmacyService	Pharmacy	IT-EDGE-01, IT-PHA-01	**PASS**
FR-36	PHA – PharmacyService	Pharmacy	UT-PHA-01, UT-PHA-02, UT-PHA-04, IT-PHA-02, ST-PHA-01	**PASS**
FR-37	PHA – PharmacyService	Pharmacy	UT-PHA-03, IT-PHA-03, IT-PHA-04, IT-PHA-05, ST-PHA-01, AT-01	**PASS**
FR-38	IPD – AdmissionService, BedService	Beds, Admission	IT-IPD-01, ST-IPD-01, AT-02	**PASS**
FR-39	IPD – AdmissionService, BedService	Beds, Admission	IT-EDGE-08, IT-EDGE-14, IT-IPD-02, IT-IPD-03, ST-IPD-01, AT-02	**PASS**
FR-40	IPD – AdmissionService, BedService	Beds, Admission	UT-IPD-05, IT-IPD-04, ST-IPD-01	**PASS**
FR-41	IPD – AdmissionService, BedService	Beds, Admission	UT-IPD-01, UT-IPD-02, IT-IPD-05, ST-IPD-01, AT-02	**PASS**
FR-42	IPD – AdmissionService, BedService	Beds, Admission	UT-IPD-03, UT-IPD-04, UT-IPD-05, UT-IPD-06, IT-IPD-06, AT-02	**PASS**
FR-43	IPD – AdmissionService, BedService	Beds, Admission	UT-IPD-04, UT-IPD-07, IT-EDGE-15, IT-IPD-07, IT-IPD-08, IT-IPD-09, AT-02	**PASS**
FR-44	BIL – BillingService	Billing, Invoice	UT-BIL-01, UT-BIL-07, UT-BIL-08, IT-BIL-01, IT-BIL-08, IT-EDGE-13, IT-EDGE-14, IT-EDGE-15, ST-BIL-01, AT-01, AT-02	**PASS**
FR-45	BIL – BillingService	Billing, Invoice	UT-BIL-01, UT-BIL-02, IT-BIL-02	**PASS**
FR-46	BIL – BillingService	Billing, Invoice	UT-BIL-03, UT-BIL-04, UT-BIL-05, UT-BIL-06, IT-BIL-03, IT-BIL-04, IT-BIL-05, IT-EDGE-13, ST-BIL-01, ST-UI-04, AT-01, AT-02	**PASS**
FR-47	BIL – BillingService	Billing, Invoice	UT-BIL-07, IT-BIL-06, IT-BIL-07, IT-EDGE-14, ST-BIL-01	**PASS**
FR-48	RPT – ReportService	Dashboard, Reports	IT-RPT-01, ST-RPT-01, ST-UI-01	**PASS**
FR-49	RPT – ReportService	Dashboard, Reports	IT-EDGE-02, IT-EDGE-03, IT-RPT-02, ST-RPT-01	**PASS**
FR-50	AUD – AuditService	Audit Trail	IT-AUD-01, IT-AUD-02, IT-AUD-03, IT-EDGE-02, IT-EDGE-06, IT-QUE-04, ST-RPT-01, AT-05	**PASS**
NFR-01	Performance	—	IT-NFR-03	**PASS**

Requirement	Design module / service	UI screen	Verified by test cases	Status
NFR-02	Security	—	IT-NFR-01, IT-NFR-02, IT-NFR-04	**PASS**
NFR-03	Security / Privacy	—	IT-AUD-01, AT-03	**PASS**
NFR-04	Reliability / Integrity	—	IT-PHA-03, ST-UI-04	**PASS**
NFR-05	Usability	—	ST-UI-01, ST-UI-02, AT-04	**PASS**
NFR-06	Maintainability	—	Verified by measurement: layered structure + domain-rule line coverage 99.2% (Part D, 10.2)	**PASS**
NFR-07	Portability	—	Tested on Linux + Chromium only; other operating systems and browsers not yet verified (Part C, 2.2)	**PASS (PARTIAL)**
NFR-08	Auditability	—	AT-05	**PASS**
NFR-09	Availability	—	Not tested – production design target (Part C, 2.2)	**NOT TESTED**
NFR-10	Scalability	—	Not tested – production design target (Part C, 2.2)	**NOT TESTED**

User requirements are traced to functional requirements in Part A, Section 7, and validated by the acceptance tests listed in Part C, Section 7.

APPENDIX B – RUNNING THE PROTOTYPE

Requirements: Node.js 20 or later (includes npm). No database server is needed – SQLite is embedded and the demo data is generated automatically on first start.

Table 4: Project commands

Command	Purpose
`npm install`	Install dependencies (once)
`npm run dev`	Start the API (port 4000) and the web app (http://localhost:5173) with live reload
`npm start`	Build the web app and serve everything from http://localhost:4000
`npm run seed`	Reset the database and regenerate demo data relative to today
`npm test`	Run unit + integration tests (Vitest)
`npm run test:coverage`	Same, with code coverage
`npx playwright install chromium` then `npm run test:e2e`	Run system and acceptance tests in a real browser
`npm run report`	Regenerate this report (DOCX + PDF) from the specification, diagrams and latest test results

Table 5: Demo accounts (password: Mediflow@123)

Username	Role	Try this
admin	Administrator	Dashboard, Reports, Users & Roles, Audit Trail, discharge override
reception	Receptionist	Register patient, book appointment, check in, queue board, admit patient on the bed board
dr.sharma	Doctor	Queue → Call next patient → consultation, prescription, lab order, complete
lab	Lab Technician	Laboratory worklist → collect sample → enter result
pharmacy	Pharmacist	Dispense prescriptions, stock alerts, restock
billing	Billing Staff	Generate invoice from “To bill”, record payment, print receipt

Project structure: `server/` (Express API: routes, middleware, services, domain rules, database), `client/` (React app), `tests/` (unit, integration, e2e), `docs/spec/` (requirements, defects), `docs/uml/` (PlantUML sources and images), `docs/test-results/` (raw results), `docs/report/` (report generator).